

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN**



KHOÁ LUẬN TỐT NGHIỆP

ĐỀ TÀI:

**"THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHÂN LOẠI RÁC TỰ ĐỘNG
ỨNG DỤNG YOLOV11, RASPBERRY PI VÀ ESP32"**

GVHD:	PGS.TS Phạm Hồng Liên
SVTH:	Huỳnh Tấn Đạt
MSSV:	22161232
SVTH:	Lê Mỹ Hoàng
MSSV:	22161257
Ngành:	Công nghệ Kỹ thuật Điện tử Viễn thông
Khóa:	2022 – 2026

Tp. Hồ Chí Minh, tháng 7 năm 2026

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN**



KHOÁ LUẬN TỐT NGHIỆP

ĐỀ TÀI:

**"THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHÂN LOẠI RÁC TỰ ĐỘNG
ỨNG DỤNG YOLOV11, RASPBERRY PI VÀ ESP32"**

GVHD:	PGS.TS Phạm Hồng Liên
SVTH:	Huỳnh Tấn Đạt
MSSV:	22161232
SVTH:	Lê Mỹ Hoàng
MSSV:	22161257
Ngành:	Công nghệ Kỹ thuật Điện tử Viễn thông
Khóa:	2022 – 2026

Tp. Hồ Chí Minh, tháng 7 năm 2026

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Huỳnh Tấn Đạt

MSSV: 22161232

Tel: 0939190513

Email: 22161232@student.hcmute.edu.vn

Họ và tên: Lê Mỹ Hoàng

MSSV: 22161257

Tel: 0337895407

Email: 22161257@student.hcmute.edu.vn

2. Thông tin đề tài

Tên của đề tài: “Thiết kế và xây dựng hệ thống phân loại rác tự động ứng dụng YOLOv11, Raspberry Pi và ESP32”

Mục đích của đề tài: Giải quyết bài toán môi trường đô thị, đồng thời phù hợp với xu hướng ứng dụng trí tuệ nhân tạo và robot vào cuộc sống

Đồ án tốt nghiệp được thực hiện tại: Bộ môn Điện Tử Viễn Thông, Khoa Điện - Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Thành Phố Hồ Chí Minh.

Thời gian thực hiện: Từ ngày 1/3/2026 đến 16/6/2026

3. Các nhiệm vụ cụ thể của đề tài

- Khảo sát bài toán phân loại rác tại bãi tập kết sau thu gom; tìm hiểu các công nghệ liên quan gồm thị giác máy tính, mô hình YOLOv11, Raspberry Pi, ESP32 DevKit V1 và Firebase Realtime Database.
- Thu thập, gán nhãn và xây dựng bộ dữ liệu ảnh cho ba nhóm rác gồm rác hữu cơ, rác nhựa và rác kim loại; tiến hành huấn luyện và đánh giá mô hình nhận diện YOLOv11.
- Chuyển đổi mô hình sau huấn luyện sang định dạng NCNN và triển khai trên Raspberry Pi nhằm phục vụ quá trình nhận diện rác từ camera trong mô hình băng tải thực nghiệm.
- Thiết kế và thi công mô hình phần cứng gồm băng tải, camera, cảm biến entry/exit, Raspberry Pi, ESP32 DevKit V1 và cơ cấu servo phân loại.
- Xây dựng chương trình xử lý trên Raspberry Pi và firmware cho ESP32 DevKit V1; thực hiện giao tiếp UART để đồng bộ quá trình nhận diện, điều khiển băng tải và servo phân loại.
- Xây dựng logic xử lý nhận diện bằng cơ chế detection window, voting và debounce cảm biến nhằm nâng cao độ ổn định khi hệ thống vận hành.
- Đồng bộ trạng thái hoạt động và kết quả phân loại lên Firebase Realtime Database; xây dựng giao diện giám sát để hiển thị số lượng từng nhóm rác và trạng thái hệ thống.
- Tích hợp, kiểm thử và đánh giá hoạt động của hệ thống thông qua các chỉ tiêu như kết quả nhận diện, tốc độ xử lý, khả năng điều khiển phân loại và độ ổn định trong quá trình vận hành.

4. Lời cam đoan của sinh viên

Chúng tôi Huỳnh Tấn Đạt và Lê Mỹ Hoàng cam đoan KLTN là công trình nghiên cứu của bản thân chúng tôi dưới sự hướng dẫn của PGS.TS Phạm Hồng Liên.

Các kết quả công bố trong KLTN là trung thực và không sao chép từ bất kỳ công trình nào khác.

TP.HCM, ngày 16 tháng 6 năm 2026

SV thực hiện đề án

Huỳnh Tấn Đạt – Lê Mỹ Hoàng

Giáo viên hướng dẫn xác nhận về mức độ hoàn thành và cho phép được bảo vệ:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Xác nhận của Bộ Môn

Tp.HCM, ngày 18 tháng 6 năm 2026

Giáo viên hướng dẫn

(Ký ghi rõ họ tên và học hàm học vị)

PGS.TS Phạm Hồng Liên

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHÂN LOẠI RÁC TỰ ĐỘNG ỨNG DỤNG YOLOV11, RASPBERRY PI VÀ ESP32.

Sinh viên: + Huỳnh Tấn Đạt

MSSV: 22161232

Sinh viên: + Lê Mỹ Hoàng

MSSV: 22161257

Hướng dẫn: PGS. TS. PHẠM HỒNG LIÊN

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Các tác giả đặt vấn đề cho khóa luận tốt nghiệp một cách rõ ràng, có mục tiêu cụ thể. Đề tài có nhiều tính mới, tính kế thừa và có khả năng ứng dụng rất cao vào thực tế.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp thực hiện đề tài hợp lý và tin cậy dựa trên cơ sở lý thuyết có phân tích và đánh giá từ đó đưa ra mô hình thiết kế phần cứng và các giải thuật phần mềm phù hợp. Đề tài thiết kế một hệ thống tích hợp hoàn chỉnh, có khả năng vừa nhận diện rác bằng AI, vừa điều khiển cơ cấu phân loại thông qua nền tảng nhúng như Raspberry Pi và ESP32 Devkit V1.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Kết quả thực hiện đề tài phù hợp với mục tiêu đề tài; thiết kế hợp lý; thi công phần cứng và phần mềm khá tối ưu. Việc kết hợp giữa camera, mô hình AI, băng tải và hệ thống điều khiển nhúng có thể tạo ra một hệ thống phân loại rác tự động hoạt động theo thời gian thực, giúp nâng cao hiệu quả phân loại, giảm sự phụ thuộc vào lao động thủ công và góp phần bảo vệ môi trường.

4. Kết luận và đề xuất:

Kết luận của khóa luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong của khóa luận nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Tác giả thể hiện các kỹ năng giao tiếp và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài khá tốt.

7. Tài liệu trích dẫn

Tài liệu trích dẫn trung thực và phù hợp với đề tài; trích dẫn theo đúng chỉ dẫn APA khá phong phú.

8 . Đánh giá về sự trùng lặp của đề tài

Đề tài không trùng lặp với các đề tài khác.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Cần sửa các lỗi .

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần, thái độ học tập, nghiên cứu của sinh viên rất nghiêm túc, thường xuyên báo cáo tiến độ với giáo viên. Sinh viên luôn độc lập và tìm hiểu các tính mới của đề tài.

Kết luận : Bảo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 18 tháng 06. năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)



PGS. TS. PHẠM HỒNG LIÊN

LỜI CẢM ƠN

Trong quá trình thực hiện khóa luận tốt nghiệp, chúng tôi đã nhận được nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ quý thầy cô, gia đình và bạn bè. Trước hết, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn, người đã tận tình định hướng đề tài, góp ý phương pháp nghiên cứu, hỗ trợ em tháo gỡ các vấn đề kỹ thuật trong quá trình thiết kế hệ thống phân loại rác tự động ứng dụng YOLOv11, Raspberry Pi và ESP32.

Chúng tôi xin chân thành cảm ơn quý thầy cô bộ môn Điện tử - Thông tin tiền thân là bộ môn (Kỹ thuật Máy tính – Viễn thông), khoa Điện – Điện tử đã truyền đạt kiến thức nền tảng về hệ thống nhúng, xử lý tín hiệu, truyền thông dữ liệu và lập trình điều khiển. Những kiến thức này là cơ sở quan trọng giúp chúng tôi triển khai từ mô hình trí tuệ nhân tạo đến phần cứng chấp hành và hệ thống đồng bộ dữ liệu.

Chúng tôi cũng xin cảm ơn gia đình đã luôn động viên, tạo điều kiện về thời gian và tinh thần, cảm ơn các bạn cùng lớp đã hỗ trợ trong quá trình thu thập dữ liệu, thử nghiệm mô hình và kiểm tra hệ thống. Do giới hạn về thời gian và kinh nghiệm, báo cáo khó tránh khỏi thiếu sót, chúng tôi kính mong nhận được góp ý của quý thầy cô để đề tài được hoàn thiện hơn.

TÓM TẮT

Phân loại rác tại bãi tập kết sau thu gom là một vấn đề cấp thiết trong bối cảnh lượng chất thải rắn sinh hoạt tại Việt Nam ngày càng gia tăng, trong khi phương pháp phân loại thủ công còn tốn nhân lực, thiếu ổn định và tiềm ẩn nguy cơ ảnh hưởng đến sức khỏe người lao động. Khóa luận này trình bày quá trình thiết kế và xây dựng hệ thống phân loại rác tự động cho ba nhóm rác gồm: hữu cơ, nhựa và kim loại. Hệ thống sử dụng camera đặt trên băng tải để thu nhận hình ảnh, mô hình YOLOv11 để phát hiện đối tượng trên Raspberry Pi, ESP32 DEVKIT-V1 để điều khiển cơ cấu servo, kết hợp với cảm biến entry/exit nhằm đồng bộ thời điểm nhận diện và phân loại.

Mô hình YOLOv11 được huấn luyện với cấu hình yolo11s.pt trong 150 epoch, batch 16 và kích thước ảnh huấn luyện 640×640 . Thư mục kết quả huấn luyện mới không lưu kèm số ảnh train/val/test, tuy nhiên file labels.jpg ghi nhận 946 nhãn đối tượng gồm 294 metal, 359 organic và 293 plastic. Kết quả tốt nhất tại epoch 140 đạt Precision = 0,9497, Recall = 0,9631, mAP50 = 0,9783 và mAP50-95 = 0,7590. Sau quá trình huấn luyện, mô hình được chuyển đổi sang định dạng NCNN với input 416×416 , batch 1 và stride 32 để triển khai trên CPU ARM của Raspberry Pi, phục vụ yêu cầu suy luận thời gian thực trên thiết bị nhúng.

Bên cạnh mô hình nhận diện, đề tài xây dựng logic xử lý thời gian thực gồm detection window, voting, debounce cảm biến, ngưỡng confidence và dominant ratio nhằm giảm sai lệch khi vật thể di chuyển liên tục trên băng tải. Raspberry Pi gửi lệnh điều khiển qua UART dưới dạng SORT:<class>, BIN:<class>, CONVEYOR:START/STOP đến ESP32 DevKit V1; ESP32 DevKit V1 phản hồi các trạng thái như ACK, DONE, ERROR, SENSOR và STATUS. Dữ liệu trạng thái, bộ đếm và lần nhận diện gần nhất được đồng bộ lên Firebase Realtime Database để phục vụ việc giám sát trên dashboard android.

Kết quả thực nghiệm ban đầu cho thấy hệ thống có khả năng phân loại rác theo thời gian thực trong điều kiện băng tải trong nhà, camera cố định và ba nhóm rác mục tiêu. Đề tài góp phần minh họa khả năng ứng dụng trí tuệ nhân tạo, hệ thống nhúng và cơ sở dữ liệu thời gian thực vào bài toán phân loại rác tự động tại bãi tập kết sau thu gom.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC.....	iv
DANH MỤC HÌNH	x
DANH MỤC BẢNG.....	xii
CÁC TỪ VIẾT TẮT	xiii
CHƯƠNG 1: TỔNG QUAN.....	1
1.1 ĐẶT VẤN ĐỀ.....	1
1.1.1 Thực trạng rác thải và phân loại rác tại bãi tập kết sau thu gom.	2
1.1.2 Hạn chế của phân loại rác thủ công.	2
1.1.3 Tiềm năng ứng dụng AI trong phân loại rác.....	3
1.2 TỔNG QUAN CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN TRONG VÀ NGOÀI NƯỚC.....	3
1.2.1 Nghiên cứu trong nước.	3
1.2.2 Nghiên cứu ngoài nước.....	3
1.2.3 Phân tích và nhận xét chung.	4
1.3 MỤC TIÊU ĐỀ TÀI.....	6
1.4 PHẠM VI NGHIÊN CỨU VÀ GIỚI HẠN ĐỀ TÀI	7
1.5 PHƯƠNG PHÁP NGHIÊN CỨU.....	8
1.6 KẾT LUẬN CHƯƠNG.....	9

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	11
2.1 LÝ THUYẾT VỀ THỊ GIÁC MÁY TÍNH.....	11
2.1.1 Tổng quan về Thị giác máy tính (Computer Vision).....	11
2.1.2 Phát hiện đối tượng (Object Detection)	12
2.1.3 Các chỉ số đánh giá mô hình.	13
2.2 KIẾN TRÚC YOLO VÀ YOLOv11	14
2.2.1 Lịch sử phát triển của YOLO.....	14
2.2.2 Kiến trúc YOLOv11	15
2.2.3 Định dạng NCNN và tối ưu cho thiết bị nhúng	16
2.3 NỀN TẢNG PHẦN CỨNG	17
2.3.1 Raspberry Pi – bộ xử lý trung tâm	17
2.3.2 ESP32 DevKit V1 và giao tiếp UART	18
2.4 FIREBASE REALTIME DATABASE	20
2.5 KẾT LUẬN CHƯƠNG.....	20
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	22
3.1 YÊU CẦU VÀ ĐẶC TẢ HỆ THỐNG	22
3.1.1 Yêu cầu chức năng.....	22
3.2 Kiến trúc tổng thể hệ thống.....	23
3.2.1 Sơ đồ khối tổng quan	23
3.2.2 Luồng dữ liệu và luồng điều khiển	27
3.3 THIẾT KẾ PHẦN CỨNG	27
3.3.1 Sơ đồ kết nối phần cứng tổng thể	27

3.3.2	Thiết kế cụm xử lý trung tâm (Raspberry Pi + Camera)	29
3.3.3	Thiết kế cụm chấp hành (ESP32 DEVKIT V1, Servo và Cảm biến)	31
3.3.4	Thiết kế cơ khí – khung phân loại và băng tải	32
3.4	THIẾT KẾ PHẦN MỀM	33
3.4.1	Kiến trúc phần mềm trên Raspberry Pi.....	34
3.4.2	Chức năng từng module Python.....	36
3.5	THIẾT KẾ LOGIC NHẬN DIỆN THỜI GIAN THỰC	38
3.5.1	Vấn đề nhận diện sai khi vật di chuyển liên tục	38
3.5.2	Cơ chế Detection Window và Voting	38
3.5.3	Debounce, Confidence threshold và Dominant ratio	39
3.5.4	Cảm biến entry/exit và sơ đồ trạng thái	39
3.6	THIẾT KẾ CƠ SỞ DỮ LIỆU FIREBASE	42
3.6.1	Cấu trúc cây dữ liệu	42
3.6.2	Thiết kế schema lưu thông kê phân loại	43
3.7	THIẾT KẾ DASHBOARD VÀ ỨNG DỤNG ANDROID	43
3.7.1	Giao diện dashboard thống kê.....	43
3.7.2	Phác thảo ứng dụng Android	45
3.8	KẾT LUẬN CHƯƠNG.....	50
CHƯƠNG 4: THI CÔNG, LẬP TRÌNH VÀ HUẤN LUYỆN MÔ HÌNH		51
4.1	THI CÔNG PHẦN CỨNG	51
4.1.1	Lắp ráp khung cơ khí và băng tải.....	51
4.1.2	Lắp đặt camera và các cảm biến	52

4.1.3	Đầu nối ESP32 DEVKIT-V1, servo và nguồn điện	54
4.2	THU THẬP VÀ TIỀN XỬ LÝ DỮ LIỆU HUẤN LUYỆN	55
4.2.1	Thu thập ảnh các nhóm rác	55
4.2.2	Gán nhãn dữ liệu	57
4.2.3	Data augmentation và phân chia tập dữ liệu	59
4.3	HUẤN LUYỆN MÔ HÌNH YOLOV11	61
4.3.1	Cấu hình môi trường và siêu tham số	61
4.3.2	Quá trình huấn luyện và theo dõi	62
4.3.3	Lựa chọn checkpoint tốt nhất	64
4.4	CHUYỂN ĐỔI MÔ HÌNH SANG ĐỊNH DẠNG NCNN	64
4.4.1	Quy trình export từ PyTorch sang NCNN	64
4.4.2	Kiểm tra tính tương thích trên Raspberry Pi	65
4.5	LẬP TRÌNH PYTHON TRÊN RASPBERRY PI	66
4.5.1	Module detect.py – nhận diện đối tượng	66
4.5.2	Module main.py và logic điều phối	68
4.5.3	Module state_store.py – quản lý trạng thái	69
4.5.4	Module uart_client.py – giao tiếp serial	69
4.5.5	Module firebase_sync.py – đồng bộ dữ liệu	70
4.5.6	Module config.py và app.py	71
4.6	LẬP TRÌNH FIRMWARE ESP32 DEVKIT-V1	71
4.6.1	Nhận lệnh qua UART và giải mã giao thức	71
4.6.2	Điều khiển servo theo nhóm rác	73

4.6.3	Xử lý phản hồi và trạng thái cơ cấu	74
4.7	TÍCH HỢP VÀ KIỂM THỬ TOÀN HỆ THỐNG	75
4.8	KẾT LUẬN CHƯƠNG.....	75
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ		77
5.1	KẾT QUẢ HUẤN LUYỆN MÔ HÌNH YOLOV11.....	77
5.1.1	Biểu đồ loss qua các epoch	77
5.1.2	Confusion matrix và phân tích nhầm lẫn nhãn	78
5.1.3	Đường cong F1, Precision, Recall theo confidence.....	80
5.1.4	Đường cong PR và giá trị mAP	83
5.2	KẾT QUẢ NHẬN DIỆN THỰC TẾ TRÊN RASPBERRY PI.....	84
5.2.1	Tốc độ xử lý (FPS) với mô hình NCNN.....	84
5.2.2	Độ chính xác phân loại theo từng nhóm rác	86
5.2.3	Kết quả trong điều kiện ánh sáng khác nhau	87
5.3	ĐÁNH GIÁ CƠ CHẾ XỬ LÝ LOGIC NHẬN DIỆN.....	87
5.3.1	Hiệu quả của voting và detection window	87
5.3.2	Đánh giá khi 2 vật đi liên tiếp trên băng tải.....	88
5.3.3	Hiệu quả debounce và entry/exit sensor	89
5.3.4	Kết quả giao tiếp UART và điều khiển servo	90
5.4	KẾT QUẢ ĐỒNG BỘ FIREBASE VÀ HIỂN THỊ THỐNG KÊ.....	90
5.4.1	Đánh giá tổng thể và so sánh với yêu cầu đặt ra.....	91
5.5	KẾT LUẬN CHƯƠNG.....	93
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		94

6.1 KẾT LUẬN	94
6.1.1 Những kết quả đạt được.....	94
6.1.2 Ưu điểm của hệ thống	95
6.1.3 Hạn chế còn tồn tại	95
6.2 HƯỚNG PHÁT TRIỂN	95
6.2.1 Phát triển ứng dụng Android tích hợp Firebase	95
6.2.2 Mở rộng nhận diện thêm nhóm rác	95
6.2.3 Cải thiện cơ cấu cơ khí và tốc độ xử lý.....	95
6.2.4 Ứng dụng AI phân tích xu hướng rác thải	96
6.2.5 Triển khai thực tế	96
TÀI LIỆU THAM KHẢO.....	97
PHỤ LỤC A – SOURCE CODE PYTHON	99
PHỤ LỤC B – FIRMWARE ESP32 DEVKIT-V1	183
PHỤ LỤC C – CẤU HÌNH HUẤN LUYỆN (DATA.YAML).....	203
PHỤ LỤC D – CẤU TRÚC FIREBASE JSON MẪU	204
PHỤ LỤC E – CÁC LƯU ĐỒ THUẬT TOÁN CHI TIẾT	205

DANH MỤC HÌNH

Hình 2. 1 Quy trình xử lý ảnh từ camera đến hiển thị và đầu ra thị giác máy tính.	11
Hình 2. 2 Minh họa sự khác nhau giữa phân loại ảnh, phát hiện đối tượng và phân đoạn ảnh trong thị giác máy tính	13
Hình 2. 3 Kiến trúc tổng quát của mô hình YOLOv11 sử dụng trong đề tài	16
Hình 2. 4 Quy trình chuyển đổi mô hình YOLOv11 sang định dạng NCNN	17
Hình 2. 5 Sơ đồ giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1	20
Hình 3. 1 Sơ đồ khối tổng thể hệ thống	24
Hình 3. 2 Lưu đồ thuật toán chính	25
Hình 3. 3 Flowchart luồng xử lý tổng thể	27
Hình 3. 4 Sơ đồ kết nối phần cứng	28
Hình 3. 5 Sơ đồ thiết kế trên protues	29
Hình 3. 6 Phần cứng 3D	29
Hình 3. 7 Sơ đồ cụm Raspberry Pi và camera	31
Hình 3. 8 Mô hình cơ khí hệ thống	33
Hình 3. 9 Sơ đồ phụ thuộc giữa các module	36
Hình 3. 10 Minh họa cơ chế voting trong detection window	39
Hình 3. 11 Sơ đồ trạng thái hệ thống nhận diện	41
Hình 3. 12 Dashboard thống kê	44
Hình 3. 13 Giao diện thống kê phân loại của ứng dụng Android	46
Hình 3. 14 Giao diện phân tích bằng AI và cảnh báo	47
Hình 3. 15 Giao diện giám sát trạng thái phần cứng	49
Hình 4. 1 Hình ảnh thực tế khung cơ khí	52
Hình 4. 2 Lắp đặt camera và cảm biến	53
Hình 4. 3 Sơ đồ đấu nối thực tế	54
Hình 4. 4 Ảnh mẫu từng nhóm rác	57
Hình 4. 5 Giao diện gán nhãn	58
Hình 4. 6 Phân bố nhãn trong tập dữ liệu	60
Hình 4. 7 Ảnh batch huấn luyện mẫu sau khi áp dụng augmentation	63
Hình 5. 1 Biểu đồ kết quả huấn luyện	77
Hình 5. 2 Confusion matrix	79
Hình 5. 3 Kết quả đường cong F1-Confidence	80
Hình 5. 4 Đường cong Precision-Confidence của mô hình	81
Hình 5. 5 Đường cong Recall-Confidence của mô hình	82
Hình 5. 6 Đường cong Precision-Recall của mô hình	83
Hình 5. 7 Ảnh nhận diện thực tế với bounding box	87
Hình 5. 8 Screenshot dữ liệu Firebase thực tế	91
Hình E. 1 Lưu đồ thuật toán nhận diện vật thể và voting	206
Hình E. 2 Lưu đồ giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1	207
Hình E. 3 Lưu đồ điều khiển servo phân loại	208

Hình E. 4 Lưu đồ xử lý cảm biến entry/exit	209
Hình E. 5 Lưu đồ cập nhật và hiển thị dữ liệu trên dashboard web.....	210

DANH MỤC BẢNG

Bảng 1. 1 So sánh một số hướng nghiên cứu liên quan	4
Bảng 2. 1 Các chỉ số đánh giá mô hình phát hiện đối tượng.	13
Bảng 2. 2 So sánh khái quát một số phiên bản YOLO.	15
Bảng 2. 3 Thông số Raspberry Pi dùng trong hệ thống	17
Bảng 2. 4 Thông số ESP32-DEVKIT-V1	18
Bảng 3. 1 Yêu cầu phi chức năng	23
Bảng 3. 2 Bảng kết nối phần cứng	28
Bảng 3. 3 Phân công GPIO ESP32 DEVKIT V1	32
Bảng 3. 4 Các module phần mềm trên Raspberry Pi	34
Bảng 3. 5 Bảng chuyển trạng thái	40
Bảng 4. 1 So sánh hiệu năng .pt vs NCNN	66
Bảng 4. 2 Góc servo tương ứng từng nhóm rác	74
Bảng 5. 1 Kết quả đánh giá mô hình theo từng lớp	84
Bảng 5. 2 Kết quả đo FPS	84
Bảng 5. 3 Kết quả phân loại thực tế	86
Bảng 5. 4 Ảnh hưởng của ánh sáng	87
Bảng 5. 5 So sánh kết quả có/không có voting	88
Bảng 5. 6 Kết quả khi 2 vật đi liên tiếp.....	88
Bảng 5. 7 Kết quả đo độ trễ UART và servo	90
Bảng 5. 8 Kết quả thống kê một phiên thực nghiệm.....	91
Bảng 5. 9 Đánh giá đạt yêu cầu hệ thống.....	91

CÁC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
AI	Artificial Intelligence - Trí tuệ nhân tạo
API	Application Programming Interface - Giao diện lập trình ứng dụng
CNN	Convolutional Neural Network - Mạng nơ-ron tích chập
ESP32 DevKit V1	ESP32 Development Kit Version 1 – Bộ mạch phát triển ESP32 phiên bản 1
FPS	Frames Per Second - Số khung hình mỗi giây
GPIO	General Purpose Input/Output - Chân vào/ra đa dụng
IoT	Internet of Things - Internet vạn vật
JSON	JavaScript Object Notation - Định dạng dữ liệu dạng cây
mAP	Mean Average Precision - Độ chính xác trung bình
NCNN	Neural Network Inference Framework - Tối ưu cho thiết bị nhúng
PWM	Pulse Width Modulation - Điều chế độ rộng xung
ReLU	Rectified Linear Unit - Hàm kích hoạt tuyến tính chỉnh lưu
UART	Universal Asynchronous Receiver/Transmitter - Truyền thông nối tiếp bất đồng bộ

USB	Universal Serial Bus
YOLO	You Only Look Once - Họ mô hình phát hiện đối tượng một giai đoạn
YOLOv11	Phiên bản YOLO thế hệ 11 của Ultralytics
SDK	Software Development Kit - Bộ công cụ phát triển phần mềm
SSD	Single Shot MultiBox Detector
OTA	Over-the-Air - Cập nhật qua mạng
PR curve	Precision-Recall curve - Đường cong Precision-Recall
F1	Trung bình điều hòa giữa Precision và Recall
IOU	Intersection over Union - Tỷ lệ giao trên hợp của hai bounding box
NMS	Non-Maximum Suppression - Loại bỏ hộp trùng lặp
Firebase	Nền tảng cơ sở dữ liệu thời gian thực của Google
Raspberry Pi (RPi)	Máy tính nhúng đơn bo mạch dùng làm bộ xử lý biên

CHƯƠNG 1: TỔNG QUAN

Trong bối cảnh lượng rác thải sinh hoạt ngày càng gia tăng, việc phân loại rác tại bãi tập kết có ý nghĩa quan trọng trong quá trình thu gom, tái chế và giảm tác động đến môi trường. Tuy nhiên, phương pháp phân loại thủ công vẫn còn nhiều hạn chế về tốc độ, độ ổn định và điều kiện làm việc của con người [1]. Từ thực tế đó, chương này trình bày tổng quan về vấn đề phân loại rác, các nghiên cứu liên quan trong và ngoài nước, đồng thời xác định mục tiêu, phạm vi và phương pháp thực hiện đề tài “Thiết kế và xây dựng hệ thống phân loại rác tự động ứng dụng YOLOv11, Raspberry Pi và ESP32”.

1.1 ĐẶT VẤN ĐỀ.

Rác thải rắn sinh hoạt tại Việt Nam đang tăng nhanh cùng quá trình đô thị hóa, thay đổi thói quen tiêu dùng và sự phổ biến của bao bì nhựa dùng một lần [1]. Tại nhiều địa phương, khâu phân loại vẫn chủ yếu dựa vào người lao động hoặc các điểm thu gom thủ công. Cách làm này khiến hiệu quả phân loại không ổn định, tỉ lệ tái chế chưa cao và người lao động phải tiếp xúc trực tiếp với rác ướt, kim loại sắc nhọn, vi sinh vật hoặc mùi khó chịu.

Phân loại thủ công phụ thuộc nhiều vào kinh nghiệm và trạng thái làm việc của từng người, do đó dễ sai lệch khi tốc độ băng tải cao hoặc khi rác có hình dạng, màu sắc tương tự nhau. Một hệ thống xử lý quy mô nhỏ nếu chỉ dựa vào thao tác tay sẽ khó mở rộng, khó ghi nhận thống kê theo thời gian thực và khó tích hợp với các nền tảng quản lý đô thị thông minh.

Sự phát triển của thị giác máy tính, mô hình YOLO và phần cứng Edge AI giúp việc triển khai hệ thống phân loại rác tự động trở nên khả thi với chi phí thấp. Raspberry Pi có thể chạy suy luận tại biên, ESP32 DEVKIT-V1 đảm nhiệm điều khiển chấp hành, còn Firebase cung cấp kênh lưu trữ và giám sát dữ liệu. Đây là hướng tiếp cận phù hợp với mô hình thí nghiệm, khuôn viên trường học hoặc trạm phân loại rác quy mô nhỏ.

1.1.1 Thực trạng rác thải và phân loại rác tại bãi tập kết sau thu gom.

Rác thải sinh hoạt phát sinh hằng ngày gồm nhiều nhóm vật liệu khác nhau như rác hữu cơ, nhựa, giấy, kim loại, thủy tinh và các loại rác khó tái chế [1]. Trong quá trình thu gom, rác từ các hộ gia đình, khu dân cư hoặc khu vực công cộng thường được tập trung và vận chuyển về bãi tập kết hoặc điểm trung chuyển trước khi đưa đến nơi xử lý tiếp theo [1]. Do rác thường được thu gom lẫn, công đoạn phân loại tại bãi tập kết có vai trò quan trọng trong việc tách riêng các nhóm vật liệu có khả năng tái chế hoặc cần được xử lý phù hợp.

Trong thực tế, quá trình phân loại rác sau thu gom vẫn còn gặp nhiều khó khăn do thành phần rác không đồng nhất, vật thể dễ bị bám bẩn, biến dạng hoặc lẫn với nhau. Đặc biệt, khi rác hữu cơ có độ ẩm cao trộn lẫn với nhựa và kim loại, chất lượng vật liệu có thể tái chế bị giảm, đồng thời gây khó khăn cho quá trình xử lý tiếp theo. Nếu việc phân loại chủ yếu dựa vào thao tác thủ công, hiệu quả xử lý còn phụ thuộc nhiều vào người lao động và điều kiện làm việc tại bãi tập kết.

Trong phạm vi đề tài, ba nhóm rác được lựa chọn để nghiên cứu gồm rác hữu cơ, rác nhựa và rác kim loại. Rác hữu cơ là nhóm thường gặp trong chất thải sinh hoạt và có đặc điểm dễ phân hủy, độ ẩm cao. Rác nhựa có khả năng tái chế nhưng dễ bị lẫn tạp chất sau quá trình thu gom. Rác kim loại có giá trị thu hồi, tuy nhiên hình dạng và bề mặt phản xạ ánh sáng có thể gây khó khăn trong quá trình nhận diện bằng hình ảnh. Việc lựa chọn ba nhóm rác này phù hợp với mô hình thực nghiệm và có ý nghĩa trong việc xây dựng hệ thống phân loại rác tự động tại bãi tập kết sau thu gom.

1.1.2 Hạn chế của phân loại rác thủ công.

Phân loại thủ công bị giới hạn bởi tốc độ thao tác của con người, sự mệt mỏi khi làm việc lâu dài và điều kiện môi trường không bảo đảm [1]. Độ chính xác để

thay đổi theo ánh sáng, trạng thái vật thể và cách người vận hành nhận biết vật liệu. Ngoài ra, khi lượng rác tăng, phương pháp thủ công khó mở rộng vì cần nhiều lao động, nhiều không gian và quy trình giám sát phức tạp.

1.1.3 Tiềm năng ứng dụng AI trong phân loại rác.

AI thị giác máy tính có khả năng học đặc trưng hình dạng, màu sắc và ngữ cảnh của vật thể từ dữ liệu ảnh. Các mô hình YOLO hiện đại cho phép phát hiện đối tượng trong thời gian thực [2], [3], phù hợp với băng tải. Khi kết hợp với NCNN và phần cứng nhúng, mô hình có thể chạy cục bộ trên Raspberry Pi mà không cần gửi ảnh lên cloud, giảm độ trễ và bảo vệ dữ liệu [4], [5].

1.2 TỔNG QUAN CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN TRONG VÀ NGOÀI NƯỚC.

1.2.1 Nghiên cứu trong nước.

Trong nước, Tran và Nguyen đề xuất hệ thống phân loại rác nhựa sử dụng mô hình YOLOv5 kết hợp robot Delta và băng tải. Hệ thống thực hiện phát hiện vật thể bằng bounding box và tiến hành thử nghiệm với hơn 1000 mẫu trong hai điều kiện ánh sáng, đạt Precision 96% và Recall 97% ở trường hợp tốt nhất [6].

Một hướng nghiên cứu khác sử dụng camera kết hợp mô hình CNN để phân loại ảnh rác tĩnh. Cách tiếp cận này cho kết quả khá tốt trong môi trường ảnh chụp riêng lẻ, nhưng thường chưa có cơ chế xử lý thời gian thực trên băng tải, chưa có entry/exit sensor và chưa tích hợp điều khiển servo qua UART. Trong lĩnh vực trí tuệ nhân tạo, tác giả N. M. Chinh (2023) [7] đã thực hiện nghiên cứu phát hiện đối tượng rác thải thông qua hình ảnh và video bằng việc ứng dụng thuật toán YOLOv8.

1.2.2 Nghiên cứu ngoài nước.

Ở nước ngoài, Yang và Thung xây dựng bộ dữ liệu TrashNet gồm sáu lớp: glass, paper, metal, plastic, cardboard và trash, phục vụ bài toán phân loại ảnh rác.

Saputra và Kusrini ứng dụng YOLOv4 và YOLOv4-tiny cho bài toán phát hiện bốn nhóm rác, trong đó YOLOv4 đạt mAP 89,59% trong thử nghiệm của nghiên cứu [8].

1.2.3 Phân tích và nhận xét chung.

Bảng 1. 1 So sánh một số hướng nghiên cứu liên quan

Sản phẩm /Năm	Phương pháp	Phần cứng	Số lớp	mAP/Accuracy	Hạn chế
Nghiên cứu thùng rác thông minh IoT, 2022	Cảm biến mức đầy, truyền dữ liệu qua Wi-Fi	ESP32/NodeMCU	Không phân lớp	Không áp dụng	Chỉ giám sát mức đầy, chưa nhận diện và phân loại rác bằng hình ảnh
Nghiên cứu phân loại rác bằng CNN, 2023	CNN phân loại ảnh tĩnh	PC/Raspberry Pi	3–5	Khoảng 85–90%	Chỉ phân loại ảnh đơn, chưa tích hợp băng tải và cơ cấu điều khiển phân loại tự động

Yang & Thung, TrashNet, 2016 [8]	CNN classification	PC/GPU	6	Accuracy phụ thuộc mô hình	Chỉ phân loại ảnh, không phát hiện vị trí vật thể và chưa tích hợp hệ thống cơ khí
Các nghiên cứu phân loại rác dùng YOLO, 2021–2024 [9]	YOLO phát hiện vật thể thời gian thực	PC/Jetson/Raspberry Pi	3–6	mAP thường >80%	Tập trung vào nhận diện, ít mô tả đầy đủ thuật toán điều khiển, đồng bộ cảm biến và cơ cấu phân loại
Đề tài này, 2026	YOLOv11 + voting + UART + điều khiển servo	Raspberry Pi + ESP32 DEVKIT-V1+ camera + cảm biến entry/exit	3	mAP50 = 0,9783	Phạm vi thử nghiệm còn trong mô hình phòng thí nghiệm,

					chưa đánh giá dài hạn trong môi trường thực tế
--	--	--	--	--	---

Qua Bảng 1.1 có thể thấy các nghiên cứu về thùng rác thông minh IoT chủ yếu tập trung vào giám sát mức đầy và truyền dữ liệu, chưa giải quyết bài toán nhận diện loại rác bằng hình ảnh. Các nghiên cứu sử dụng CNN có khả năng phân loại rác dựa trên ảnh, tuy nhiên phần lớn chỉ xử lý ảnh tĩnh, chưa tích hợp với băng tải, cảm biến và cơ cấu chấp hành để tạo thành một hệ thống phân loại tự động hoàn chỉnh. Trong khi đó, các nghiên cứu sử dụng YOLO cho kết quả tốt trong phát hiện vật thể thời gian thực, nhưng thường tập trung vào mô hình nhận diện, ít trình bày chi tiết quá trình đồng bộ giữa phát hiện hình ảnh, cảm biến vị trí và điều khiển cơ cấu phân loại.

Khoảng trống nghiên cứu mà đề tài tập trung giải quyết là xây dựng một hệ thống phân loại rác tự động hoàn chỉnh, kết hợp mô hình YOLOv11 trên Raspberry Pi, thuật toán voting để ổn định kết quả nhận diện, truyền dữ liệu UART đến ESP32 DEVKIT-V1 và điều khiển servo theo tín hiệu cảm biến entry/exit. Hệ thống hướng đến phân loại ba nhóm rác gồm hữu cơ, nhựa và kim loại trong điều kiện mô hình băng tải thực nghiệm.

1.3 MỤC TIÊU ĐỀ TÀI.

Mục tiêu tổng quát của đề tài là xây dựng hệ thống phân loại rác tự động ứng dụng thị giác máy tính và điều khiển nhúng, có khả năng nhận diện và phân loại rác thành ba nhóm: rác hữu cơ, rác nhựa và rác kim loại. Hệ thống hướng đến việc giảm thao tác phân loại thủ công, nâng cao tính tự động hóa và hỗ trợ giám sát dữ liệu phân loại thông qua nền tảng Firebase.

Các mục tiêu cụ thể của đề tài bao gồm:

1. Xây dựng mô hình nhận diện rác sử dụng YOLOv11 để phân loại ba nhóm rác gồm: hữu cơ, nhựa và kim loại.
2. Huấn luyện, đánh giá và tối ưu mô hình nhận diện nhằm đạt độ chính xác phù hợp với yêu cầu thực nghiệm của hệ thống.
3. Chuyển đổi mô hình sang định dạng NCNN để triển khai trên Raspberry Pi, hướng đến khả năng suy luận thời gian thực.
4. Thiết kế logic xử lý nhận diện ổn định bằng các cơ chế như voting, debounce và dominant ratio nhằm hạn chế sai lệch khi nhiều vật thể đi liên tục trên băng tải.
5. Xây dựng cơ chế giao tiếp giữa Raspberry Pi và ESP32 thông qua UART để đồng bộ quá trình nhận diện, điều khiển băng tải, cảm biến và cơ cấu servo phân loại.
6. Thiết kế và vận hành cơ cấu phân loại rác bằng servo, đảm bảo rác sau khi nhận diện được đưa về đúng ngăn tương ứng.
7. Đồng bộ dữ liệu kết quả phân loại lên Firebase Realtime Database để phục vụ việc lưu trữ, thống kê và giám sát hệ thống.

1.4 PHẠM VI NGHIÊN CỨU VÀ GIỚI HẠN ĐỀ TÀI .

Đề tài tập trung nghiên cứu, thiết kế và xây dựng hệ thống phân loại rác tự động trong phạm vi mô hình thực nghiệm. Hệ thống thực hiện nhận diện và phân loại ba nhóm rác chính gồm: rác hữu cơ, rác nhựa và rác kim loại. Quá trình thử nghiệm được thực hiện trong môi trường băng tải quy mô nhỏ, với camera được lắp cố định và điều kiện ánh sáng tương đối ổn định.

Về phạm vi nhận diện, hệ thống ưu tiên xử lý từng vật thể rác đi qua vùng quan sát của camera tại một thời điểm. Trường hợp nhiều vật thể đặt quá gần nhau hoặc che khuất lẫn nhau có thể ảnh hưởng đến kết quả nhận diện và phân loại. Để hạn chế sai lệch trong quá trình vận hành, hệ thống sử dụng cảm biến entry/exit,

hàng đợi xử lý và trạng thái cơ cấu phân loại nhằm đồng bộ giữa quá trình nhận diện và điều khiển.

Về phạm vi triển khai, mô hình nhận diện được chạy trên Raspberry Pi, kết hợp với ESP32 để điều khiển cảm biến, băng tải và servo phân loại. Dữ liệu kết quả phân loại được đồng bộ lên Firebase Realtime Database nhằm phục vụ lưu trữ và thống kê. Trong khuôn khổ đề tài, ứng dụng Android và dashboard giám sát mới dừng ở mức xây dựng giao diện, cấu trúc dữ liệu và định hướng tích hợp, chưa triển khai đầy đủ toàn bộ chức năng phân tích, điều khiển qua app và tích hợp phân tích đầy đủ, chính xác. Đồng thời, do thời gian có hạn và kinh nghiệm còn hạn chế, đặc biệt trong lĩnh vực cơ khí chúng tôi chưa có kinh nghiệm thiết kế nên mô hình còn thô sơ, đơn giản.

1.5 PHƯƠNG PHÁP NGHIÊN CỨU

Để thực hiện đề tài, khóa luận sử dụng kết hợp giữa phương pháp nghiên cứu lý thuyết, phương pháp thực nghiệm và phương pháp đánh giá hệ thống. Cụ thể như sau:

1. Phương pháp nghiên cứu tài liệu:

Tìm hiểu các tài liệu liên quan đến phân loại rác tự động, thị giác máy tính, mô hình YOLOv11, định dạng NCNN, Raspberry Pi, ESP32 DEVKIT-V1, cảm biến, servo và Firebase Realtime Database.

2. Phương pháp thiết kế hệ thống:

Phân tích yêu cầu bài toán và xây dựng kiến trúc tổng thể của hệ thống, bao gồm khối thu nhận hình ảnh, khối xử lý nhận diện, khối điều khiển cơ cấu phân loại và khối lưu trữ dữ liệu.

3. Phương pháp xây dựng dữ liệu:

Thu thập hình ảnh rác thuộc ba nhóm gồm hữu cơ, nhựa và kim loại; tiến hành gán nhãn dữ liệu theo định dạng YOLO và chia bộ dữ liệu thành các tập huấn luyện, kiểm định và kiểm thử.

4. Phương pháp huấn luyện và đánh giá mô hình:

Huấn luyện mô hình YOLOv11 trên bộ dữ liệu đã xây dựng, sau đó đánh giá kết quả thông qua các chỉ số như độ chính xác, độ bao phủ, mAP và ma trận nhầm lẫn để lựa chọn mô hình phù hợp.

5. Phương pháp triển khai trên thiết bị nhúng:

Chuyển đổi mô hình nhận diện sang định dạng NCNN để triển khai trên Raspberry Pi. Đồng thời xây dựng chương trình xử lý nhận diện bằng Python và chương trình điều khiển cảm biến, băng tải, servo trên ESP32 DEVKIT V1.

6. Phương pháp tích hợp và kiểm thử hệ thống:

Tích hợp các thành phần phần cứng và phần mềm, kiểm thử quá trình giao tiếp UART giữa Raspberry Pi và ESP32 DEVKIT-V1 kiểm tra hoạt động của cảm biến entry/exit, servo phân loại và quá trình đồng bộ dữ liệu lên Firebase.

7. Phương pháp đánh giá kết quả thực nghiệm:

Đánh giá hệ thống dựa trên khả năng nhận diện, tốc độ xử lý, độ ổn định khi vận hành, khả năng phân loại đúng từng nhóm rác và mức độ đáp ứng các yêu cầu chức năng, phi chức năng đã đặt ra.

1.6 KẾT LUẬN CHƯƠNG

Trong chương này, nhóm thực hiện đã trình bày thực trạng rác thải sau quá trình thu gom và sự cần thiết của việc phân loại rác tại bãi tập kết. Qua phân tích các hướng nghiên cứu liên quan, có thể thấy việc ứng dụng thị giác máy tính kết hợp với hệ thống nhúng và cơ cấu điều khiển tự động là giải pháp phù hợp để giảm thao tác thủ công và nâng cao khả năng xử lý rác hỗn hợp sau thu gom. Từ đó, đề tài xác định mục tiêu xây dựng mô hình phân loại tự động cho ba nhóm rác gồm

hữu cơ, nhựa và kim loại bằng YOLOv11, Raspberry Pi, ESP32 DevKit V1 và Firebase. Đây là cơ sở để trình bày các lý thuyết và công nghệ được sử dụng trong chương tiếp theo.

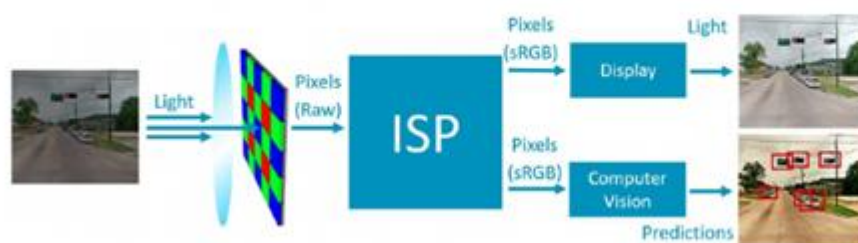
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Để xây dựng hệ thống phân loại rác tự động hoạt động tại bãi tập kết, đề tài cần kết hợp nhiều kiến thức thuộc lĩnh vực thị giác máy tính, học sâu, thiết bị nhúng và truyền thông dữ liệu. Chương này trình bày các cơ sở lý thuyết liên quan đến bài toán phát hiện đối tượng, mô hình YOLOv11, định dạng NCNN, Raspberry Pi, ESP32 DevKit V1, giao tiếp UART và Firebase Realtime Database. Các nội dung này là nền tảng cho việc thiết kế hệ thống có khả năng nhận diện và phân loại rác sau khi được đưa từ xe thu gom lên băng tải xử lý.

2.1 LÝ THUYẾT VỀ THỊ GIÁC MÁY TÍNH.

2.1.1 Tổng quan về Thị giác máy tính (Computer Vision)

Thị giác máy tính là lĩnh vực nghiên cứu giúp máy tính có khả năng thu nhận, xử lý và hiểu thông tin từ ảnh hoặc video. Trong các hệ thống tự động hóa, thị giác máy tính đóng vai trò như một “cảm biến thông minh”, hỗ trợ hoặc thay thế con người trong các nhiệm vụ như kiểm tra chất lượng, phát hiện vật thể, định vị sản phẩm và phân loại vật liệu. Một quy trình cơ bản của thị giác máy tính thường gồm các bước: thu nhận ảnh, tiền xử lý ảnh, trích xuất đặc trưng, suy luận mô hình và đưa ra quyết định [10].

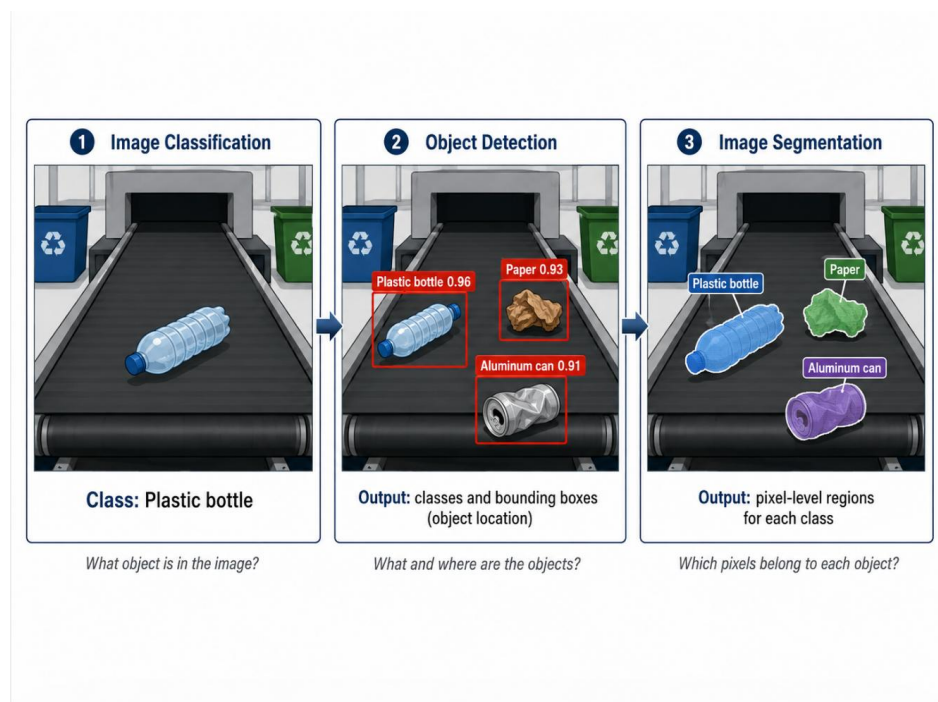


Hình 2. 1 Quy trình xử lý ảnh từ camera đến hiển thị và đầu ra thị giác máy tính.

Hình 2.1 minh họa quy trình xử lý ảnh trong hệ thống thị giác máy tính. Ảnh được camera thu nhận, xử lý để cải thiện chất lượng và đưa vào mô hình nhằm phát hiện, nhận dạng hoặc phân tích đối tượng [10]. Trong đề tài, ảnh rác trên băng tải được đưa vào mô hình YOLOv11 để xác định vị trí và lớp rác. Kết quả từ nhiều khung hình được tổng hợp bằng cơ chế detection window và voting nhằm nâng cao độ ổn định của quyết định phân loại.

2.1.2 Phát hiện đối tượng (Object Detection)

Phát hiện đối tượng là bài toán xác định đồng thời loại và vị trí của vật thể trong ảnh thông qua khung bao (bounding box) [2], [10]. Đối với hệ thống phân loại rác trên băng tải, phương pháp này phù hợp vì giúp nhận biết loại rác khi vật thể đi vào vùng xử lý. Trong các mô hình phát hiện đối tượng, YOLO thực hiện dự đoán vị trí và lớp trong một lần suy luận, nhờ đó đáp ứng tốt yêu cầu xử lý thời gian thực và phù hợp để triển khai trên thiết bị nhúng.



Hình 2. 2 Minh họa sự khác nhau giữa phân loại ảnh, phát hiện đối tượng và phân đoạn ảnh trong thị giác máy tính

Hình 2.2 minh họa sự khác nhau giữa ba bài toán phổ biến trong thị giác máy tính. Với bài toán phân loại ảnh, hệ thống chỉ xác định ảnh đầu vào thuộc lớp nào mà không cung cấp thông tin vị trí của vật thể. Trong khi đó, phát hiện đối tượng không chỉ xác định lớp của vật thể mà còn xác định vị trí thông qua khung bao quanh đối tượng. Phân đoạn ảnh cung cấp thông tin chi tiết hơn bằng cách xác định vùng điểm ảnh tương ứng với từng đối tượng [10]. Đối với hệ thống phân loại rác trên băng tải, phát hiện đối tượng là lựa chọn phù hợp vì hệ thống cần biết loại rác và vị trí của vật thể trong vùng nhận diện để phục vụ quá trình điều khiển phân loại.

2.1.3 Các chỉ số đánh giá mô hình.

Bảng 2. 1 Các chỉ số đánh giá mô hình phát hiện đối tượng.

Chỉ số	Công thức	Ý nghĩa
Precision	$TP/(TP+FP)$	Tỉ lệ dự đoán dương tính đúng trong toàn bộ dự đoán dương tính [11].
Recall	$TP/(TP+FN)$	Tỉ lệ đối tượng thật được phát hiện đúng [11].
F1-score	$2PR/(P+R)$	Trung bình điều hòa giữa Precision và Recall [11].

mAP50	AP trung bình tại IoU=0,5	Đánh giá chất lượng phát hiện với ngưỡng IoU 50% [11].
mAP50-95	Trung bình AP từ IoU 0,5 đến 0,95	Chỉ số nghiêm ngặt hơn, phản ánh cả định vị bbox [11].
IoU	$\text{Area}(A \cap B) / \text{Area}(A \cup B)$	Mức chồng lấp giữa bbox dự đoán và bbox thật [11].
FPS	Số frame xử lý/giây	Thước đo tốc độ suy luận trong hệ thống thời gian thực.

Precision cao giúp giảm phân loại nhầm, còn Recall cao giúp giảm bỏ sót vật thể. Đối với phân loại rác trên băng tải, false positive có thể làm servo chuyển sai thùng, trong khi false negative làm vật đi qua mà không phân loại. Vì vậy hệ thống dùng thêm voting và cảm biến để cân bằng hai loại lỗi.

2.2 KIẾN TRÚC YOLO VÀ YOLOv11

2.2.1 Lịch sử phát triển của YOLO

YOLO (You Only Look Once) là họ mô hình phát hiện đối tượng thời gian thực, được giới thiệu lần đầu bởi Redmon và cộng sự [2]. Khác với các phương pháp xử lý nhiều giai đoạn, YOLO dự đoán trực tiếp vị trí khung bao và lớp đối tượng trong một lần suy luận trên ảnh đầu vào, nhờ đó có ưu thế về tốc độ xử lý.

Qua nhiều phiên bản, YOLO được cải tiến nhằm nâng cao độ chính xác và hiệu quả triển khai. Trong đề tài này, YOLOv11 [3] được lựa chọn để nhận diện ba nhóm rác trên băng tải vì phù hợp với yêu cầu phát hiện đối tượng theo thời gian thực và có thể chuyển đổi sang định dạng NCNN để triển khai trên Raspberry Pi..

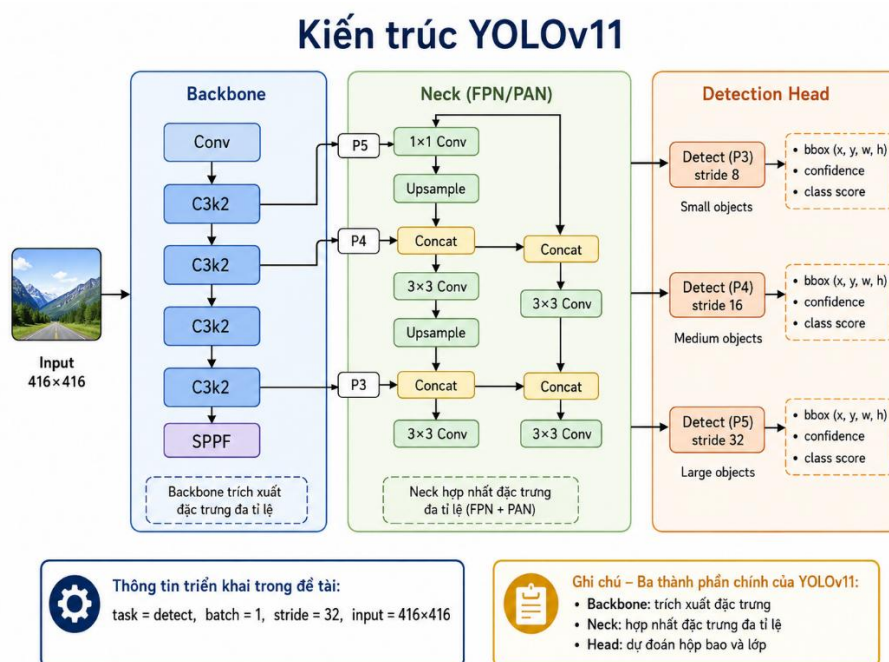
Bảng 2. 2 So sánh khái quát một số phiên bản YOLO.

Phiên bản	Năm giới thiệu	Đặc điểm chính	Ưu điểm	Hạn chế
YOLOv1	2016	Xem phát hiện đối tượng như bài toán hồi quy một lần trên toàn ảnh	Tốc độ nhanh, kiến trúc đơn giản	Độ chính xác với vật thể nhỏ và vật thể gần nhau còn hạn chế
YOLOv3	2018	Sử dụng backbone Darknet-53, phát hiện ở nhiều tỉ lệ đặc trưng	Cải thiện khả năng phát hiện vật thể nhiều kích thước	Mô hình lớn hơn, yêu cầu tài nguyên cao hơn
YOLOv5	2020	Dễ huấn luyện, dễ triển khai, hỗ trợ nhiều kích thước mô hình	Phổ biến, có hệ sinh thái triển khai mạnh	Không phải phiên bản gốc từ nhóm tác giả YOLO ban đầu
YOLOv8	2023	Kiến trúc hiện đại hơn, hỗ trợ nhiều tác vụ thị giác máy tính	Cân bằng tốt giữa tốc độ và độ chính xác	Cần tối ưu thêm khi chạy trên thiết bị nhúng yếu
YOLOv11	2024	Tiếp tục tối ưu kiến trúc và hiệu quả suy luận	Phù hợp cho các bài toán thời gian thực và triển khai edge	Hiệu quả phụ thuộc vào dữ liệu huấn luyện và cấu hình phần cứng

2.2.2 Kiến trúc YOLOv11

YOLOv11 được tổ chức theo ba thành phần chính gồm Backbone, Neck và Detection Head. Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào thông qua các khối xử lý như Conv, C3k2 và SPPF. Neck hợp nhất đặc trưng ở nhiều tỉ lệ nhằm hỗ trợ nhận diện các vật thể có kích thước khác nhau. Cuối cùng, Detection Head dự đoán vị trí khung bao, độ tin cậy và lớp của vật thể tại các mức đặc trưng P3, P4 và P5 [3], [12].

Trong đề tài, mô hình YOLOv11 được sử dụng cho bài toán phát hiện ba nhóm rác trên băng tải. Mô hình sau khi huấn luyện được triển khai với kích thước ảnh đầu vào 416×416 , task detect, batch 1 và stride 32, phù hợp với quá trình suy luận trên Raspberry Pi.



Hình 2. 3 Kiến trúc tổng quát của mô hình YOLOv11 sử dụng trong đề tài

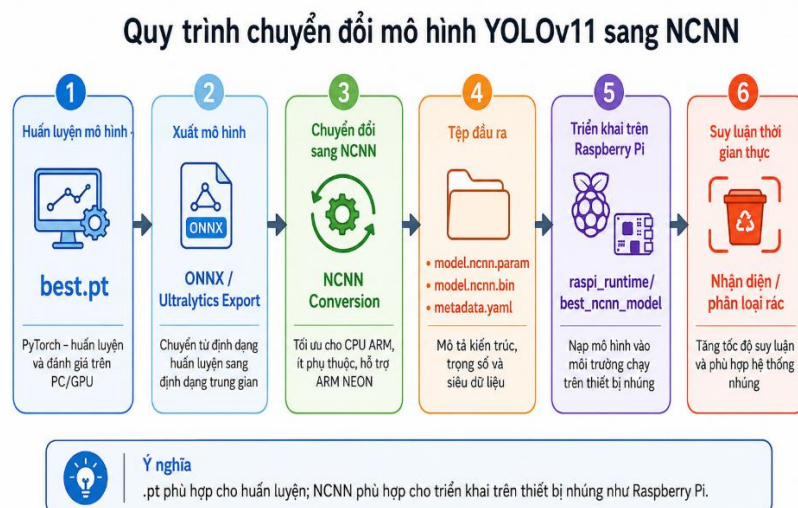
Hình 2.3 trình bày kiến trúc khái quát của mô hình YOLOv11 gồm ba phần: Backbone, Neck và Detection Head. Backbone thực hiện trích xuất đặc trưng từ ảnh đầu vào; Neck hợp nhất đặc trưng đa tỉ lệ; Detection Head dự đoán vị trí và lớp của vật thể tại các mức P3, P4 và P5. Trong đề tài, mô hình sử dụng ảnh đầu vào 416×416 , task detect, batch 1 và stride 32 để phục vụ nhận diện rác trên Raspberry Pi.

2.2.3 Định dạng NCNN và tối ưu cho thiết bị nhúng

Mô hình YOLOv11 sau khi huấn luyện thường được lưu dưới định dạng .pt, phù hợp cho quá trình đánh giá và kiểm thử trong môi trường PyTorch. Tuy nhiên, để

triển khai trên Raspberry Pi, đề tài sử dụng định dạng NCNN nhằm giảm phụ thuộc thư viện và nâng cao hiệu quả suy luận trên nền tảng CPU ARM [4].

NCNN là framework suy luận mạng nơ-ron được tối ưu cho các thiết bị di động và nhúng. Ultralytics hỗ trợ chuyển đổi trực tiếp mô hình YOLO sang định dạng NCNN để phục vụ triển khai thực tế [5]. Trong đề tài, mô hình sau chuyển đổi được sử dụng trên Raspberry Pi để nhận diện ba nhóm rác trong quá trình vận hành hệ thống.



Hình 2. 4 Quy trình chuyển đổi mô hình YOLOv11 sang định dạng NCNN

Hình 2.4 trình bày quy trình chuyển đổi mô hình YOLOv11 từ tập trọng số .pt sang định dạng NCNN. Mô hình sau chuyển đổi được đưa vào Raspberry Pi để thực hiện suy luận nhận diện rác trong thời gian thực.

2.3 NỀN TẢNG PHẦN CỨNG

2.3.1 Raspberry Pi – bộ xử lý trung tâm

Bảng 2. 3 Thông số Raspberry Pi dùng trong hệ thống

Thông số	Giá trị
CPU	ARM Cortex-A series, chạy Linux
RAM	Tùy phiên bản, khuyến nghị từ 4 GB
OS	Raspberry Pi OS/Linux
GPIO	UART, digital I/O
Camera interface	USB hoặc CSI
USB	Kết nối camera/serial nếu cần
Power	5 V, nguồn ổn định

Raspberry Pi [13] đóng vai trò bộ xử lý trung tâm của hệ thống, có nhiệm vụ tiếp nhận hình ảnh từ camera, thực hiện suy luận mô hình YOLOv11/NCNN và gửi lệnh phân loại đến ESP32 DevKit V1 thông qua giao tiếp UART. Bên cạnh đó, Raspberry Pi còn hỗ trợ cập nhật dữ liệu trạng thái và kết quả phân loại phục vụ quá trình giám sát hệ thống.

2.3.2 ESP32 DevKit V1 và giao tiếp UART

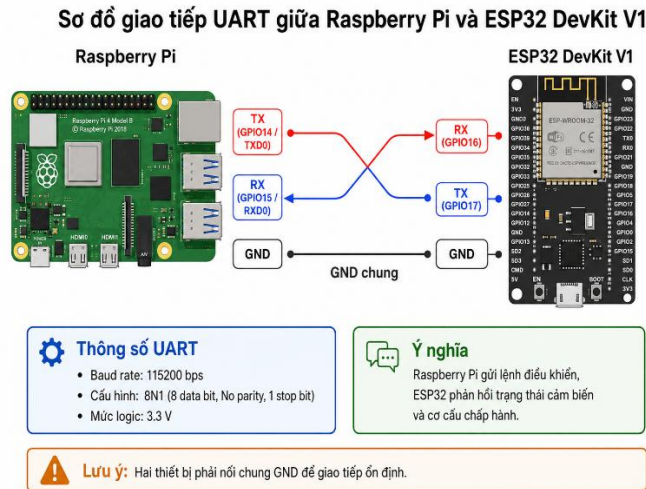
Bảng 2. 4 Thông số ESP32-DEVKIT-V1

Thông số	Giá trị
Vi điều khiển	ESP32-WROOM-32
CPU	Dual-core Xtensa LX6, xung nhịp tối đa 240 MHz
Bộ nhớ Flash	Thường 4 MB, tùy phiên bản module

Bộ nhớ RAM	520 KB SRAM tích hợp
Wi-Fi	2.4 GHz, chuẩn 802.11 b/g/n
Bluetooth	Bluetooth Classic và Bluetooth Low Energy
GPIO	Hỗ trợ nhiều chân digital I/O, ADC, DAC, UART, I2C, SPI
PWM	Hỗ trợ PWM thông qua bộ LEDC, nhiều kênh điều khiển
Điện áp logic	3.3 V
Nguồn cấp board	5 V qua cổng USB hoặc chân VIN
Ghi chú nguồn tải	Servo và động cơ nên dùng nguồn riêng, nối chung GND với ESP32

ESP32 DevKit V1 [14] sử dụng module ESP32-WROOM-32, tích hợp CPU hai lõi, các giao tiếp UART và khả năng phát xung PWM phù hợp để đọc cảm biến và điều khiển servo. Trong hệ thống, ESP32 DevKit V1 đóng vai trò khối chấp hành: nhận lệnh từ Raspberry Pi, điều khiển cơ cấu phân loại và gửi trạng thái phản hồi.

UART là giao tiếp nối tiếp bất đồng bộ sử dụng các đường TX, RX và GND chung. Hai thiết bị phải thống nhất tốc độ truyền và định dạng khung dữ liệu [15]. Trong đề tài, UART được chọn vì đơn giản, dễ kiểm tra và phù hợp với khoảng cách kết nối ngắn trong mô hình thí nghiệm. Chuỗi lệnh và trạng thái như SORT, ACK, DONE, SENSOR và STATUS là giao thức do nhóm xây dựng, được trình bày trong Chương 3 và Chương 4.



Hình 2. 5 Sơ đồ giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1

Hình 2.5 minh họa giao tiếp UART hai chiều giữa Raspberry Pi và ESP32 DevKit V1. Raspberry Pi gửi lệnh điều khiển phân loại, còn ESP32 phản hồi trạng thái thực thi và tín hiệu cảm biến. Trong hệ thống, UART được cấu hình ở tốc độ 115200 bps, định dạng 8N1, đáp ứng yêu cầu truyền lệnh và giám sát trạng thái trong mô hình thực nghiệm [15].

2.4 FIREBASE REALTIME DATABASE

Firebase Realtime Database là cơ sở dữ liệu đám mây lưu dữ liệu dưới dạng JSON và đồng bộ thay đổi đến các ứng dụng kết nối theo thời gian thực [16]. Đặc điểm này phù hợp với hệ thống cần hiển thị trạng thái vận hành, bộ đếm phân loại và kết quả nhận diện gần nhất trên dashboard hoặc ứng dụng di động.

Trong đề tài, chương trình trên Raspberry Pi sử dụng Firebase Admin SDK để cập nhật dữ liệu từ phía thiết bị xử lý lên cơ sở dữ liệu. Firebase phục vụ giám sát và thống kê; quyết định phân loại chính vẫn được xử lý cục bộ giữa Raspberry Pi và ESP32 nhằm tránh phụ thuộc vào độ trễ mạng.

2.5 KẾT LUẬN CHƯƠNG

Chương 2 đã trình bày các kiến thức nền tảng và công nghệ chính được sử dụng trong hệ thống phân loại rác tại bãi tập kết. Trong đó, YOLOv11 phù hợp với nhiệm vụ phát hiện và phân loại rác trên băng tải nhờ khả năng xử lý hình ảnh theo thời gian thực. Định dạng NCNN hỗ trợ triển khai mô hình trên Raspberry Pi, trong khi ESP32 DevKit V1, giao tiếp UART và Firebase tạo nền tảng cho việc điều khiển cơ cấu phân loại và giám sát dữ liệu. Những nội dung này là cơ sở để xây dựng thiết kế tổng thể của hệ thống trong Chương 3.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Dựa trên bài toán phân loại rác tại bãi tập kết và các cơ sở lý thuyết đã trình bày, chương này tập trung thiết kế hệ thống phân loại rác tự động sau quá trình thu gom. Trong mô hình đề xuất, rác từ xe thu gom được đưa lên băng tải, di chuyển qua vùng quan sát của camera để Raspberry Pi thực hiện nhận diện bằng YOLOv11. Sau khi xác định loại rác, ESP32 DevKit V1 điều khiển cơ cấu servo để đưa vật thể vào ngăn chứa tương ứng. Chương này trình bày yêu cầu hệ thống, kiến trúc tổng thể, thiết kế phần cứng, phần mềm, logic nhận diện và phương án lưu trữ dữ liệu giám sát.

3.1 YÊU CẦU VÀ ĐẶC TẢ HỆ THỐNG

3.1.1 Yêu cầu chức năng

1. Camera phải thu hình liên tục từ băng tải và cung cấp frame cho mô hình nhận diện.
2. Raspberry Pi phải chạy YOLOv11/NCNN để phát hiện organic, plastic và metal.
3. Logic nhận diện phải chỉ ra quyết định sau khi đủ detection window và voting.
4. Hệ thống phải dừng băng tải khi entry sensor báo có vật và tiếp tục sau khi tạo job.
5. ESP32 DEVKIT V1 phải nhận lệnh UART và điều khiển servo đến đúng vị trí thùng.
6. Exit sensor phải kích hoạt việc gửi lệnh phân loại cho vật đã được queue.
7. Bộ đếm phân loại phải tăng sau khi nhận DONE từ ESP32 DEVKIT V1.

8. Firebase phải phản ánh trạng thái, counters và last_detection để phục vụ dashboard.

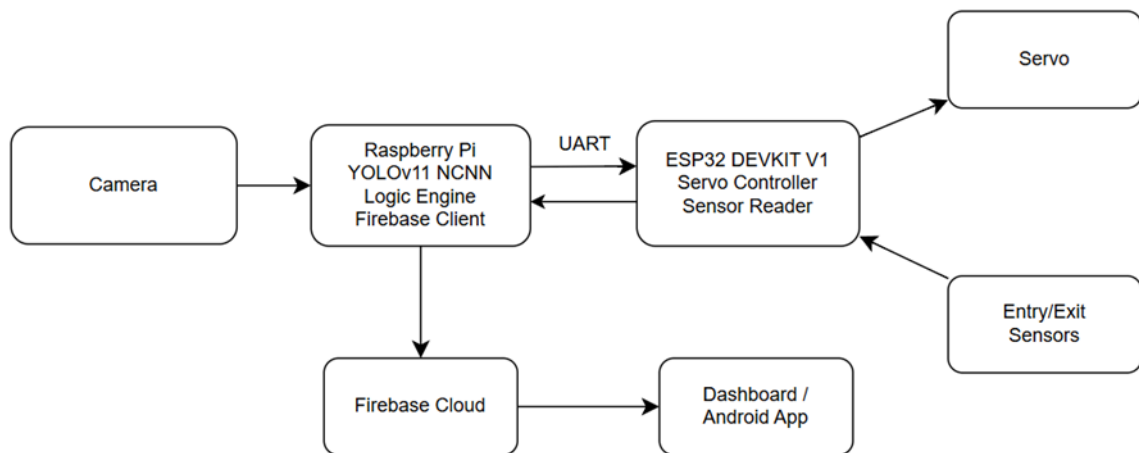
Bảng 3. 1 Yêu cầu phi chức năng

Chỉ tiêu	Yêu cầu	Ghi chú
FPS	≥ 5	Với mô hình NCNN trên Raspberry Pi
Độ chính xác	$\geq 80\%$	Đánh giá theo validation và thử nghiệm thực tế
Độ trễ UART	< 200 ms	Lệnh ngán ở baudrate 115200
Thời gian phân loại/vật	< 3 s	Phụ thuộc detection window và cơ cấu servo
Đồng bộ Firebase	< 5 s	config sync interval = 1.0 s
Hoạt động liên tục	> 8 h	Cần nguồn ổn định và tản nhiệt

3.2 Kiến trúc tổng thể hệ thống

3.2.1 Sơ đồ khối tổng quan

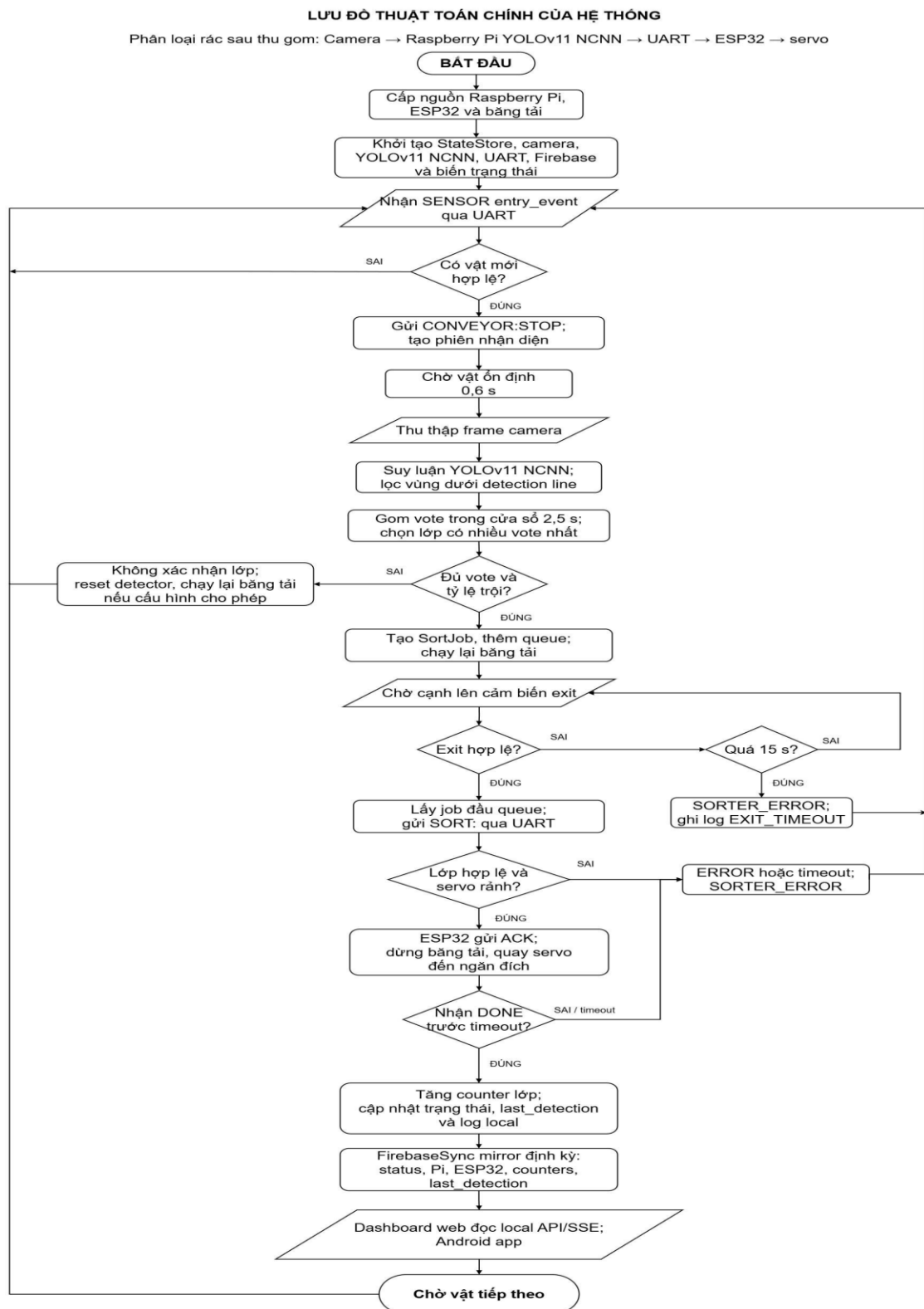
Kiến trúc tổng thể gồm camera, Raspberry Pi, ESP32 DEVKIT V1, servo, cảm biến entry/exit và Firebase Cloud. Camera gửi hình ảnh đến Raspberry Pi; Raspberry Pi chạy YOLOv11 NCNN, logic engine và Firebase client. Raspberry Pi trao đổi UART hai chiều với ESP32 DEVKIT V1. ESP32 DEVKIT V1 đọc cảm biến, điều khiển servo và phản hồi trạng thái.



Hình 3. 1 Sơ đồ khối tổng thể hệ thống

Hình 3.1 mô tả các luồng xử lý chính trong hệ thống phân loại rác tự động. Luồng dữ liệu thị giác bắt đầu từ camera, sau đó được Raspberry Pi xử lý bằng mô hình YOLOv11 và đưa vào khối logic voting để xác định loại rác. Luồng điều khiển được truyền từ Raspberry Pi đến ESP32 DevKit V1 thông qua giao tiếp UART dưới dạng lệnh văn bản. ESP32 DevKit V1 tiếp nhận lệnh, điều khiển servo, băng tải và đọc tín hiệu từ các cảm biến entry/exit. Bên cạnh đó, luồng giám sát được thực hiện thông qua StateStore cục bộ và FirebaseSync, nhằm đồng bộ trạng thái hệ thống, bộ đếm phân loại và kết quả nhận diện lên Firebase Realtime Database.

Để mô tả trình tự hoạt động tổng quát của hệ thống phân loại rác tự động sau thu gom, đề tài xây dựng lưu đồ thuật toán chính thể hiện toàn bộ quá trình từ khởi tạo hệ thống, nhận diện vật thể, ra quyết định phân loại đến cập nhật dữ liệu giám sát. Lưu đồ này là cơ sở để triển khai các thuật toán con ở các mục tiếp theo.



Hình 3. 2 Lưu đồ thuật toán chính

Hình 3.2 trình bày lưu đồ thuật toán chính của hệ thống phân loại rác tự động. Quá trình bắt đầu khi Raspberry Pi, ESP32 và băng tải được cấp nguồn và khởi tạo các thành phần cần thiết như camera, mô hình YOLOv11 NCNN, UART, Firebase và các biến trạng thái. Khi Raspberry Pi nhận tín hiệu 'entry_event' từ ESP32 qua UART, hệ thống kiểm tra sự xuất hiện của vật thể mới. Nếu vật thể hợp lệ, băng tải được dừng tạm thời để tạo phiên nhận diện, sau đó hệ thống chờ vật ổn định rồi thu thập các khung hình từ camera.

Các khung hình thu được sẽ được đưa vào mô hình YOLOv11 NCNN để suy luận và lọc vùng phát hiện phù hợp. Kết quả nhận diện không được quyết định từ một khung hình đơn lẻ mà được tổng hợp trong một khoảng thời gian xác định bằng cơ chế voting. Nếu số lượng phiếu và tỉ lệ lớp trội đạt ngưỡng yêu cầu, hệ thống tạo một tác vụ phân loại và đưa vào hàng đợi; ngược lại, bộ nhận diện được đặt lại và băng tải tiếp tục hoạt động nếu cấu hình cho phép.

Khi cảm biến exit phát hiện vật thể đi tới vị trí phân loại, Raspberry Pi lấy tác vụ đầu hàng đợi và gửi lệnh 'SORT' đến ESP32 qua UART. ESP32 tiếp nhận lệnh, phản hồi 'ACK', dừng băng tải và điều khiển servo quay đến ngăn chứa tương ứng với lớp rác đã nhận diện. Sau khi hoàn tất, ESP32 gửi tín hiệu 'DONE' về Raspberry Pi. Nếu xảy ra lỗi hoặc quá thời gian chờ, hệ thống ghi nhận trạng thái lỗi để phục vụ giám sát và xử lý.

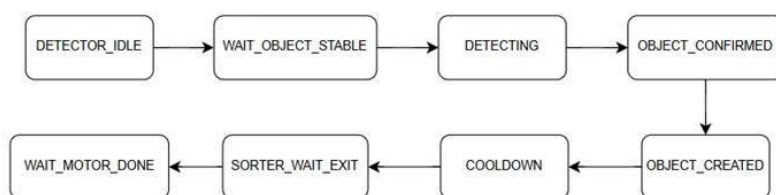
Sau khi phân loại thành công, Raspberry Pi cập nhật bộ đếm, trạng thái hệ thống, kết quả nhận diện gần nhất và ghi log cục bộ. Dữ liệu này tiếp tục được đồng bộ lên Firebase để phục vụ dashboard web và ứng dụng Android. Sau đó hệ thống quay lại trạng thái chờ để xử lý vật thể tiếp theo.

Lưu đồ thuật toán chính thể hiện chu trình hoạt động tổng thể của hệ thống, từ thời điểm phát hiện vật thể đến khi hoàn tất phân loại và cập nhật dữ liệu giám sát. Để tránh làm gián đoạn nội dung trình bày của chương, các lưu đồ chi tiết

cho từng khối xử lý như nhận diện và voting, xử lý cảm biến entry/exit, giao tiếp UART, điều khiển servo và đồng bộ Firebase được trình bày tại Phụ lục E.

3.2.2 Luồng dữ liệu và luồng điều khiển

1. Entry sensor báo có vật, Raspberry Pi gửi CONVEYOR: STOP và bắt đầu detection session.
2. Camera tiếp tục thu khung hình trong thời gian vật ổn định.
3. Mỗi frame đủ chu kỳ được đưa vào YOLOv11 NCNN để tạo detection.
4. Kết quả được tích lũy trong detection window và đếm vote theo lớp.
5. Nếu confidence, số vote và dominant ratio đạt ngưỡng, lớp thắng được xác nhận.
6. Job được đưa vào queue; khi exit sensor kích hoạt, Raspberry Pi gửi SORT:<class>.
7. ESP32 DEVKIT V1 di chuyển servo đến vị trí thùng tương ứng và trả DONE: SORT:<class>.
8. Raspberry Pi tăng counter, cập nhật StateStore và Firebase.



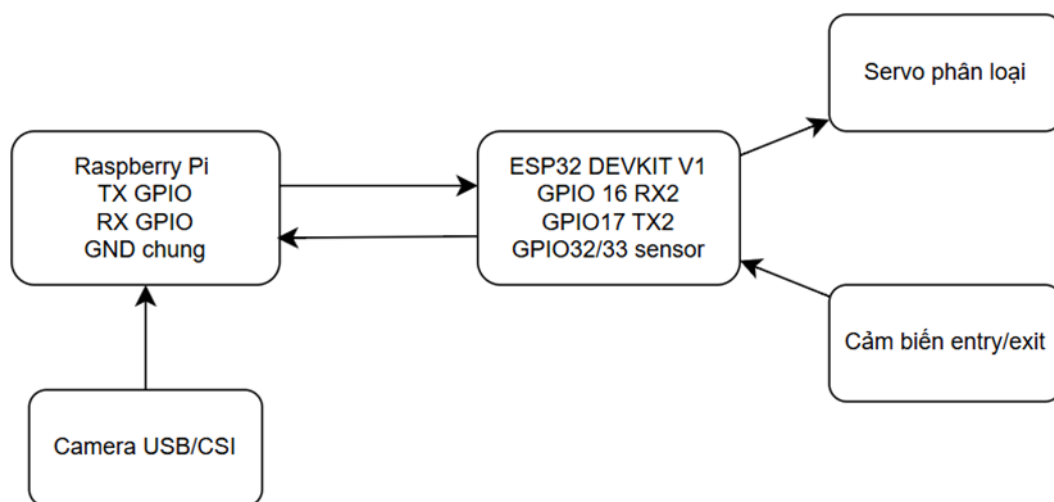
Hình 3. 3 Flowchart luồng xử lý tổng thể

3.3 THIẾT KẾ PHẦN CỨNG

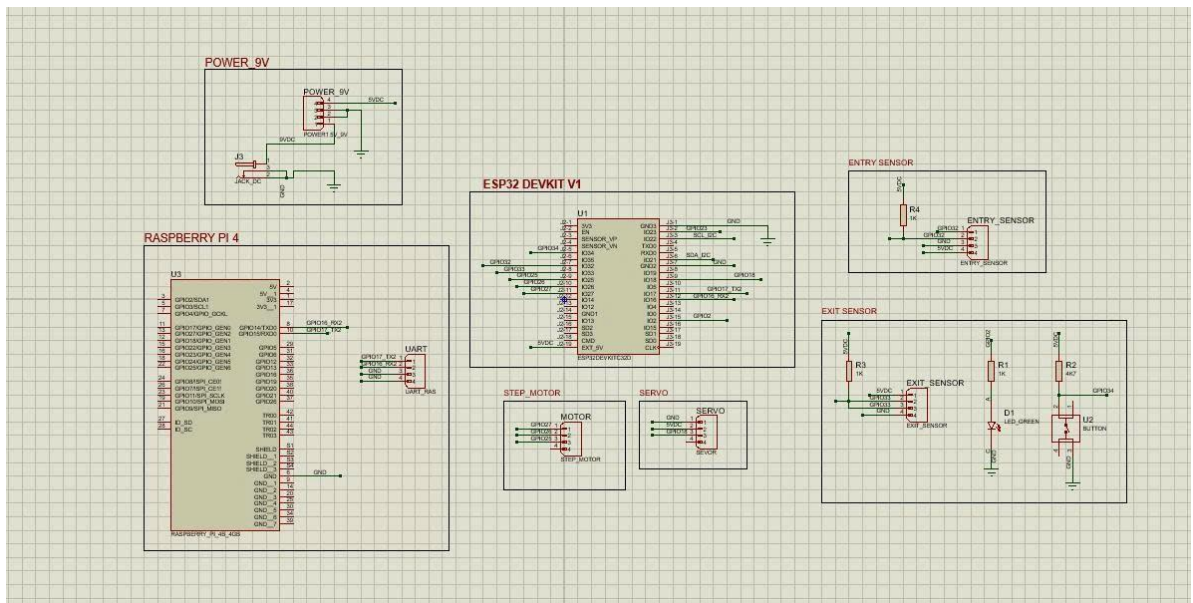
3.3.1 Sơ đồ kết nối phần cứng tổng thể

Bảng 3. 2 Bảng kết nối phần cứng

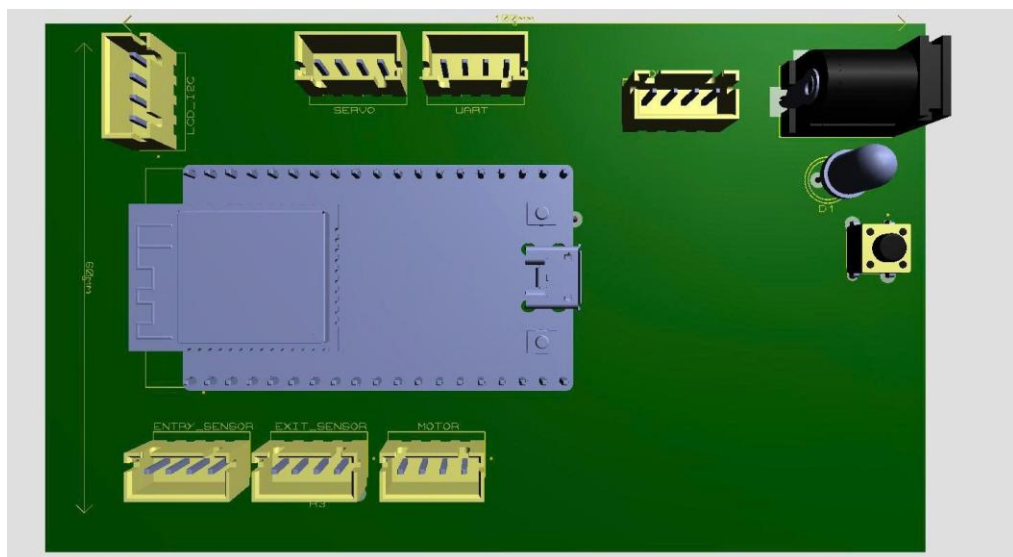
Thiết bị nguồn	Chân/Cổng	Thiết bị đích	Chân/Cổng	Ghi chú
Raspberry Pi	GPIO TX	ESP32 DEVKIT V1	GPIO16 RX2	TX/RX đầu chéo
Raspberry Pi	GPIO RX	ESP32 DEVKIT V1	GPIO17 TX2	Nhận phản hồi ACK/DONE
Raspberry Pi	GND	ESP32 DEVKIT V1	GND	Bắt buộc chung mass
ESP32 DEVKIT V1	GPIO18 PWM	Servo	Signal	Điều khiển góc thùng
Camera	USB/CSI	Raspberry Pi	USB/CSI	Nguồn hình ảnh
Entry sensor	OUT	ESP32 DEVKIT V1	GPIO32	Active low theo firmware
Exit sensor	OUT	ESP32 DEVKIT V1	GPIO33	Kích hoạt dispatch job
Stepper driver	STEP/DIR/EN	ESP32 DEVKIT V1	GPIO27/26/25	Điều khiển bằng tải



Hình 3. 4 Sơ đồ kết nối phần cứng



Hình 3. 5 Sơ đồ thiết kế trên protues



Hình 3. 6 Phần cứng 3D

3.3.2 Thiết kế cụm xử lý trung tâm (Raspberry Pi + Camera)

Cụm xử lý trung tâm của hệ thống bao gồm Raspberry Pi và camera, có nhiệm vụ thu nhận hình ảnh từ băng tải, thực hiện nhận diện rác bằng mô hình YOLOv11 và đưa ra quyết định phân loại để gửi đến khối chấp hành. Trong đó, camera

đóng vai trò cảm biến thị giác, còn Raspberry Pi đảm nhiệm chức năng xử lý ảnh, chạy mô hình suy luận, thực hiện logic quyết định và giao tiếp với ESP32 DevKit V1.

Camera được lắp cố định phía trên băng tải, với góc quan sát hướng xuống vùng làm việc nhằm đảm bảo vật thể đi qua luôn nằm trong khung hình. Vị trí lắp đặt được lựa chọn sao cho vùng nhận diện tập trung ở nửa dưới ảnh, giúp hệ thống dễ xác định thời điểm vật thể thực sự đi vào vùng xử lý. Trong chương trình detect.py, đường nhận diện được thiết lập thông qua tham số

`DETECTION_LINE_Y_RATIO = 0.45`, nghĩa là chỉ những vật thể có phần lớn hộp bao nằm phía dưới đường này mới được xem là đối tượng hợp lệ để xử lý.

Về phần cứng xử lý, Raspberry Pi nhận luồng ảnh từ camera thông qua giao tiếp USB hoặc CSI, sau đó tiến hành xử lý ảnh và đưa vào mô hình YOLOv11 đã được chuyển đổi sang NCNN để tăng tốc suy luận trên thiết bị nhúng. Cấu hình camera mặc định được sử dụng trong hệ thống là 640×480 pixel, tốc độ 20 FPS, với backend V4L2 trên Linux. Cấu hình này đáp ứng được yêu cầu quan sát liên tục băng tải, đồng thời đảm bảo cân bằng giữa chất lượng hình ảnh và tốc độ xử lý.

Sau khi mô hình nhận diện tạo ra kết quả phát hiện, Raspberry Pi tiếp tục áp dụng cơ chế voting trong cửa sổ thời gian để tăng độ ổn định của quyết định phân loại. Kết quả cuối cùng sẽ được chuyển thành lệnh điều khiển và gửi sang ESP32 DevKit V1 thông qua giao tiếp UART để thực hiện điều khiển servo và băng tải. Nhờ đó, cụm Raspberry Pi và camera đóng vai trò là trung tâm xử lý thông minh của toàn hệ thống..



Hình 3. 7 Sơ đồ cụm Raspberry Pi và camera

Hình 3.7 minh họa cụm xử lý trung tâm của hệ thống, bao gồm camera và Raspberry Pi. Camera có nhiệm vụ thu nhận hình ảnh từ băng tải và truyền dữ liệu về Raspberry Pi thông qua giao tiếp USB hoặc CSI. Raspberry Pi tiếp nhận khung hình, thực hiện suy luận bằng mô hình YOLOv11/NCNN, xử lý logic voting và gửi lệnh điều khiển đến ESP32 DevKit V1 thông qua UART.

3.3.3 Thiết kế cụm chấp hành (ESP32 DEVKIT V1, Servo và Cảm biến)

Firmware sử dụng servo 180 độ với ba vị trí: organic ở 90 độ, plastic ở 5 độ và metal ở 175 độ. Khi nhận lệnh SORT, ESP32 DEVKIT V1 dừng băng tải, di chuyển servo trong 700 ms, gửi DONE và có thể khởi động lại băng tải. Cảm biến entry/exit dùng active-low, debounce 40 ms ở firmware và được Raspberry Pi lọc thêm bằng debounce mức hệ thống.

Bảng 3. 3 Phân công GPIO ESP32 DEVKIT V1

GPIO	Chức năng	Mức tín hiệu	Ghi chú
16	UART2 RX	Serial 3.3 V	Nhận lệnh từ Raspberry Pi
17	UART2 TX	Serial 3.3 V	Gửi ACK/DONE/SENSOR/STATUS
18	PWM servo	50 Hz	Servo phân loại
25	Stepper EN	Active low	Bật/tắt driver
26	Stepper DIR	Digital	Chiều quay băng tải
27	Stepper STEP	Xung	Tốc độ mặc định 140 sps
32	Entry sensor	Active low	Phát hiện vật vào
33	Exit sensor	Active low	Phát hiện vật ra
34	Home button	Active low	Cần điện trở ngoài

3.3.4 Thiết kế cơ khí – khung phân loại và băng tải

Mô hình cơ khí gồm băng tải đưa vật qua vùng camera, cụm cảm biến đầu vào/đầu ra và cơ cấu servo chuyển hướng vật vào thùng tương ứng. Tốc độ băng tải mặc định trong firmware là 140 step/s, có thể điều chỉnh qua lệnh SPEED: <value> trong khoảng 40 đến 600 step/s. Kết cấu nên dùng vật liệu nhẹ, cứng vững và dễ hiệu chỉnh như nhôm định hình, mica hoặc in 3D.



Hình 3. 8 Mô hình cơ khí hệ thống

3.4 THIẾT KẾ PHẦN MỀM

Phần mềm của hệ thống được xây dựng theo kiến trúc chia module nhằm tách biệt các chức năng chính như thu nhận hình ảnh, nhận diện đối tượng, xử lý logic phân loại, giao tiếp UART, lưu trạng thái cục bộ và đồng bộ dữ liệu lên Firebase. Cách tổ chức này giúp chương trình dễ mở rộng, dễ kiểm thử và thuận tiện trong quá trình bảo trì.

Trong hệ thống, Raspberry Pi đóng vai trò là bộ xử lý trung tâm cho phần mềm nhận diện. Thiết bị này tiếp nhận hình ảnh từ camera, chạy mô hình YOLOv11/NCNN, xử lý kết quả nhận diện và đưa ra quyết định phân loại. Sau

khi xác định loại rác, Raspberry Pi gửi lệnh điều khiển đến ESP32 DevKit V1 thông qua giao tiếp UART. Đồng thời, trạng thái hệ thống, bộ đếm phân loại và kết quả nhận diện được lưu cục bộ và đồng bộ lên Firebase để phục vụ giám sát.

3.4.1 Kiến trúc phần mềm trên Raspberry Pi

Phần mềm trên Raspberry Pi được tổ chức thành nhiều module Python độc lập, mỗi module đảm nhiệm một nhóm chức năng riêng. Module `main.py` đóng vai trò chương trình điều phối chính, chịu trách nhiệm khởi tạo camera, mô hình nhận diện, hàng đợi phân loại, kết nối UART và cơ chế đồng bộ dữ liệu. Module `detect.py` thực hiện quá trình nhận diện rác từ khung hình camera, bao gồm tiền xử lý ảnh, suy luận mô hình YOLOv11/NCNN và giải mã kết quả phát hiện.

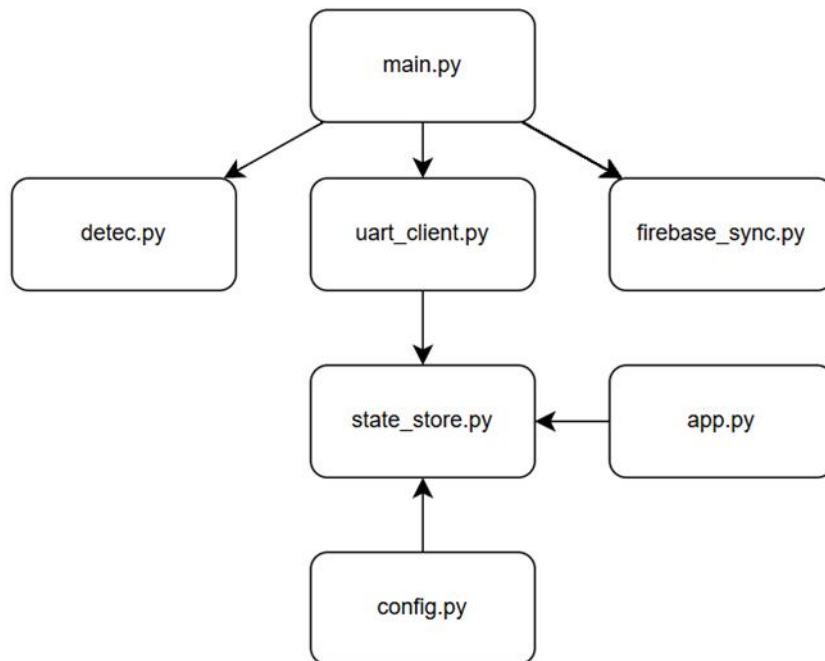
Bên cạnh đó, `uart_client.py` phụ trách giao tiếp nối tiếp với ESP32 DevKit V1, bao gồm gửi lệnh điều khiển và nhận phản hồi trạng thái như ACK hoặc DONE. Module `state_store.py` được sử dụng để lưu trạng thái cục bộ của hệ thống, bao gồm bộ đếm từng loại rác, kết quả nhận diện gần nhất, log hoạt động và các lệnh đang chờ xử lý. Module `firebase_sync.py` thực hiện đồng bộ dữ liệu từ bộ nhớ cục bộ lên Firebase Realtime Database. Ngoài ra, `app.py` cung cấp API Flask để phục vụ dashboard hoặc ứng dụng giám sát, còn `config.py` tập trung các tham số cấu hình như camera, UART, Firebase và các ngưỡng nhận diện.

Cách chia module như trên giúp hệ thống hạn chế sự phụ thuộc giữa các thành phần, đồng thời hỗ trợ xử lý song song các tác vụ như đọc camera, nhận diện, giao tiếp UART và đồng bộ dữ liệu. Điều này phù hợp với yêu cầu vận hành của hệ thống phân loại rác tự động, trong đó các thao tác nhận diện, điều khiển và giám sát cần được thực hiện ổn định theo thời gian thực.

Bảng 3. 4 Các module phần mềm trên Raspberry Pi

File/Module	Vai trò	Chức năng chính	Phụ thuộc
<code>main.py</code>	Runtime chính	Khởi tạo camera, mô hình YOLO, hàng	OpenCV, Serial, Firebase

		đội phân loại, UART và Firebase	
detect.py	Nhận diện đối tượng	Tải mô hình YOLOv11/NCNN, xử lý frame, phát hiện bbox và lớp rác	OpenCV, NCNN/Ultralytics
app.py	Dashboard/API	Cung cấp API Flask, trạng thái hệ thống và lệnh điều khiển	Flask, StateStore
config.py	Cấu hình hệ thống	Lưu thông số camera, UART, Firebase, ngưỡng nhận diện	Python config
state_store.py	Lưu trạng thái cục bộ	Lưu bộ đếm, log, trạng thái pending command	JSON/File storage
uart_client.py	Giao tiếp UART	Gửi lệnh đến ESP32 và nhận phản hồi ACK/DONE	PySerial
firebase_sync.py	Đồng bộ Firebase	Đồng bộ trạng thái, bộ đếm và kết quả phân loại lên Firebase	Firebase SDK



Hình 3. 9 Sơ đồ phụ thuộc giữa các module

Hình 3.9 mô tả mối quan hệ phụ thuộc giữa các module phần mềm trên Raspberry Pi. Trong đó, `main.py` là module trung tâm, có nhiệm vụ điều phối các thành phần còn lại của hệ thống. Module này gọi `detect.py` để thực hiện nhận diện rác, sử dụng `uart_client.py` để giao tiếp với ESP32 DevKit V1 và gọi `firebase_sync.py` để đồng bộ dữ liệu lên Firebase. Module `state_store.py` đóng vai trò lưu trữ trạng thái trung gian, được sử dụng bởi các thành phần như `app.py`, `uart_client.py` và `firebase_sync.py`. Module `config.py` cung cấp các tham số cấu hình chung cho toàn bộ hệ thống.

3.4.2 Chức năng từng module Python

3.4.2.1. Module `main.py`

‘`main.py`’ tạo ‘`TrashSortingRuntime`’, quản lý ‘`CameraStream`’, ‘`YoloDetector`’, ‘`UartClient`’, ‘`FirestoreSync`’ và queue ‘`SortJob`’. File này định nghĩa hai nhóm

trạng thái song song: detector và sorter. Vòng lặp ‘_loop()’ đọc camera, tính FPS, thực hiện voting khi detector ở trạng thái DETECTOR_DETECTING, tạo job sau khi xác nhận lớp và xử lý timeout.

3.4.2.2. Module app.py

‘app.py’ cung cấp dashboard API bằng Flask. Hàm ‘normalize_command()’ chuẩn hóa lệnh người dùng; ‘command_to_uart()’ chuyển lệnh web thành chuỗi UART như CONVEYOR:START, SPEED:<n>, BIN:<class> hoặc SORT:<class>. Các endpoint ‘/api/status’, ‘/api/events’, ‘/api/command’ và ‘/api/reset_counter’ giao tiếp với ‘StateStore’.

3.4.2.3. Module detec.py

‘detec.py’ định nghĩa ‘Detection’, ‘Track’, ‘DetectionTracker’ và ‘YoloDetector’. Module hỗ trợ backend NCNN hoặc Ultralytics, tiền xử lý ảnh về YOLO_IMGSZ, giải mã output NCNN, NMS theo lớp, đánh dấu vùng nhận diện dưới line y và vẽ overlay phục vụ stream.

3.4.2.4. Module config.py

‘config.py’ tập trung toàn bộ tham số runtime, bao gồm camera, UART, YOLO, detection window, debounce, Firebase, command list và ‘HARDWARE_PROFILE’. Các biến có thể override bằng environment variable, giúp triển khai linh hoạt giữa Windows thử nghiệm và Raspberry Pi.

3.4.2.5. Module state_store.py

‘state_store.py’ cung cấp lớp ‘StateStore’ lưu trạng thái vào file JSON bằng thao tác ghi nguyên tử qua tempfile rồi ‘os.replace’. Module này quản lý status, counters, esp32, raspberry_pi, last_detection, logs, pending_command và payload dashboard.

3.4.2.6. Module uart_client.py

‘uart_client.py’ tạo lớp ‘UartClient’ chạy thread đọc serial, tự reconnect theo danh sách cổng, gửi line UTF-8 kết thúc bằng newline và parse phản hồi thành ‘UartEvent’. Parser nhận ACK, DONE, ERROR, SENSOR và STATUS.

3.4.2.7. Module firebase_sync.py

‘firebase_sync.py’ định nghĩa ‘FirebaseSync’, chạy best-effort mirror. Hàm ‘sync_once()’ lấy ‘dashboard_payload()’ rồi cập nhật các node Firebase đã khai báo trong config. Lỗi Firebase chỉ được ghi trạng thái, không can thiệp vào luồng phân loại thời gian thực.

3.4.2.8. Firmware ESP32 DEVKIT V1

Firmware Arduino/ESP32 đọc lệnh từ Serial2 và Serial USB, parse theo dòng kết thúc newline. Các lệnh chính gồm SORT:<class>, BIN:<class>, CONVEYOR: START, CONVEYOR: STOP, SPEED:<value>, HOME, RESET_COUNTER và STATUS. Firmware dùng class ‘DebouncedInput’ để lọc cảm biến, task FreeRTOS ‘conveyorStepperTask()’ để phát xung stepper và các hàm ‘handleSortCommand()’, ‘handleBinCommand()’, ‘startServoMove()’ để điều khiển servo.

3.5 THIẾT KẾ LOGIC NHẬN DIỆN THỜI GIAN THỰC

3.5.1 Vấn đề nhận diện sai khi vật di chuyển liên tục

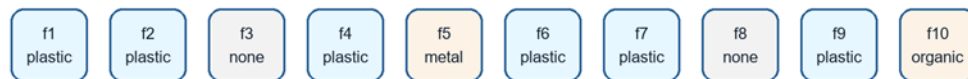
Khi vật di chuyển trên băng tải, khung hình có thể chỉ chứa một phần vật thể ở biên ảnh hoặc hai vật thể nằm gần nhau. Nhãn có thể dao động qua từng frame, đặc biệt với rác hữu cơ và nhựa có màu sắc tương tự. Nếu gửi lệnh phân loại ngay từ một frame đơn, servo có thể chuyển sai thùng hoặc gửi nhiều lệnh cho cùng một vật.

3.5.2 Cơ chế Detection Window và Voting

Hệ thống không dùng một dự đoán đơn mà thu thập vote trong cửa sổ thời gian $W = 2.5$ giây theo ‘DETECTION_WINDOW_SEC’. Camera mặc định 20 FPS

và 'ENTRY_DETECT_EVERY_N_FRAMES = 2', nên số frame suy luận xấp xỉ $N = 2,5 \times 20 / 2 = 25$. Mỗi kết quả hợp lệ di thuộc tập {organic, plastic, metal} được cộng vào 'detection_votes'; lớp thắng là $\text{argmax}(\text{count})$.

Điều kiện xác nhận là số vote của lớp thắng $\geq$ 'MIN_VOTES_TO_CONFIRM = 3' và $\text{dominant ratio} = \text{vote_winner} / \text{total_valid_votes} \geq$ 'MIN_DOMINANT_RATIO = 0.50'. Confidence tốt nhất của lớp thắng được lưu trong 'detection_best_confidence' và dùng làm confidence của job.



winner = plastic; dominant ratio = 6/10

Trong hệ thống thật: cửa sổ 2,5 s, detect mỗi 2 frame, ngưỡng ratio = 0,5

Hình 3. 10 Minh họa cơ chế voting trong detection window

3.5.3 Debounce, Confidence threshold và Dominant ratio

Ngưỡng YOLO_CONFIDENCE và MIN_CONFIDENCE đều có giá trị mặc định 0,35. Detection có confidence thấp hơn sẽ không được đưa vào vote. Entry sensor debounce ở Raspberry Pi là 0,25 s, clear debounce 0,15 s, exit sensor debounce 0,2 s; firmware ESP32 DEVKIT V1 cũng lọc cảm biến với SENSOR_DEBOUNCE_MS = 40 ms. Cách lọc hai tầng giúp giảm trigger giả do nhiễu hoặc vật che cảm biến kéo dài.

3.5.4 Cảm biến entry/exit và sơ đồ trạng thái

Cảm biến entry và exit được sử dụng để đồng bộ quá trình nhận diện, tạo job phân loại và điều khiển cơ cấu chấp hành. Cảm biến entry có nhiệm vụ phát hiện vật thể đi vào vùng xử lý, từ đó kích hoạt một phiên nhận diện mới. Khi cảm biến

entry được kích hoạt, hệ thống tạm dừng băng tải trong một khoảng thời gian ngắn để vật thể ổn định trong khung hình, sau đó bắt đầu quá trình nhận diện bằng mô hình YOLOv11/NCNN.

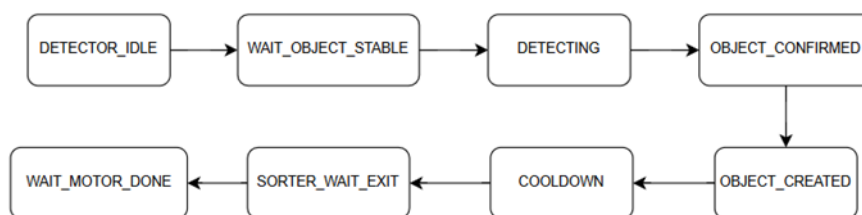
Trong giai đoạn nhận diện, kết quả từ nhiều khung hình liên tiếp được tích lũy trong cửa sổ thời gian và xử lý bằng cơ chế voting. Khi lớp rác được xác nhận, hệ thống tạo một job phân loại tương ứng và đưa vào hàng đợi. Sau đó, băng tải tiếp tục di chuyển để đưa vật thể đến vị trí cơ cấu phân loại.

Cảm biến exit được đặt gần cuối băng tải nhằm xác định thời điểm vật thể đi tới vùng phân loại. Khi exit sensor được kích hoạt và trong hàng đợi có job hợp lệ, Raspberry Pi gửi lệnh phân loại dạng `SORT:<class>` đến ESP32 DevKit V1 thông qua UART. ESP32 điều khiển servo quay đến vị trí thùng tương ứng và phản hồi trạng thái hoàn tất bằng tín hiệu `DONE: SORT`. Sau khi nhận phản hồi, Raspberry Pi cập nhật bộ đếm, lưu trạng thái và cho phép hệ thống quay lại trạng thái chờ vật thể tiếp theo.

Bảng 3. 5 Bảng chuyển trạng thái

Trạng thái hiện tại	Tín hiệu	Điều kiện	Trạng thái tiếp theo	Hành động
DETECTOR_IDLE	Entry sensor kích hoạt	Hàng đợi chưa đầy	WAIT_OBJECT_STABLE	Dừng băng tải, bắt đầu phiên nhận diện
WAIT_OBJECT_STABLE	Hết thời gian ổn định	Entry còn hợp lệ	DETECTING	Bắt đầu voting
DETECTING	Hết detection window	Kết quả voting đạt ngưỡng	OBJECT_CONFIRMED	Xác nhận lớp rác

OBJECT_CONFIRMED	Tạo job	Job hợp lệ	OBJECT_CREATE	Đưa job vào hàng đợi
OBJECT_CREATE	Job đã tạo	—	SORTER_WAIT_EXIT	Cho băng tải chạy tiếp
SORTER_WAIT_EXIT	Exit sensor kích hoạt	Hàng đợi có job	SORT_COMMAND_SENT	Gửi lệnh SORT:<class>
SORT_COMMAND_SENT	ESP32 phản hồi ACK	Lệnh hợp lệ	WAIT_MOTOR_DONE	Chờ servo hoàn tất
WAIT_MOTOR_DONE	ESP32 phản hồi DONE: SORT	Job khớp	DETECTOR_IDLE	Tăng bộ đếm, lưu trạng thái



Hình 3. 11 Sơ đồ trạng thái hệ thống nhận diện

Hình 3.11 mô tả các trạng thái chính trong quá trình nhận diện và phân loại rác. Hệ thống bắt đầu ở trạng thái chờ vật thể. Khi cảm biến entry phát hiện vật thể, hệ thống chuyển sang trạng thái chờ vật thể ổn định, sau đó thực hiện nhận diện và xác nhận lớp bằng cơ chế voting. Khi lớp rác được xác nhận, một job phân loại được tạo và lưu vào hàng đợi. Hệ thống tiếp tục chờ cảm biến exit kích hoạt để gửi lệnh phân loại đến ESP32 DevKit V1. Sau khi ESP32 phản hồi hoàn tất, hệ thống cập nhật bộ đếm và quay lại trạng thái chờ.

3.6 THIẾT KẾ CƠ SỞ DỮ LIỆU FIREBASE

3.6.1 Cấu trúc cây dữ liệu

```
{
  "trash_sorting_system": {
    "status": {
      "system_status": "RUNNING",
      "runtime_state": "DETECTOR_IDLE/SORTER_IDLE",
      "camera_status": "ONLINE",
      "uart_status": "CONNECTED",
      "fps": 0.0,
      "queue_depth": 0,
      "current_class": "unknown",
      "confirmed_class": null
    },
    "raspberrypi": {
      "online": true,
      "last_seen_ms": 0,
      "fps": 0.0,
      "ai_enabled": true
    },
    "esp32": {
      "online": true,
      "last_seen_ms": 0,
      "sensor_entry": false,
      "sensor_exit": false,
      "current_bin": "organic"
    },
    "counters": {
      "organic": 0,
      "metal": 0,
      "plastic": 0,
      "total": 0
    },
    "last_detection": {
      "class_name": "plastic",
```

```
    "confidence": 0.92,  
    "object_id": 1,  
    "source": "entry",  
    "votes": {  
        "plastic": 6  
    }  
}  
}
```

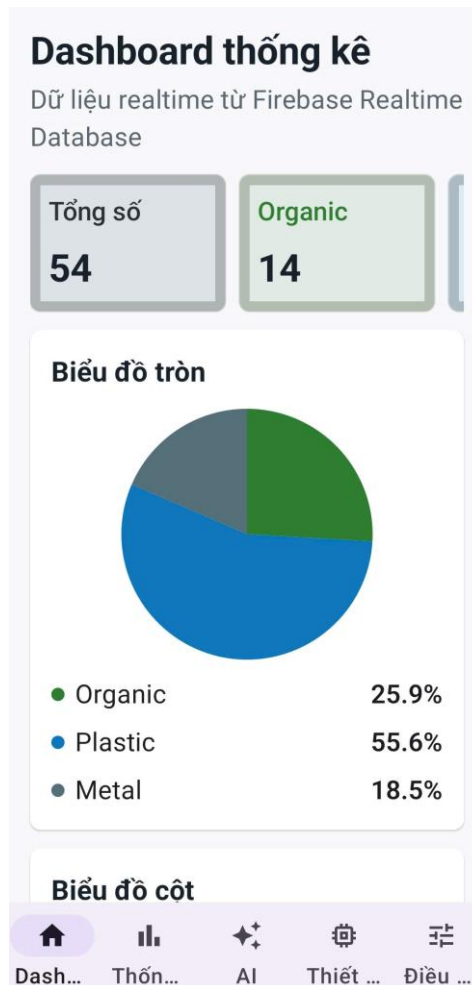
3.6.2 Thiết kế schema lưu thống kê phân loại

Khác với schema giả định có node history riêng, mã nguồn hiện tại mirror các node thực tế gồm status, raspberry_pi, esp32, counters và last_detection. Counter chỉ tăng sau khi ESP32 Devkit V1 trả DONE cho lệnh SORT/BIN hợp lệ, nhờ đó thống kê phản ánh vật đã được cơ cấu xử lý chứ không chỉ là dự đoán YOLO.

3.7 THIẾT KẾ DASHBOARD VÀ ỨNG DỤNG ANDROID

3.7.1 Giao diện dashboard thống kê

Giao diện dashboard được thiết kế nhằm hiển thị các thông tin tổng quan của hệ thống, bao gồm: tổng số vật thể đã phân loại, số lượng vật thể theo từng lớp, biểu đồ tròn biểu diễn tỷ lệ phân bố rác, biểu đồ thống kê theo thời gian và danh sách các lần nhận diện gần nhất. Bên cạnh đó, hệ thống cung cấp các API phục vụ giám sát, điều khiển và kiểm tra trạng thái, bao gồm: /api/status, /api/events, /api/command, /api/reset_counter và /api/firebase-debug.



Hình 3. 12 Dashboard thống kê

Hình 3.12 minh họa giao diện dashboard thống kê của hệ thống phân loại rác tự động. Giao diện này được thiết kế nhằm hiển thị dữ liệu phân loại được đồng bộ từ Firebase Realtime Database theo thời gian thực. Phần đầu giao diện hiển thị các thẻ thống kê tổng quan, bao gồm tổng số vật thể đã được phân loại và số lượng của từng nhóm rác như organic, plastic và metal.

Bên dưới là biểu đồ tròn thể hiện tỷ lệ phân bố giữa các nhóm rác. Thông qua biểu đồ này, người dùng có thể nhanh chóng quan sát nhóm rác nào chiếm tỷ lệ cao hơn trong quá trình vận hành hệ thống. Ngoài ra, dashboard còn có khu vực

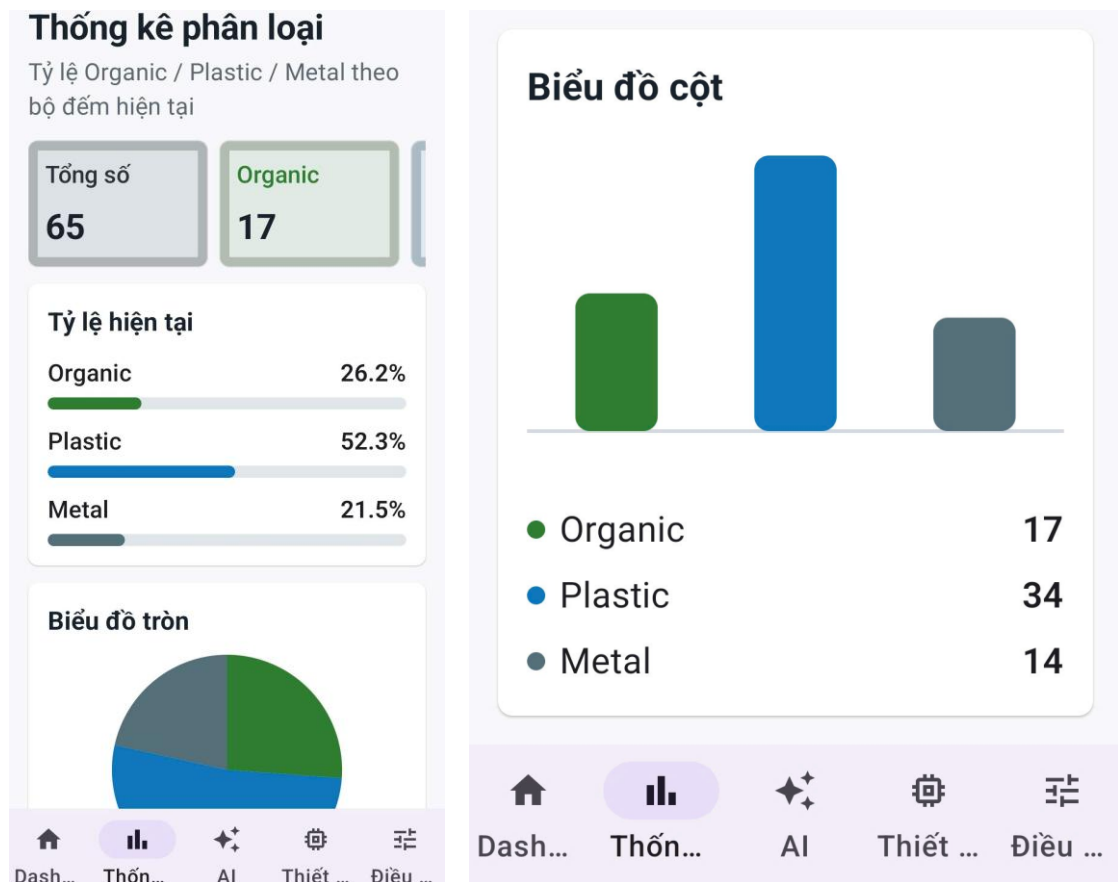
biểu đồ cột và các tab chức năng ở thanh điều hướng phía dưới, phục vụ việc xem thống kê, phân tích AI, thiết lập hệ thống và điều khiển.

Giao diện dashboard giúp người dùng theo dõi trạng thái phân loại một cách trực quan, hỗ trợ đánh giá hiệu quả hoạt động của hệ thống và cung cấp dữ liệu phục vụ thống kê sau quá trình thử nghiệm.

3.7.2 Phác thảo ứng dụng Android

Ứng dụng Android được xây dựng nhằm hỗ trợ người dùng theo dõi hoạt động của hệ thống phân loại rác từ xa. Ứng dụng kết nối với Firebase Realtime Database để hiển thị tổng số vật thể đã phân loại, số lượng theo từng nhóm rác gồm organic, plastic và metal, đồng thời thể hiện dữ liệu bằng các thẻ thống kê và biểu đồ trực quan.

Bên cạnh chức năng thống kê, ứng dụng còn hiển thị trạng thái hoạt động của Raspberry Pi, ESP32 DevKit V1, cảm biến, băng tải và kết quả nhận diện gần nhất. Giao diện cũng được định hướng bổ sung cảnh báo khi thiết bị mất kết nối hoặc số lượng rác trong ngăn chứa đạt ngưỡng cần xử lý. Trong phạm vi khóa luận, ứng dụng tập trung vào chức năng giám sát và hiển thị dữ liệu; các chức năng điều khiển từ xa và phân tích nâng cao sẽ được tiếp tục hoàn thiện trong hướng phát triển của đề tài.



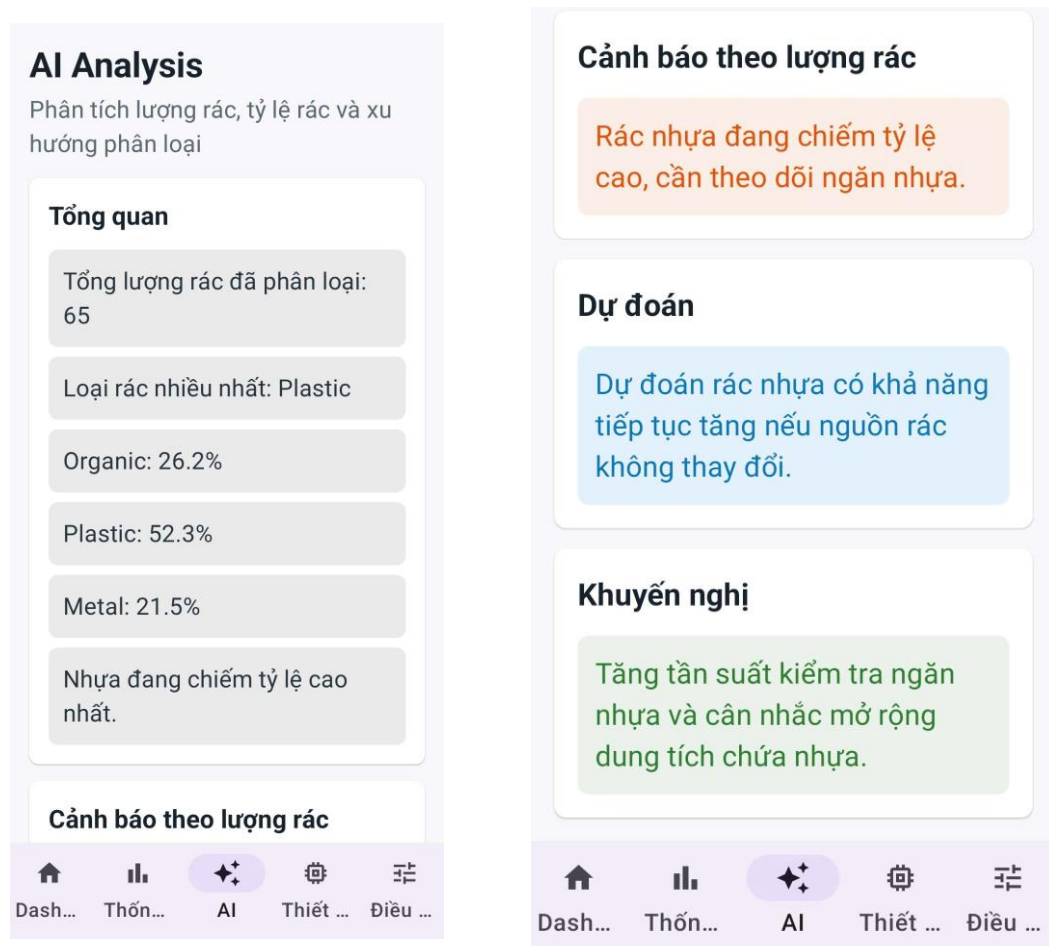
Hình 3. 13 Giao diện thống kê phân loại của ứng dụng Android

Hình 3.13 minh họa giao diện thống kê của ứng dụng Android. Giao diện hiển thị tổng số rác đã phân loại, tỷ lệ hiện tại của các nhóm organic, plastic và metal, cùng với biểu đồ tròn và biểu đồ cột để trực quan hóa dữ liệu thống kê. Các số liệu này được đồng bộ từ Firebase Realtime Database nhằm hỗ trợ người dùng theo dõi hiệu quả hoạt động của hệ thống.

Màn hình AI Analysis được thiết kế nhằm hỗ trợ người dùng đánh giá xu hướng phân loại rác dựa trên dữ liệu hiện có. Giao diện này tổng hợp các thông tin như tổng lượng rác đã phân loại, loại rác xuất hiện nhiều nhất, tỷ lệ từng nhóm rác và

đưa ra một số nhận xét tổng quan. Trong ví dụ minh họa, hệ thống xác định rằng plastic là loại rác có tỷ lệ cao nhất.

Bên cạnh đó, ứng dụng còn đưa ra cảnh báo, dự đoán và khuyến nghị. Chẳng hạn, nếu rác nhựa chiếm tỷ lệ lớn trong một khoảng thời gian, hệ thống có thể sinh cảnh báo rằng ngăn chứa rác nhựa cần được theo dõi. Đồng thời, phần dự đoán có thể cho biết xu hướng tiếp tục tăng của rác nhựa nếu nguồn rác đầu vào không thay đổi. Từ đó, ứng dụng đưa ra khuyến nghị như tăng tần suất kiểm tra ngăn chứa hoặc cân nhắc mở rộng dung tích của ngăn tương ứng.



Hình 3. 14 Giao diện phân tích bằng AI và cảnh báo

Hình 3. 14 minh họa giao diện phân tích AI của ứng dụng Android. Giao diện này tổng hợp số liệu thống kê, xác định loại rác chiếm tỷ lệ cao nhất, đồng thời đưa ra cảnh báo, dự đoán và khuyến nghị nhằm hỗ trợ người dùng giám sát hệ thống hiệu quả hơn.

Màn hình trạng thái phần cứng được thiết kế để hiển thị tình trạng hoạt động của các thành phần chính trong hệ thống, bao gồm ESP32 DevKit V1, Raspberry Pi, camera, cảm biến và pipeline xử lý. Thông qua giao diện này, người dùng có thể biết được thiết bị nào đang ở trạng thái online/offline, trạng thái băng tải đang chạy hay dừng, giá trị tốc độ, trạng thái cảm biến entry/exit, trạng thái home switch và thời điểm cập nhật gần nhất.

Đối với khối ESP32 DevKit V1, ứng dụng hiển thị các thông tin như trạng thái hoạt động hiện tại, thùng phân loại đang được chọn, tốc độ băng tải, trạng thái conveyor, trạng thái cảm biến entry/exit và công tắc home. Đối với Raspberry Pi, ứng dụng thể hiện các thông tin như trạng thái online, tình trạng AI và số khung hình xử lý trên giây (FPS). Ngoài ra, phần Pipeline cho biết trạng thái camera, trạng thái detector, pha nhận diện, nguồn kích hoạt detection và trạng thái phiên làm việc hiện tại. Nhờ đó, người dùng có thể đánh giá nhanh toàn bộ hệ thống có đang vận hành ổn định hay không.



Hình 3. 15 Giao diện giám sát trạng thái phần cứng

Hình 3.15 trình bày giao diện giám sát trạng thái phần cứng của ứng dụng Android. Giao diện hiển thị thông tin hoạt động của ESP32 DevKit V1, Raspberry Pi và pipeline xử lý, bao gồm trạng thái online/offline, trạng thái băng tải, cảm biến, FPS và thời điểm cập nhật gần nhất. Màn hình này giúp người dùng kiểm tra tình trạng vận hành của hệ thống một cách trực quan.

Trong phạm vi khóa luận, ứng dụng Android mới dừng ở mức phác thảo giao diện và định hướng chức năng, chưa triển khai hoàn chỉnh toàn bộ khả năng phân tích và điều khiển. Tuy nhiên, các giao diện đã thể hiện rõ hướng phát triển của hệ thống theo mô hình giám sát thông minh, trong đó người dùng có thể theo dõi dữ liệu phân loại, kiểm tra trạng thái phản ứng và nhận các phân tích hỗ trợ từ xa thông qua thiết bị di động.

3.8 KẾT LUẬN CHƯƠNG

Trong chương này, nhóm thực hiện đã thiết kế kiến trúc tổng thể của hệ thống phân loại rác tự động tại bãi tập kết sau thu gom. Hệ thống gồm khối băng tải tiếp nhận rác, khối camera và Raspberry Pi thực hiện nhận diện, khối ESP32 DevKit V1 điều khiển cảm biến và servo, cùng khối Firebase phục vụ lưu trữ và giám sát dữ liệu. Bên cạnh đó, các cơ chế detection window, voting, debounce và hàng đợi xử lý được xây dựng nhằm hạn chế sai lệch khi rác di chuyển liên tục trên băng tải. Các thiết kế này là cơ sở để triển khai phần cứng, huấn luyện mô hình và lập trình hệ thống trong chương tiếp theo.

CHƯƠNG 4: THI CÔNG, LẬP TRÌNH VÀ HUẤN LUYỆN MÔ HÌNH

Sau khi hoàn thiện thiết kế hệ thống, chương này trình bày quá trình thi công mô hình và xây dựng chương trình điều khiển phục vụ việc phân loại rác tại bãi tập kết. Nội dung chương bao gồm lắp ráp băng tải, bố trí camera và cảm biến, thu thập dữ liệu từ các mẫu rác, gán nhãn và huấn luyện mô hình YOLOv11. Bên cạnh đó, mô hình sau huấn luyện được chuyển đổi sang định dạng NCNN để triển khai trên Raspberry Pi, đồng thời firmware ESP32 DevKit V1 được xây dựng để điều khiển servo và tiếp nhận tín hiệu cảm biến trong quá trình phân loại.

4.1 THI CÔNG PHẦN CỨNG

4.1.1 Lắp ráp khung cơ khí và băng tải

Khung cơ khí được lắp theo dạng băng tải thí nghiệm, bảo đảm camera nhìn rõ vùng phân loại và vật thể di chuyển ổn định. Băng tải cần được căng vừa đủ để tránh trượt, đồng thời không tạo rung lớn làm mờ ảnh. Cụm thùng phân loại được bố trí sau vùng exit sensor để servo có đủ thời gian chuyển hướng vật.



Hình 4. 1 Hình ảnh thực tế khung cơ khí

Hình 4.1 minh họa mô hình thực tế của hệ thống sau khi lắp ráp, bao gồm khung cơ khí, băng tải, camera, cảm biến và cụm servo phân loại. Băng tải có nhiệm vụ đưa vật thể đi qua vùng quan sát của camera, sau đó đến khu vực phân loại. Camera được cố định phía trên băng tải để thu nhận hình ảnh, còn cụm servo ở cuối băng tải thực hiện chuyển hướng vật thể vào các ngăn rác tương ứng. Hình ảnh này cho thấy bố trí tổng thể của hệ thống phục vụ quá trình kiểm thử thực nghiệm.

4.1.2 Lắp đặt camera và các cảm biến

Camera được cố định phía trên băng tải, hướng vuông góc hoặc gần vuông góc với mặt băng để giảm méo hình. Entry sensor đặt trước vùng camera để kích hoạt detection session; exit sensor đặt gần cơ cấu phân loại để đảm bảo lệnh servo được gửi đúng thời điểm vật đến vị trí gạt.



Hình 4. 2 Lắp đặt camera và cảm biến

Hình 4.2 minh họa vị trí lắp đặt camera và các cảm biến trên mô hình băng tải phân loại rác. Camera được cố định phía trên băng tải bằng giá đỡ, hướng quan sát xuống vùng di chuyển của vật thể để thu nhận hình ảnh phục vụ quá trình nhận diện. Cách bố trí này giúp camera quan sát tương đối đầy đủ khu vực làm việc, hạn chế sai lệch góc nhìn và đảm bảo vật thể nằm trong vùng nhận diện của mô hình YOLOv11.

Các cảm biến được bố trí dọc theo băng tải để đồng bộ quá trình nhận diện và phân loại. Cảm biến entry được đặt trước vùng quan sát của camera nhằm phát hiện vật thể đi vào, từ đó kích hoạt phiên nhận diện mới. Cảm biến exit được đặt gần khu vực cơ cấu phân loại để xác định thời điểm vật thể đến vị trí cần gạt. Khi cảm biến exit được kích hoạt, hệ thống gửi lệnh điều khiển servo nhằm đưa vật thể vào đúng ngăn rác tương ứng.

Việc bố trí camera và cảm biến như trong Hình 4.2 giúp hệ thống xác định đúng thời điểm thu nhận ảnh, xử lý nhận diện và điều khiển cơ cấu phân loại. Nhờ đó, quá trình vận hành giữa băng tải, camera, cảm biến và servo được đồng bộ hơn trong quá trình thử nghiệm thực tế.

4.1.3 Đấu nối ESP32 DEVKIT-V1, servo và nguồn điện

ESP32 DEVKIT-V1 giao tiếp Raspberry Pi qua UART2 ở 115200 bps. Servo nên dùng nguồn riêng phù hợp dòng tải, nối chung GND với ESP32 DEVKIT-V1 và Raspberry Pi để tín hiệu PWM/UART có cùng mức tham chiếu. Stepper driver dùng các chân STEP/DIR/EN theo firmware và cần nguồn động cơ riêng.



Hình 4. 3 Sơ đồ đấu nối thực tế

Hình 4.3 minh họa phần đấu nối thực tế của khối điều khiển sử dụng ESP32 DevKit V1. Trong mô hình, ESP32 DevKit V1 được đặt trên mạch kết nối trung tâm, đảm nhiệm vai trò nhận lệnh từ Raspberry Pi thông qua UART, đọc tín hiệu cảm biến và điều khiển các cơ cấu chấp hành. Các dây tín hiệu được đấu nối đến

các chân GPIO tương ứng để điều khiển servo phân loại, driver động cơ băng tải và các cảm biến entry/exit.

Bên cạnh ESP32 DevKit V1 là driver điều khiển động cơ bước, dùng để nhận các tín hiệu STEP, DIR và EN từ ESP32 nhằm điều khiển chuyển động của băng tải. Khối nguồn được bố trí riêng để cấp điện cho các tải như servo và động cơ, trong khi các thiết bị trong hệ thống được nối chung GND để bảo đảm cùng mức tham chiếu điện áp. Việc nối chung mass giữa ESP32, Raspberry Pi, servo, driver động cơ và nguồn ngoài giúp tín hiệu PWM và UART hoạt động ổn định hơn trong quá trình vận hành.

Hình ảnh này cho thấy quá trình lắp ráp thực tế của phần điều khiển phần cứng, bao gồm mạch ESP32, driver động cơ, đầu nối nguồn và các dây tín hiệu. Đây là cơ sở để kiểm thử các chức năng như giao tiếp UART, điều khiển servo, điều khiển băng tải và đọc trạng thái cảm biến trong hệ thống phân loại rác tự động.

4.2 THU THẬP VÀ TIỀN XỬ LÝ DỮ LIỆU HUẤN LUYỆN

4.2.1 Thu thập ảnh các nhóm rác

Bộ dữ liệu sử dụng trong lần huấn luyện mới được khai báo trong args.yaml tại đường dẫn

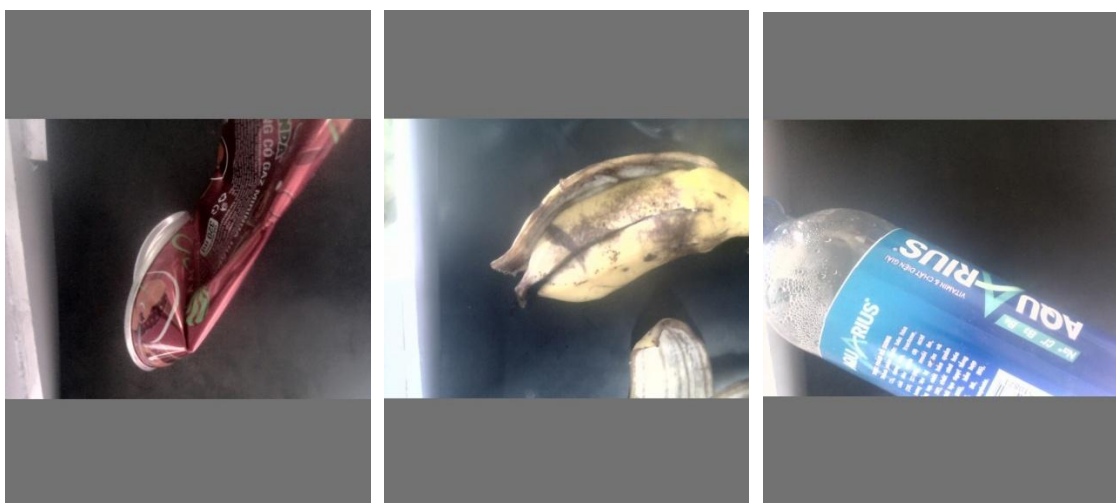
/content/drive/MyDrive/KLTN_YOLO11/trash_detection_2_6/data.yaml, gồm ba lớp rác: metal, organic và plastic. Dữ liệu được thu thập từ nhiều nguồn khác nhau, bao gồm ảnh chụp trực tiếp bằng camera của hệ thống, ảnh bổ sung từ các vật mẫu và ảnh trong các điều kiện nền, góc nhìn, ánh sáng khác nhau. Việc thu thập dữ liệu đa dạng giúp mô hình có khả năng học được nhiều đặc trưng của vật thể, từ đó cải thiện khả năng nhận diện trong quá trình thử nghiệm thực tế.

Trong thư mục kết quả huấn luyện mới, không có data.yaml hoặc log thể hiện số ảnh gốc theo từng lớp, nên báo cáo không cập nhật lại số lượng ảnh train/val/test bằng phỏng đoán. Thay vào đó, phân bố nhãn được lấy trực tiếp từ labels.jpg: metal có 294 instance, organic có 359 instance và plastic có 293 instance, tổng

cộng 946 instance. Nhìn chung, số nhãn giữa ba lớp tương đối cân bằng, trong đó organic có số lượng cao nhất.

Bảng 4.1 Số lượng nhãn đối tượng theo từng lớp

Nhãn	Số instance	Nguồn	Tỉ lệ
metal	294	labels.jpg	31,1%
organic	359	labels.jpg	38,0%
plastic	293	labels.jpg	31,0%
Tổng	946		100%



Hình 4. 4 Ảnh mẫu từng nhóm rác

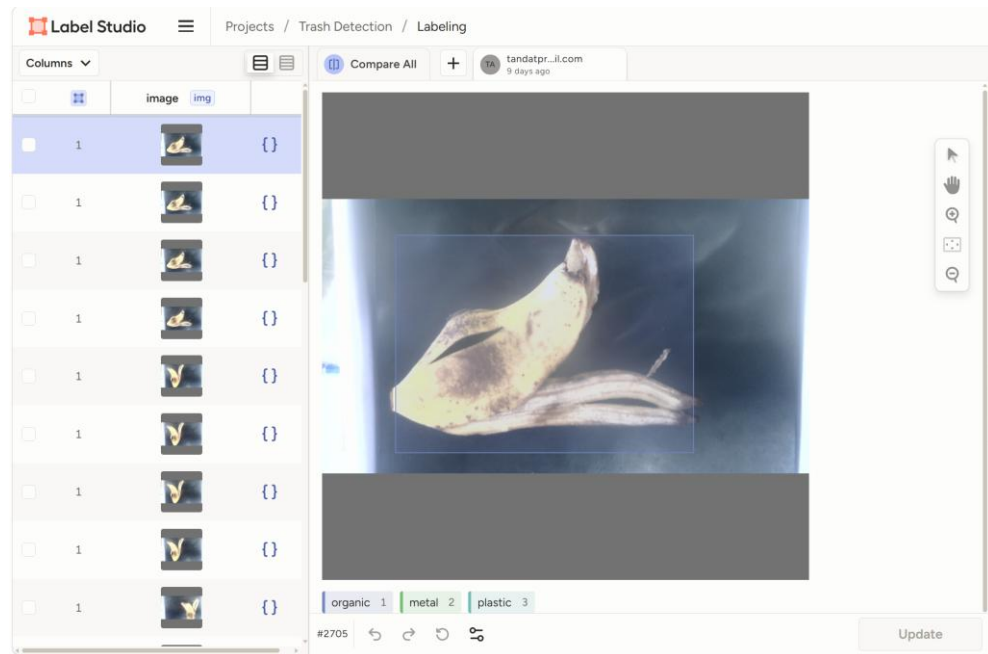
Hình 4.4 minh họa một số ảnh mẫu trong bộ dữ liệu dùng để huấn luyện mô hình nhận diện rác. Các ảnh được thu thập từ môi trường thực nghiệm với góc chụp từ camera đặt phía trên băng tải. Bộ dữ liệu gồm ba nhóm chính: organic, plastic và metal. Trong đó, ảnh mẫu của nhóm organic thể hiện các loại rác hữu cơ như vỏ chuối; nhóm plastic gồm các vật thể nhựa như chai nhựa; và nhóm metal gồm các vật thể kim loại như lon nước.

Các ảnh mẫu có sự khác nhau về vị trí vật thể, góc đặt, độ sáng và nền ảnh. Điều này giúp mô hình YOLOv11 học được đặc trưng của từng nhóm rác trong nhiều điều kiện quan sát khác nhau. Việc xây dựng bộ dữ liệu thực tế từ chính môi trường băng tải giúp mô hình phù hợp hơn với điều kiện vận hành của hệ thống sau khi triển khai.

4.2.2 Gán nhãn dữ liệu

Dữ liệu được gán nhãn theo định dạng YOLO: mỗi ảnh có file .txt tương ứng, mỗi dòng gồm class_id, x_center, y_center, width, height sau chuẩn hóa về khoảng 0-1 [17]. Thứ tự lớp trong ‘data.yaml’ là 0: metal, 1: organic, 2: plastic.

Quá trình kiểm tra nhãn cần đảm bảo bbox ôm sát vật thể, không nhầm lớp và không bỏ sót vật trong vùng quan sát.



Hình 4. 5 Giao diện gán nhãn

Hình 4.5 minh họa giao diện gán nhãn dữ liệu bằng công cụ Label Studio. Trong quá trình xây dựng bộ dữ liệu, mỗi ảnh rác được mở trên giao diện gán nhãn và đối tượng cần nhận diện được khoanh vùng bằng bounding box. Sau khi vẽ hộp bao quanh vật thể, người gán nhãn chọn lớp tương ứng cho đối tượng, gồm organic, metal hoặc plastic.

Dữ liệu sau khi gán nhãn được xuất theo định dạng YOLO, trong đó mỗi ảnh có một tệp nhãn .txt tương ứng. Mỗi dòng trong tệp nhãn gồm mã lớp và tọa độ hộp bao đã được chuẩn hóa về khoảng từ 0 đến 1, bao gồm: class_id, x_center, y_center, width và height. Cách biểu diễn này giúp mô hình YOLOv11 có thể học được cả thông tin về lớp đối tượng và vị trí của đối tượng trong ảnh.

Trong quá trình gán nhãn, cần đảm bảo bounding box ôm sát vật thể, không bỏ sót các đối tượng nằm trong vùng quan sát và không gán sai lớp. Chất lượng gán

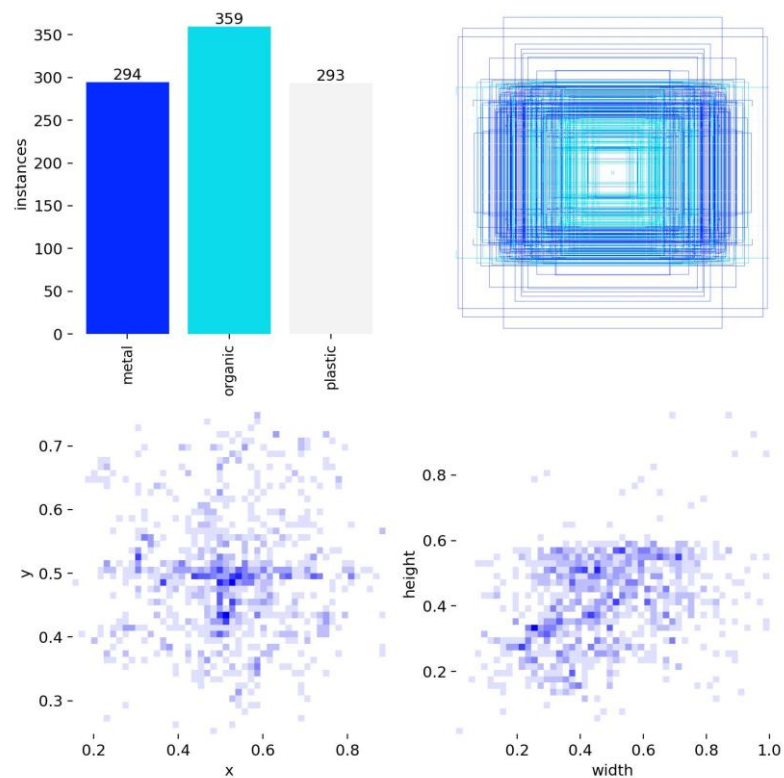
nhân ảnh hưởng trực tiếp đến kết quả huấn luyện mô hình, do đó bước kiểm tra và hiệu chỉnh nhân được thực hiện nhằm giảm sai lệch dữ liệu trước khi đưa vào huấn luyện.

4.2.3 Data augmentation và phân chia tập dữ liệu

Theo args.yaml, quá trình huấn luyện sử dụng các kỹ thuật augmentation như HSV, translate, scale, shear, flip trái phải, mosaic, mixup và random erasing [18]. Tập args.yaml ghi nhận mô hình được đánh giá trên split val; không thấy thông tin về tập test riêng trong thư mục kết quả.

Bảng 4.4 Phân chia tập dữ liệu theo kết quả huấn luyện mới

Tập	Số ảnh	Tỉ lệ	Mục đích
train	821	80%	Huấn luyện
val	206	20%	Đánh giá validation theo split=val



Hình 4. 6 Phân bố nhãn trong tập dữ liệu

Hình 4.6 trình bày thống kê tổng quan về nhãn trong bộ dữ liệu dùng để huấn luyện mô hình YOLOv11. Biểu đồ cột ở góc trên bên trái thể hiện số lượng instance theo từng lớp, gồm metal, organic và plastic. Trong đó, lớp organic có 359 instance, lớp metal có 294 instance và lớp plastic có 293 instance. Kết quả này cho thấy số lượng nhãn giữa ba lớp tương đối cân bằng.

Biểu đồ ở góc trên bên phải thể hiện sự phân bố của các bounding box sau khi chuẩn hóa. Các hộp bao được chồng lên nhau nhằm quan sát kích thước và vị trí tương đối của vật thể trong ảnh. Phần lớn bounding box tập trung ở vùng trung tâm ảnh, cho thấy vật thể thường xuất hiện trong vùng quan sát chính của camera trên băng tải. Điều này phù hợp với cách bố trí camera cố định phía trên băng tải trong mô hình thực nghiệm.

Biểu đồ phân tán ở góc dưới bên trái biểu diễn vị trí tâm của bounding box theo hai trục x và y. Các điểm tập trung nhiều quanh vùng giữa ảnh, phản ánh rằng phần lớn vật thể được đặt hoặc di chuyển qua khu vực trung tâm khung hình. Trong khi đó, biểu đồ ở góc dưới bên phải thể hiện mối quan hệ giữa width và height của bounding box. Sự phân bố này cho thấy kích thước vật thể trong bộ dữ liệu khá đa dạng, giúp mô hình học được đặc trưng của các vật thể có kích thước khác nhau.

Nhìn chung, hình thống kê cho thấy bộ dữ liệu có đủ ba lớp rác cần nhận diện, các bounding box tập trung chủ yếu trong vùng quan sát của camera và có sự đa dạng về kích thước vật thể. Do thư mục kết quả huấn luyện mới không kèm file thống kê số ảnh theo từng tập, phần này chỉ sử dụng số lượng instance đọc được từ labels.jpg và không suy luận thêm số ảnh train/val.

4.3 HUẤN LUYỆN MÔ HÌNH YOLOV11

4.3.1 Cấu hình môi trường và siêu tham số

Quá trình huấn luyện được thực hiện trên nền tảng Ultralytics YOLO với trọng số khởi tạo yolo11s.pt. Việc sử dụng mô hình khởi tạo sẵn giúp rút ngắn thời gian huấn luyện và tận dụng các đặc trưng đã học từ tập dữ liệu lớn trước đó. Trong cấu hình huấn luyện, mô hình sử dụng kích thước ảnh đầu vào 640×640 pixel, batch size bằng 16, thiết bị huấn luyện device = 0, bộ tối ưu AdamW, epochs = 150, patience = 40 và cos_lr = True. Các thông số này được ghi nhận trong tệp args.yaml sinh ra sau quá trình huấn luyện. Sau khi hoàn tất, checkpoint tốt nhất được lựa chọn và chuyển đổi sang định dạng NCNN để triển khai trên Raspberry Pi. Theo tệp metadata của mô hình sau khi export, phiên bản Ultralytics được sử dụng trong quá trình chuyển đổi là 8.4.60 [19].

Bảng 4.5 Siêu tham số huấn luyện

Tham số	Giá trị	Giải thích
epochs	150	Giá trị đọc từ args.yaml
imgsz	640	Giá trị đọc từ args.yaml
batch	16	Giá trị đọc từ args.yaml
lr0	0.001	Giá trị đọc từ args.yaml
lrf	0.01	Giá trị đọc từ args.yaml
momentum	0.937	Giá trị đọc từ args.yaml
weight_decay	0.0005	Giá trị đọc từ args.yaml
optimizer	AdamW	Giá trị đọc từ args.yaml
model	yolo11s.pt	Giá trị đọc từ args.yaml
device	0	Giá trị đọc từ args.yaml

4.3.2 Quá trình huấn luyện và theo dõi

Mô hình YOLOv11 được huấn luyện trong 150 epoch với các tham số đã thiết lập. Trong quá trình huấn luyện, hệ thống ghi nhận các giá trị loss và các chỉ số đánh giá trên tập validation để lựa chọn checkpoint phù hợp. Checkpoint tốt nhất được xác định tại epoch 140 theo kết quả validation và được lưu dưới dạng best.pt. Kết quả chi tiết của quá trình huấn luyện và đánh giá mô hình được trình bày tại Chương 5.



Hình 4. 7 Ảnh batch huấn luyện mẫu sau khi áp dụng augmentation

Hình 4.7 minh họa một batch ảnh huấn luyện mẫu được tạo trong quá trình huấn luyện mô hình YOLOv11. Hình gồm nhiều ảnh rác được ghép lại trong cùng một khung, thể hiện các đối tượng thuộc ba lớp nhận diện gồm metal, organic và plastic. Trên mỗi vật thể có các bounding box và mã lớp tương ứng, trong đó các nhãn số biểu thị class ID của từng loại rác theo cấu hình dữ liệu YOLO.

Các ảnh trong batch có sự thay đổi về vị trí, kích thước, góc xoay, độ sáng và nền ảnh. Đây là kết quả của các kỹ thuật tăng cường dữ liệu như mosaic, xoay, dịch chuyển, thay đổi tỉ lệ, biến đổi màu sắc và các phép biến đổi hình học khác trong quá trình huấn luyện. Việc tăng cường dữ liệu giúp mô hình tiếp xúc với nhiều tình huống khác nhau, từ đó cải thiện khả năng tổng quát hóa khi nhận diện rác trong môi trường thực tế.

Hình này cũng cho thấy các bounding box được vẽ quanh vật thể để mô hình học đồng thời hai nhiệm vụ: xác định lớp rác và xác định vị trí vật thể trong ảnh. Nhờ đó, YOLOv11 có thể phát hiện các vật thể rác trên băng tải và cung cấp thông tin đầu ra phục vụ cho logic phân loại của hệ thống.

4.3.3 Lựa chọn checkpoint tốt nhất

Ultralytics lưu best.pt và last.pt trong thư mục weights của lần huấn luyện mới. Dựa trên results.csv và train_metrics trong best.pt, checkpoint tốt nhất tương ứng epoch 140, đạt Precision = 0,9497, Recall = 0,9631, mAP50 = 0,9783 và mAP50-95 = 0,7590. last.pt là trọng số ở epoch cuối, còn best.pt được dùng để export NCNN và triển khai runtime.

4.4 CHUYỂN ĐỔI MÔ HÌNH SANG ĐỊNH DẠNG NCNN

4.4.1 Quy trình export từ PyTorch sang NCNN

Sau khi hoàn tất quá trình huấn luyện, mô hình có kết quả tốt nhất được lưu dưới dạng tệp trọng số best.pt. Đây là định dạng mô hình của PyTorch, phù hợp cho quá trình huấn luyện và kiểm thử trên máy tính. Tuy nhiên, để triển khai trên Raspberry Pi, mô hình cần được chuyển đổi sang định dạng NCNN nhằm giảm phụ thuộc thư viện và tối ưu tốc độ suy luận trên CPU ARM [4], [5].

Quá trình chuyển đổi mô hình được thực hiện bằng thư viện Ultralytics YOLO thông qua phương thức export(). Đoạn mã sử dụng trong đề tài như sau:

```
from ultralytics import YOLO

MODEL_PATH =
"/content/drive/MyDrive/KLTN_YOLO11/runs4/trash_plastic_metal_organic_
2_06/weights/best.pt"
model = YOLO(MODEL_PATH)

model.export(

    format="ncnn",

    imgsz=416,
```

```
half=False,  
simplify=True  
)
```

Trong đoạn mã trên, biến `MODEL_PATH` trỏ đến tệp trọng số `best.pt` sau huấn luyện. Mô hình được nạp vào thông qua lệnh `YOLO(MODEL_PATH)`, sau đó được export sang định dạng NCNN bằng tham số `format="ncnn"`.

Tham số `imgsz=416` quy định kích thước ảnh đầu vào khi triển khai là 416×416 pixel. Việc giảm kích thước đầu vào so với cấu hình huấn luyện giúp giảm chi phí tính toán, phù hợp hơn với khả năng xử lý của Raspberry Pi. Tham số `half=False` cho biết mô hình không sử dụng kiểu dữ liệu FP16, nhằm bảo đảm khả năng tương thích khi chạy trên CPU ARM. Tham số `simplify=True` được sử dụng để đơn giản hóa đồ thị mô hình trong quá trình chuyển đổi, giúp quá trình suy luận ổn định và tối ưu hơn.

Sau khi export, Ultralytics tạo ra thư mục `best_ncnn_model` chứa các tệp phục vụ triển khai, bao gồm `model.ncnn.param`, `model.ncnn.bin`, `metadata.yaml` và `model_ncnn.py`. Trong đó, tệp `model.ncnn.param` mô tả cấu trúc mạng, tệp `model.ncnn.bin` chứa trọng số của mô hình, còn `metadata.yaml` lưu các thông tin cấu hình như tác vụ detect, kích thước ảnh 416×416 , `batch = 1`, `stride = 32` và danh sách lớp gồm `metal`, `organic`, `plastic`. Các tệp này được đưa vào môi trường chạy trên Raspberry Pi để thực hiện nhận diện rác theo thời gian thực.

4.4.2 Kiểm tra tính tương thích trên Raspberry Pi

Trong quá trình triển khai trên Raspberry Pi, runtime trong file `detect.py` được thiết kế để tự động lựa chọn backend suy luận phù hợp. Nếu đường dẫn mô hình trỏ đến một thư mục chứa hai tệp `model.ncnn.param` và `model.ncnn.bin`, chương trình sẽ nhận diện đây là mô hình định dạng NCNN và khởi tạo quá trình suy luận bằng backend NCNN.

Cụ thể, hàm `_load_ncnn()` chịu trách nhiệm khởi tạo đối tượng `ncnn.Net()`, cấu hình số luồng xử lý thông qua tham số `NCNN_NUM_THREADS = 4`, sau đó lần lượt nạp tệp mô tả kiến trúc mạng `model.ncnn.param` và tệp trọng số `model.ncnn.bin`. Việc thiết lập số luồng giúp tận dụng khả năng xử lý đa nhân của CPU trên Raspberry Pi, từ đó cải thiện tốc độ suy luận.

Trong quá trình chạy, ảnh đầu vào sau khi tiền xử lý được đưa vào mạng thông qua tensor đầu vào `in0`. Sau khi mô hình thực hiện suy luận, kết quả được lấy ra từ tensor đầu ra `out0`. Kết quả này tiếp tục được giải mã để tạo ra các bounding box, độ tin cậy và lớp dự đoán tương ứng, phục vụ cho logic voting và quá trình phân loại rác của hệ thống.

Bảng 4. 1 So sánh hiệu năng .pt vs NCNN

Định dạng	FPS trung bình	RAM sử dụng	CPU%	Ghi chú
.pt	Chưa đo trong artifact	Cao hơn	Cao	Phù hợp kiểm thử PC hơn
NCNN	Mục tiêu ≥ 5 FPS	Thấp hơn	Tối ưu ARM	Định dạng triển khai thực tế

4.5 LẬP TRÌNH PYTHON TRÊN RASPBERRY PI

4.5.1 Module `detect.py` – nhận diện đối tượng

Pipeline nhận diện của hệ thống gồm các bước chính: nạp backend suy luận, tiền xử lý frame, chạy mô hình YOLOv11/NCNN, giải mã kết quả bounding box, áp dụng NMS, lọc theo vùng nhận diện và vẽ overlay kết quả lên ảnh. Với mỗi frame đầu vào từ camera, hàm `detect()` thực hiện chuẩn hóa dữ liệu ảnh, đưa ảnh vào mô hình để suy luận, sau đó chuyển đầu ra của mô hình thành các thông tin

gồm tọa độ hộp bao, độ tin cậy và lớp dự đoán. Các kết quả trùng lặp được loại bỏ bằng NMS, đồng thời chỉ những vật thể nằm trong vùng nhận diện hợp lệ mới được giữ lại để phục vụ logic voting và phân loại. Đoạn mã dưới đây là hàm detect() thực tế được sử dụng trong file detect.py.

```
def detect(self, frame: Any) -> tuple[list[Detection], Any]:
    if frame is None:
        self.last_all_detections = []
        self.last_active_detections = []
        self.last_rejected_detections = []
        return [], None
    self.frame_index += 1
    if config.MOCK_DETECTION:
        detection =
Detection(config.normalize_class(config.MOCK_CLASS), 0.99,
self._mock_bbox(frame))
        active_detections = self._mark_detection_zone(frame,
[detection])
        self.last_all_detections = [detection]
        self.last_active_detections = list(active_detections)
        self.last_rejected_detections = [item for item in
[detection] if not item.below_line]
        self.tracker.update(active_detections, self.frame_index)
        return active_detections, self.draw(frame, [detection])
    if not self.enabled or self.model is None:
        self.last_all_detections = []
        self.last_active_detections = []
        self.last_rejected_detections = []
        return [], self._draw_message(frame, self.last_error or
"YOLO not ready")
    try:
        if self.backend == "ncnn":
            detections = self._predict_ncnn(frame)
        else:
            detections = self._predict_ultralytics(frame)
    except Exception as exc:
        self.last_error = f"DETECTION_INFERENCE_ERROR:{exc}"
        logger.exception(self.last_error)
        self.last_all_detections = []
        self.last_active_detections = []
        self.last_rejected_detections = []
        return [], self._draw_message(frame, self.last_error)
    active_detections = self._mark_detection_zone(frame,
detections)
    self.last_all_detections = list(detections)
    self.last_active_detections = list(active_detections)
    self.last_rejected_detections = [detection for detection in
detections if not detection.below_line]
    self.tracker.update(active_detections, self.frame_index)
    return active_detections, self.draw(frame, detections)
```

4.5.2 Module main.py và logic điều phối

Runtime chính đọc frame, điều khiển trạng thái detector/sorter, ghi vote, xác nhận lớp và gửi lệnh UART. Vòng lặp loop() là trung tâm phối hợp camera, YOLO và state machine.

```
def _loop(self) -> None:
    last_frame_at = 0.0
    while self.running:
        self._consume_web_command()
        frame = self.camera.read()
        if frame is None:

self.store.update_status(camera_status=self.camera.status,
uart_status="CONNECTED" if self.uart.connected else "DISCONNECTED",
last_error=self.camera.last_error)
            if self.detector_state in
{DETECTOR_WAIT_OBJECT_STABLE, DETECTOR_DETECTING}:
                self.store.add_log(
                    f"[DETECT] camera frame unavailable
status={self.camera.status} error={self.camera.last_error}",
                    "ERROR",
                )
                self._check_timeout()
                time.sleep(0.1)
                continue

            self.frame_index += 1
            now = time.time()
            if last_frame_at > 0:
                instant_fps = 1.0 / max(0.001, now - last_frame_at)
                self.fps = instant_fps if self.fps <= 0 else
(self.fps * 0.85) + (instant_fps * 0.15)
                last_frame_at = now

            annotated = frame
            detections: list[Detection] = []
            entry_detect_due = self.frame_index -
self.last_entry_detect_frame >= max(1,
config.ENTRY_DETECT_EVERY_N_FRAMES)

            if self.detector_state == DETECTOR_WAIT_OBJECT_STABLE:
                annotated = self.detector.draw_idle(frame)
                if self.entry_trigger_time > 0 and now -
self.entry_trigger_time >= config.OBJECT_STABLE_DELAY_SEC:
                    self._start_voting(now)
            elif self.detector_state == DETECTOR_DETECTING:
                if self.ai_enabled and entry_detect_due:
                    self.last_entry_detect_frame = self.frame_index
```



```

        detections, annotated =
self.detector.detect(frame)
        self._log_detect_debug(frame, detections)
        if detections:
            self._record_detection_vote(detections[0])
        else:
            annotated = self.detector.draw_idle(frame)
            if self.detect_start_time > 0 and now -
self.detect_start_time >= config.DETECTION_WINDOW_SEC:
                if self._confirm_detection_votes():
                    self._create_object_once()
                else:
                    self.store.add_log("[ENTRY] Detection
failed: no confirmed class", "WARN")
                    self._reset_detector_cycle("detect timeout",
"ERROR")
            else:
                annotated = self.detector.draw_idle(frame)
                annotated = self._draw_runtime_overlay(annotated)
                self.camera.set_annotated(annotated)
                self._check_timeout()
                self._publish_local_status()
                time.sleep(config.MAIN_LOOP_SLEEP_SEC)

```

4.5.3 Module state_store.py – quản lý trạng thái

StateStore ghi trạng thái ra file JSON an toàn bằng tempfile rồi thay thế nguyên tử. Counter chỉ tăng cho ba lớp hợp lệ và total được tính lại từ các lớp.

```

def increment_counter(self, class_name: str) -> None:
    with self.lock:
        data = self._read()
        counters = data.setdefault("counters", {"organic": 0,
"metal": 0, "plastic": 0, "total": 0})
        if class_name in {"organic", "metal", "plastic"}:
            counters[class_name] =
as_int(counters.get(class_name), 0) + 1
            counters["total"] = as_int(counters.get("organic"), 0) +
as_int(counters.get("metal"), 0) + as_int(counters.get("plastic"),
0)
        self.write(data)

```

4.5.4 Module uart_client.py – giao tiếp serial

UART client gửi mỗi lệnh như một dòng UTF-8 kết thúc bằng newline. Khi không kết nối được, client thử reconnect theo danh sách cổng cấu hình.

```

def send_line(self, line: str) -> bool:
    text = str(line or "").strip()
    if not text:
        self.last_error = "UART empty line"

```

```

        return False
    if not self._ensure_connected():
        return False
    try:
        with self.lock:
            self.serial.write((text + "\n").encode("utf-8"))
            self.serial.flush()
            self.logger.info("UART -> ESP32: %s", text)
            return True
    except Exception as exc:
        self.last_error = f"UART_WRITE_FAILED:{exc}"
        self.logger.exception(self.last_error)
        self.close()
        return False

```

4.5.5 Module firebase_sync.py – đồng bộ dữ liệu

FirestoreSync đồng bộ định kỳ các node trạng thái. Lỗi kết nối chỉ cập nhật trạng thái Firestore, không làm dừng luồng phân loại.

```

def sync_once(self) -> bool:
    if not config.FIREBASE_ENABLED:
        self.store.set_firebase_status(False, "disabled")
        return False
    if not self._connect():
        self.store.set_firebase_status(False, self.last_error)
        return False
    try:
        payload = self.store.dashboard_payload()

        self.db.reference(config.FIREBASE_STATUS_PATH).update(payload["status"])

        self.db.reference(config.FIREBASE_RASPBERRY_PI_PATH).update(payload["raspberry_pi"])

        self.db.reference(config.FIREBASE_ESP32_PATH).update(payload["esp32"])

        self.db.reference(config.FIREBASE_COUNTER_PATH).set(payload["counter"])

        if payload.get("last_detection"):
            self.db.reference(config.FIREBASE_LAST_DETECTION_PATH).set(payload["last_detection"])
            self.last_error = ""
            self.store.set_firebase_status(True, "")
            return True
    except Exception as exc:
        self.last_error = f"FIREBASE_SYNC_FAILED:{exc}"
        self.logger.warning(self.last_error)
        self.store.set_firebase_status(False, self.last_error)
        return False

```

4.5.6 Module config.py và app.py

Config chứa các ngưỡng thời gian thực, còn app.py chuyển lệnh dashboard thành dòng UART. Các hằng số quan trọng được trích dưới đây.

```
UART_BAUDRATE = int(os.getenv("UART_BAUDRATE", "115200"))
YOLO_CONFIDENCE = float(os.getenv("YOLO_CONFIDENCE", "0.35"))
YOLO_IMGSZ = int(os.getenv("YOLO_IMGSZ", "416"))
ENTRY_SENSOR_DEBOUNCE_SEC =
float(os.getenv("ENTRY_SENSOR_DEBOUNCE_SEC", "0.25"))
EXIT_SENSOR_DEBOUNCE_SEC =
float(os.getenv("EXIT_SENSOR_DEBOUNCE_SEC", "0.2"))
DETECTION_WINDOW_SEC = float(os.getenv("DETECTION_WINDOW_SEC",
"2.5"))
MIN_VOTES_TO_CONFIRM = int(os.getenv("MIN_VOTES_TO_CONFIRM", "3"))
MIN_CONFIDENCE = float(os.getenv("MIN_CONFIDENCE", "0.35"))
MIN_DOMINANT_RATIO = float(os.getenv("MIN_DOMINANT_RATIO", "0.5"))
SORTING_COMMAND_PREFIX = os.getenv("SORTING_COMMAND_PREFIX",
"SORT").strip().upper()
if SORTING_COMMAND_PREFIX not in {"SORT", "BIN"}:
    SORTING_COMMAND_PREFIX = "SORT"
    "uart": {"esp32_rx2": 16, "esp32_tx2": 17, "baudrate":
UART_BAUDRATE},
```

4.6 LẬP TRÌNH FIRMWARE ESP32 DEVKIT-V1

4.6.1 Nhận lệnh qua UART và giải mã giao thức

Firmware đọc Serial2 và Serial USB bằng hàm readStream(). Khi gặp ký tự newline, buffer được đưa vào handleCommand(). Lệnh được chuẩn hóa uppercase để so khớp, trong khi class_name được đưa về lowercase.

```
#include <Arduino.h>
#include <ESP32Servo.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// ESP32 realtime UART firmware for Trash Sorting System.
// Raspberry Pi UART:
//   Pi TX -> ESP32 RX2 GPIO16
//   Pi RX -> ESP32 TX2 GPIO17
//   GND   -> ESP32 GND
//
// Bin actuator:
//   Servo 180 signal -> GPIO18
//   home position    -> organic
//
// Detection LED:
//   LED -> GPIO2
```

```

//
// Home button:
//   Button -> GPIO34, press to return servo to organic angle.
//   GPIO34 has no internal pull-up/pull-down, so use an external
//   resistor.

constexpr uint32_t UART_BAUDRATE = 115200;

constexpr int UART_RX2_PIN = 16;
constexpr int UART_TX2_PIN = 17;

constexpr int CONVEYOR_STEP_PIN = 27;
constexpr int CONVEYOR_DIR_PIN = 26;
constexpr int CONVEYOR_EN_PIN = 25;

constexpr int BIN_SERVO_PIN = 18;
constexpr int DETECTED_LED_PIN = 2;
constexpr int HOME_BUTTON_PIN = 34;

constexpr int SENSOR_ENTRY_PIN = 32;
constexpr int SENSOR_EXIT_PIN = 33;

constexpr bool STEPPER_ENABLE_ACTIVE_LOW = true;
constexpr bool SENSOR_ACTIVE_LOW = true;
constexpr bool HOME_BUTTON_ACTIVE_LOW = true;
constexpr bool DETECTED_LED_ACTIVE_HIGH = true;

constexpr float CONVEYOR_DEFAULT_SPEED_SPS = 140.0f;
constexpr float CONVEYOR_MIN_SPEED_SPS = 40.0f;
constexpr float CONVEYOR_MAX_SPEED_SPS = 600.0f;
constexpr bool CONVEYOR_FORWARD_POSITIVE = true;
constexpr bool STEPPER_HOLD_WHEN_STOPPED = false;
constexpr uint32_t A4988_STEP_HIGH_US = 4;
constexpr uint32_t STEPPER_TASK_IDLE_MS = 5;
constexpr uint32_t STEPPER_TASK_STACK_WORDS = 2048;
constexpr UBaseType_t STEPPER_TASK_PRIORITY = 3;
constexpr BaseType_t STEPPER_TASK_CORE = 0;

// 180-degree servo bin positions. Adjust these values on hardware
// if needed.
constexpr int SERVO_ORGANIC_ANGLE = 90;
constexpr int SERVO_PLASTIC_ANGLE = 5;
constexpr int SERVO_METAL_ANGLE = 175;
constexpr uint32_t SERVO_MOVE_MS = 700;
constexpr uint32_t DETECTED_LED_HOLD_MS = 1500;

constexpr uint32_t SENSOR_DEBOUNCE_MS = 40;
constexpr uint32_t STATUS_INTERVAL_MS = 1000;
constexpr uint32_t SENSOR_REPORT_INTERVAL_MS = 250;
// Entry is reported as both a one-shot object event and the live
// beam state.
constexpr uint32_t ENTRY_OBJECT_TIMEOUT_MS = 8000;
constexpr uint32_t ENTRY_REARM_CLEAR_MS = 150;

enum class RunState : uint8_t {
    IDLE,

```

```

    RUNNING,
    SORTING_SERVO,
    BIN_SERVO,
    ERROR_STATE
};

class DebouncedInput {
public:
    void begin(int gpioPin, bool activeLow, uint8_t mode) {
        pin = gpioPin;
        inverted = activeLow;
        pinMode(pin, mode);
        stableState = readActive();
        lastRawState = stableState;
        changedAt = millis();
    }

    void update() {
        const bool raw = readActive();
        const uint32_t now = millis();
        roseEvent = false;
        fellEvent = false;

        if (raw != lastRawState) {
            lastRawState = raw;

```

4.6.2 Điều khiển servo theo nhóm rác

Servo được điều khiển bằng thư viện ESP32Servo. Hàm `angleForBin()` ánh xạ lớp sang góc quay; `startServoMove()` ghi góc mới và chờ đủ `SERVO_MOVE_MS` trước khi gửi DONE.

```

#include <Arduino.h>
#include <ESP32Servo.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// ESP32 realtime UART firmware for Trash Sorting System.
// Raspberry Pi UART:
//   Pi TX -> ESP32 RX2 GPIO16
//   Pi RX -> ESP32 TX2 GPIO17
//   GND   -> ESP32 GND
//
// Bin actuator:
//   Servo 180 signal -> GPIO18
//   home position   -> organic
//
// Detection LED:
//   LED -> GPIO2
//
// Home button:
//   Button -> GPIO34, press to return servo to organic angle.
//   GPIO34 has no internal pull-up/pull-down, so use an external

```

```

resistor.

constexpr uint32_t UART_BAUDRATE = 115200;

constexpr int UART_RX2_PIN = 16;
constexpr int UART_TX2_PIN = 17;

constexpr int CONVEYOR_STEP_PIN = 27;
constexpr int CONVEYOR_DIR_PIN = 26;
constexpr int CONVEYOR_EN_PIN = 25;

constexpr int BIN_SERVO_PIN = 18;
constexpr int DETECTED_LED_PIN = 2;
constexpr int HOME_BUTTON_PIN = 34;

```

Bảng 4. 2 Góc servo tương ứng từng nhóm rác

Nhóm rác	Góc servo (độ)	Thời gian giữ (ms)	Ghi chú
organic	90	700	Vị trí home
plastic	5	700	Thùng nhựa
metal	175	700	Thùng kim loại

4.6.3 Xử lý phản hồi và trạng thái cơ cấu

Trong quá trình giao tiếp UART, ESP32 DevKit V1 phản hồi về Raspberry Pi theo các dạng lệnh văn bản nhằm xác nhận trạng thái thực thi. Khi nhận được lệnh hợp lệ, ESP32 gửi phản hồi ACK:<command> để xác nhận lệnh đã được tiếp nhận. Sau khi hoàn tất quá trình điều khiển, ví dụ như quay servo đến vị trí phân loại hoặc xử lý xong một tác vụ, ESP32 gửi phản hồi DONE:<command>. Trong trường hợp xảy ra lỗi trong quá trình nhận lệnh hoặc thực thi, firmware gửi lệnh ERROR:<message> để Raspberry Pi ghi nhận và xử lý.

Ngoài các lệnh phản hồi theo yêu cầu, firmware trên ESP32 DevKit V1 còn gửi định kỳ các lệnh cập nhật trạng thái của cảm biến và cơ cấu chấp hành. Lệnh

dạng `SENSOR:entry=...,entry_event=...,entry_active=...,exit=...,home=...` được dùng để cập nhật trạng thái cảm biến entry, cảm biến exit và công tắc home.

Trong khi đó, lệnh `STATUS:conveyor=...,current_bin=...,state=...,speed=...` cung cấp thông tin về trạng thái băng tải, thùng phân loại hiện tại, trạng thái hoạt động của ESP32 và tốc độ vận hành.

Các lệnh này giúp Raspberry Pi theo dõi được trạng thái thực tế của khối chấp hành, đồng bộ quá trình nhận diện với quá trình phân loại và cập nhật dữ liệu lên StateStore hoặc Firebase. Nhờ sử dụng giao thức dạng văn bản, quá trình kiểm tra, debug và mở rộng lệnh điều khiển cũng trở nên đơn giản hơn trong quá trình phát triển hệ thống

4.7 TÍCH HỢP VÀ KIỂM THỬ TOÀN HỆ THỐNG

Quá trình tích hợp bắt đầu bằng kiểm tra độc lập từng khối: camera stream, load model NCNN, UART loopback, servo movement, sensor debounce và Firebase connection. Sau đó hệ thống được chạy toàn chu trình: entry sensor kích hoạt, băng tải dừng, voting xác nhận lớp, job vào queue, exit sensor dispatch lệnh SORT và DONE làm tăng counter. Các tình huống kiểm thử gồm một vật đơn, hai vật liên tiếp, mất kết nối UART tạm thời, khởi động lại nguồn và Firebase chậm do Wi-Fi yếu.

4.8 KẾT LUẬN CHƯƠNG

Chương 4 đã trình bày quá trình hiện thực hệ thống từ thiết kế sang mô hình thực nghiệm. Khung cơ khí, băng tải, camera, cảm biến, Raspberry Pi, ESP32 DevKit V1 và servo đã được lắp ráp nhằm mô phỏng quá trình rác sau thu gom được đưa vào dây chuyền phân loại tại bãi tập kết. Bộ dữ liệu ba nhóm rác được xây dựng, gán nhãn và sử dụng để huấn luyện mô hình YOLOv11; sau đó mô hình được chuyển đổi sang định dạng NCNN để phục vụ triển khai thực tế. Đồng thời, các module phần mềm và firmware điều khiển đã được xây dựng để liên kết quá trình

nhận diện với cơ cấu phân loại. Đây là cơ sở để tiến hành thực nghiệm và đánh giá hiệu quả hệ thống ở Chương 5.

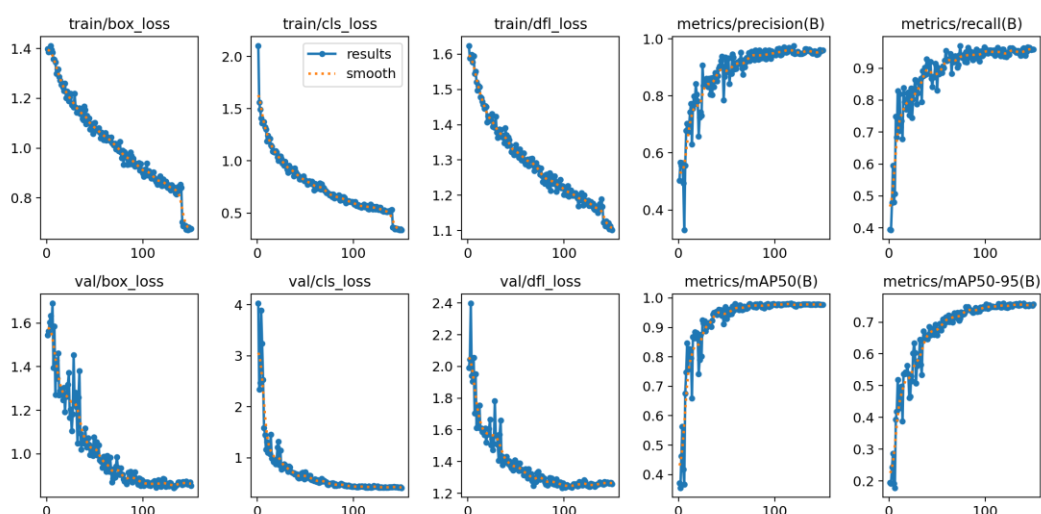
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Sau khi hoàn thành mô hình phân loại rác tự động, chương này trình bày các kết quả thực nghiệm nhằm đánh giá khả năng hoạt động của hệ thống trong điều kiện mô phỏng tại bãi tập kết. Các nội dung đánh giá bao gồm kết quả huấn luyện mô hình YOLOv11, tốc độ xử lý trên Raspberry Pi, khả năng phân loại các nhóm rác, ảnh hưởng của điều kiện ánh sáng, hiệu quả của cơ chế voting và detection window, giao tiếp UART, điều khiển servo và quá trình đồng bộ dữ liệu lên Firebase. Qua đó, mức độ đáp ứng của hệ thống đối với yêu cầu đề tài được xem xét và phân tích.

5.1 KẾT QUẢ HUẤN LUYỆN MÔ HÌNH YOLOV11

5.1.1 Biểu đồ loss qua các epoch

Để đánh giá quá trình huấn luyện mô hình nhận diện rác, các giá trị loss và các chỉ số đánh giá được theo dõi qua từng epoch. Kết quả huấn luyện của mô hình được thể hiện trong Hình 5.1.



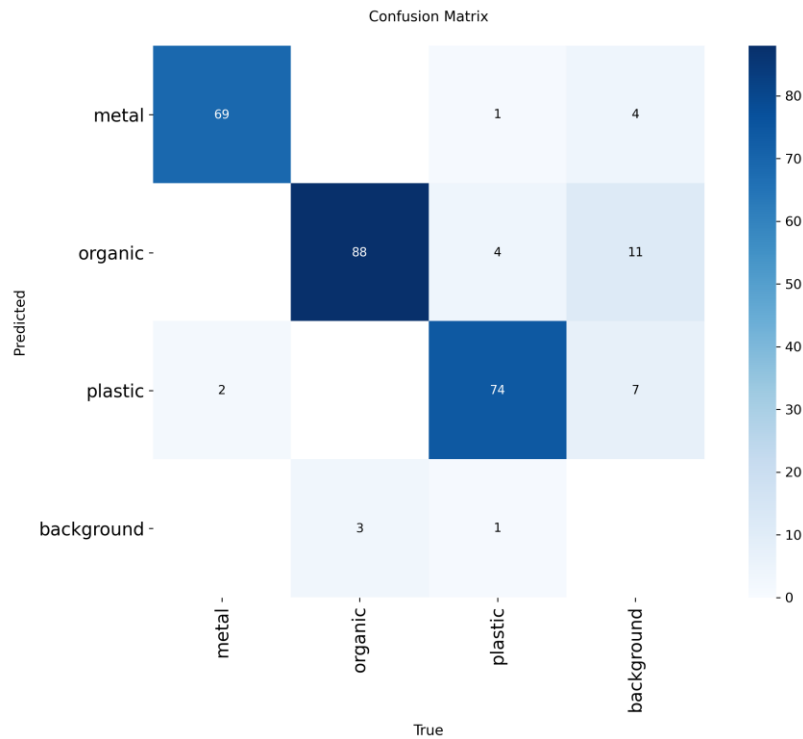
Hình 5.1 Biểu đồ kết quả huấn luyện

Hình 5.1 trình bày kết quả huấn luyện mô hình qua 150 epoch, bao gồm các hàm mất mát trên tập huấn luyện và tập kiểm định, cùng các chỉ số đánh giá như Precision, Recall, mAP@50 và mAP@50-95. Kết quả cho thấy các giá trị loss trên tập huấn luyện giảm dần theo thời gian. Cụ thể, train/box_loss giảm từ 1,398 xuống 0,676; train/cls_loss giảm từ 2,105 xuống 0,339; và train/df_l_loss giảm từ 1,624 xuống 1,101. Điều này cho thấy mô hình đã học được đặc trưng của các đối tượng trong dữ liệu huấn luyện.

Đối với tập kiểm định, các giá trị validation loss có dao động ở giai đoạn đầu nhưng nhìn chung có xu hướng giảm và ổn định về cuối quá trình huấn luyện. Checkpoint tốt nhất tại epoch 140 đạt Precision = 0,9497, Recall = 0,9631, mAP@50 = 0,9783 và mAP@50-95 = 0,7590. Kết quả này cho thấy mô hình có khả năng nhận diện và phân loại tốt ba nhóm rác trong tập validation.

5.1.2 Confusion matrix và phân tích nhầm lẫn lẫn

Ma trận nhầm lẫn được sử dụng để đánh giá chi tiết khả năng phân loại của mô hình đối với từng lớp đối tượng. Kết quả ma trận nhầm lẫn của mô hình trên tập kiểm định được trình bày trong Hình 5.2.



Hình 5.2 Confusion matrix

Quan sát hình 5.2 cho thấy mô hình phân loại đúng phần lớn các mẫu ở ba lớp chính gồm metal, organic và plastic. Các giá trị trên đường chéo chính của ma trận có giá trị cao, trong đó lớp organic đạt 88 mẫu đúng, lớp plastic đạt 74 mẫu đúng và lớp metal đạt 69 mẫu đúng. Điều này cho thấy mô hình đã học được đặc trưng nhận diện của từng nhóm rác và có khả năng phân biệt tương đối tốt giữa các lớp.

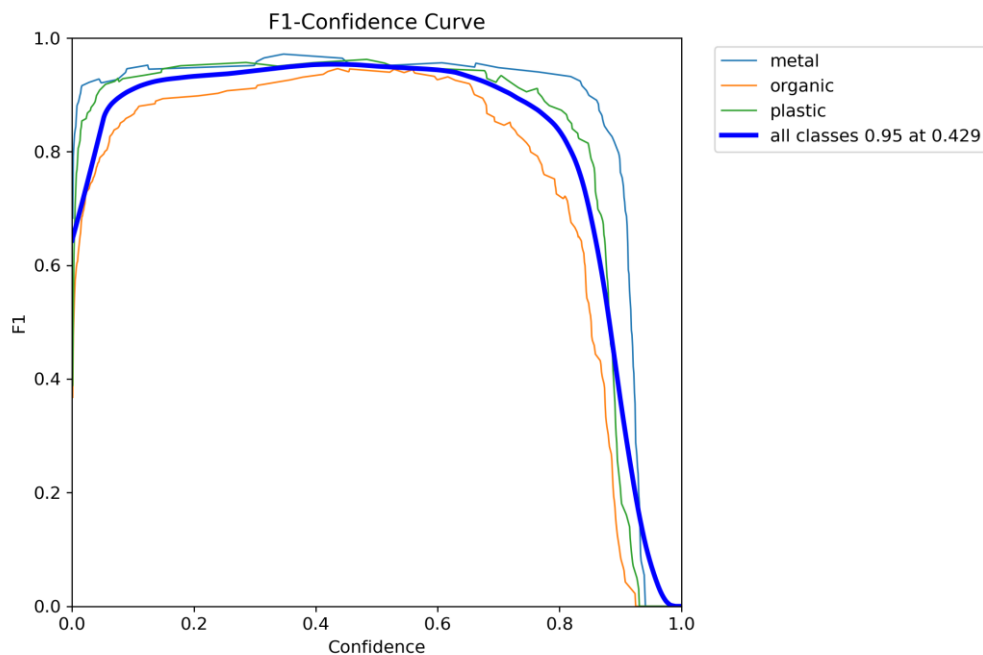
Tuy nhiên, vẫn tồn tại một số trường hợp nhầm lẫn giữa các lớp và với background. Cụ thể, có một số mẫu plastic bị dự đoán thành organic hoặc metal, một số mẫu organic bị đưa về background, đồng thời mô hình vẫn có dự đoán trên vùng background. Nguyên nhân có thể đến từ đặc điểm hình ảnh của dữ liệu rác, chẳng hạn như bề mặt vật thể phản xạ ánh sáng, vật thể bị che khuất một phần, góc chụp chưa ổn định hoặc nền băng tải có màu sắc gần giống với vật thể.

Nhìn chung, kết quả ma trận nhầm lẫn cho thấy mô hình đạt khả năng phân loại tốt, nhưng vẫn cần cải thiện thêm dữ liệu để giảm các trường hợp nhầm lẫn với background.

5.1.3 Đường cong F1, Precision, Recall theo confidence

Để đánh giá ảnh hưởng của ngưỡng confidence đến hiệu quả nhận diện của mô hình, các đường cong F1-Confidence, Precision-Confidence và Recall-Confidence được sử dụng. Các đường cong này cho phép xác định mối quan hệ giữa ngưỡng confidence và các chỉ số đánh giá, từ đó hỗ trợ lựa chọn ngưỡng nhận diện phù hợp cho hệ thống phân loại rác tự động.

Kết quả đường cong F1-Confidence được trình bày trong Hình 5.3.

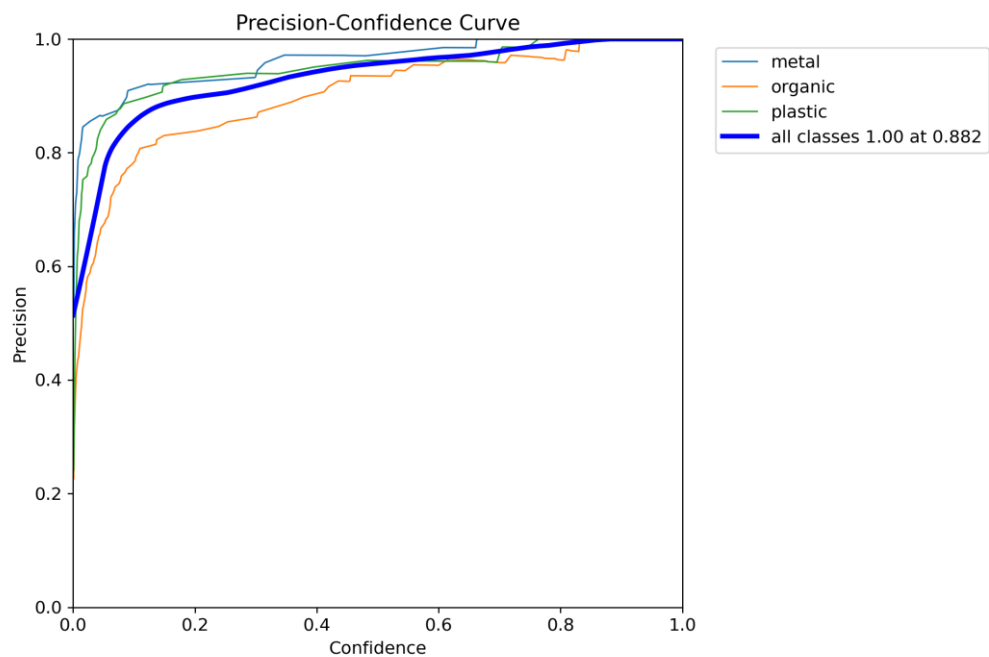


Hình 5.3 Kết quả đường cong F1-Confidence

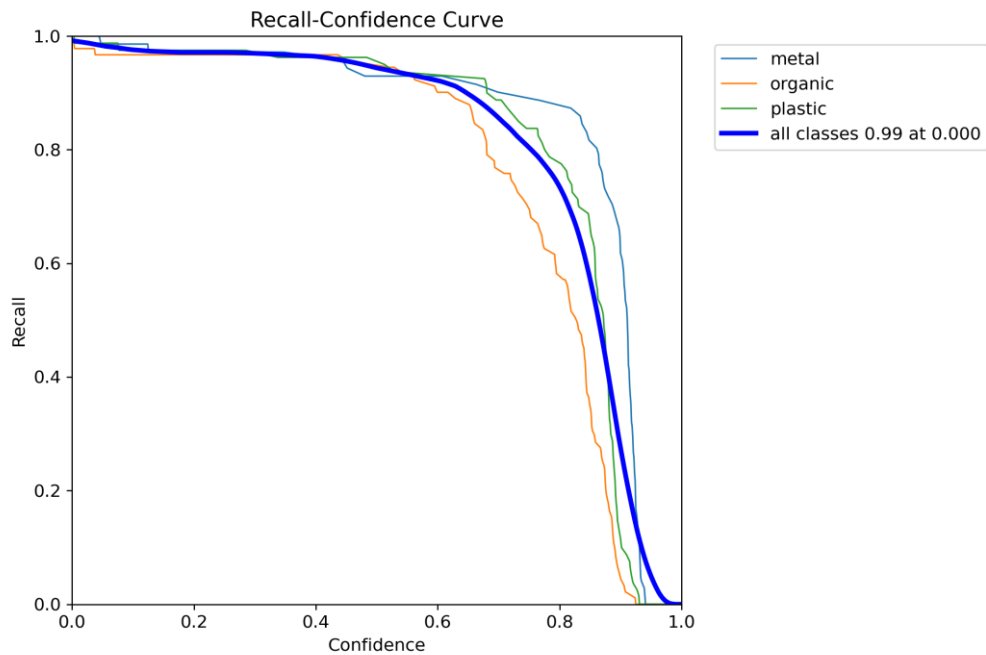
Quan sát hình 5.3 cho thấy giá trị F1-score của mô hình tăng nhanh khi ngưỡng confidence tăng từ mức thấp và đạt trạng thái ổn định trong khoảng confidence trung bình. Theo kết quả đánh giá, mô hình đạt F1-score cao nhất khoảng 0,95 tại

ngưỡng confidence khoảng 0,429. Điều này cho thấy tại ngưỡng này, mô hình đạt được sự cân bằng tốt giữa khả năng phát hiện đúng đối tượng và hạn chế dự đoán sai.

Bên cạnh đó, đường cong Precision-Confidence và Recall-Confidence được trình bày lần lượt trong Hình 5.4 và Hình 5.5.



Hình 5.4 Đường cong Precision-Confidence của mô hình



Hình 5.5 Đường cong Recall-Confidence của mô hình

Từ hình 5.4 có thể thấy, khi ngưỡng confidence tăng, chỉ số Precision có xu hướng tăng lên. Điều này là do mô hình chỉ giữ lại các dự đoán có độ tin cậy cao, từ đó làm giảm số lượng dự đoán sai. Tuy nhiên, việc đặt ngưỡng confidence quá cao có thể khiến một số đối tượng thật bị bỏ qua.

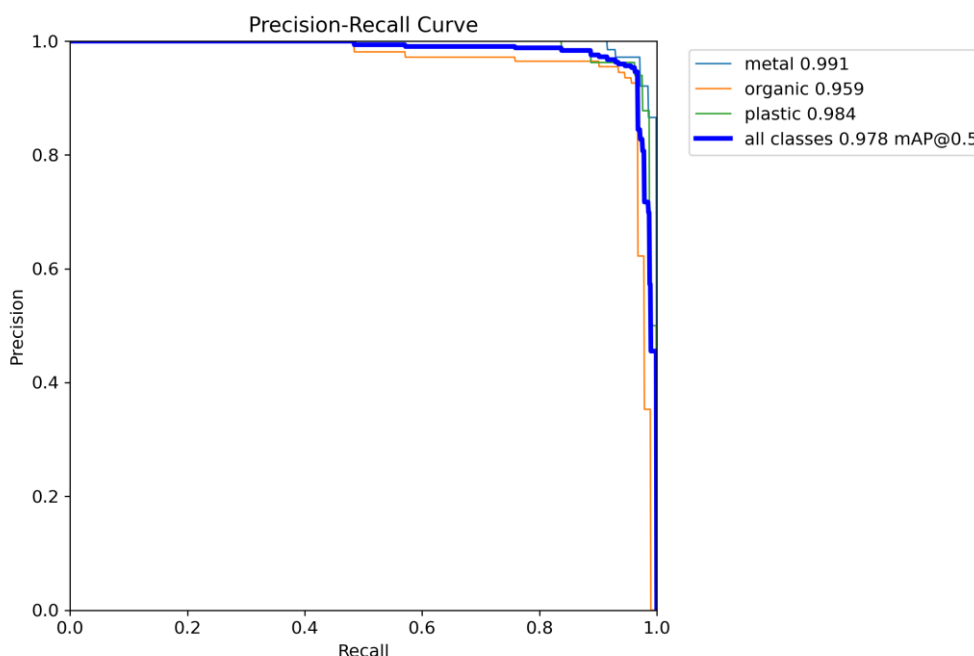
Ngược lại, hình 5.5 cho thấy chỉ số Recall có xu hướng giảm khi ngưỡng confidence tăng. Ở ngưỡng confidence thấp, mô hình phát hiện được nhiều đối tượng hơn, nhưng đồng thời có thể xuất hiện nhiều dự đoán sai. Khi ngưỡng confidence tăng, mô hình trở nên nghiêm ngặt hơn trong việc đưa ra dự đoán, làm giảm khả năng phát hiện các đối tượng có độ tin cậy thấp.

Từ các kết quả trên, có thể thấy việc lựa chọn ngưỡng confidence cần cân bằng giữa Precision và Recall. Trong hệ thống phân loại rác tự động, ngưỡng confidence không nên đặt quá thấp để tránh nhận diện sai, nhưng cũng không nên đặt quá cao để hạn chế bỏ sót vật thể. Theo đường cong F1, mức confidence

khoảng 0,429 là điểm cân bằng tốt trên tập validation; khi vận hành thực tế có thể tinh chỉnh quanh vùng 0,4-0,5 tùy yêu cầu giảm nhầm hoặc giảm bỏ sót.

5.1.4 Đường cong PR và giá trị mAP

Đường cong Precision–Recall được sử dụng để đánh giá mối quan hệ giữa độ chính xác và khả năng phát hiện đối tượng của mô hình. Thông qua đường cong này, có thể xem xét hiệu quả nhận diện của từng lớp và giá trị mAP của toàn bộ mô hình. Kết quả đường cong Precision–Recall được trình bày trong hình 5.6.



Hình 5.6 Đường cong Precision–Recall của mô hình

Quan sát hình 5.6 cho thấy các đường cong Precision–Recall của ba lớp metal, organic và plastic đều duy trì ở mức cao trên phần lớn miền Recall. Điều này chứng tỏ mô hình có khả năng phát hiện tốt các đối tượng rác, đồng thời vẫn đảm bảo độ chính xác trong quá trình phân loại.

Kết quả đánh giá theo từng lớp trên chú thích đường cong Precision–Recall cho thấy lớp metal đạt $\text{mAP}@50 = 0,991$, lớp plastic đạt 0,984 và lớp organic đạt

0,959. Đối với toàn bộ các lớp, mô hình đạt $mAP@50 = 0,9783$ và $mAP50-95 = 0,7590$. Ngoài ra, tại epoch tốt nhất, mô hình đạt $Precision = 0,9497$ và $Recall = 0,9631$. Các kết quả này cho thấy mô hình có khả năng nhận diện và phân loại tốt ba nhóm rác trong tập dữ liệu kiểm định.

Bảng 5.1 Kết quả đánh giá mô hình theo từng lớp

Lớp	Precision	Recall	F1	mAP50	mAP50-95
metal	Không tách riêng theo lớp	Không tách riêng theo lớp	Không tách riêng theo lớp	0,991	Không tách riêng theo lớp
organic	Không tách riêng theo lớp	Không tách riêng theo lớp	Không tách riêng theo lớp	0,959	Không tách riêng theo lớp
plastic	Không tách riêng theo lớp	Không tách riêng theo lớp	Không tách riêng theo lớp	0,984	Không tách riêng theo lớp
all classes	0,9497	0,9631	0,95 tại confidence 0,429	0,9783	0,7590

Nhìn chung, các chỉ số đánh giá cho thấy mô hình đạt hiệu quả tốt trong bài toán nhận diện ba nhóm rác. Giá trị $mAP@50 = 0,9783$ phản ánh khả năng phát hiện đối tượng chính xác, trong khi $mAP50-95 = 0,7590$ cho thấy mô hình vẫn còn có thể cải thiện thêm về độ chính xác định vị hộp bao trong các ngưỡng IoU cao hơn.

5.2 KẾT QUẢ NHẬN DIỆN THỰC TẾ TRÊN RASPBERRY PI

5.2.1 Tốc độ xử lý (FPS) với mô hình NCNN

Bảng 5. 1 Kết quả đo FPS

Điều kiện	FPS trung bình	FPS min	FPS max	Ghi chú
Không có vật	73,46	13,07	104,42	Hệ thống ở trạng thái chờ, không

				thực hiện inference
Có 1 vật	13,81	1,20	104,07	Đo trong 5 phút, backend NCNN, AI bật
Có 2 vật cùng lúc	11,10	1,26	105,04	Đo khoảng 5 phút, backend NCNN, AI bật
Sau nửa giờ liên tục	32,07	1,26	103,43	Đo liên tục khoảng 35 phút, có vật đưa qua theo chu kỳ

Trong điều kiện không có vật đi qua vùng quan sát, hệ thống được vận hành liên tục trong khoảng 5 phút để đánh giá tốc độ xử lý ở trạng thái chờ. Kết quả ghi log cho thấy FPS trung bình đạt 73,46 FPS, FPS nhỏ nhất là 13,07 FPS và FPS lớn nhất là 104,42 FPS. Trong toàn bộ quá trình đo, hệ thống ở trạng thái DETECTOR_IDLE, không thực hiện inference và không phát hiện đối tượng. Do đó, FPS ở điều kiện này cao hơn so với các trường hợp có vật, vì Raspberry Pi chủ yếu xử lý luồng camera và trạng thái hệ thống mà chưa phải thực hiện suy luận mô hình. Nhiệt độ CPU dao động từ 41,4°C đến 46,3°C, cho thấy hệ thống hoạt động ổn định trong thời gian thử nghiệm.

Trong điều kiện có một vật đi qua vùng quan sát, hệ thống được vận hành trong khoảng 5 phút trên Raspberry Pi với backend NCNN và chế độ AI bật. Kết quả ghi log cho thấy FPS trung bình đạt 13,81 FPS, FPS nhỏ nhất là 1,20 FPS và FPS lớn nhất là 104,07 FPS. So với điều kiện không có vật, FPS trung bình giảm do hệ thống phải thực hiện suy luận mô hình YOLOv11 NCNN và xử lý các trạng thái phát hiện vật. Cụ thể, khi không thực hiện inference, FPS tức thời trung bình đạt khoảng 68,97 FPS; trong khi đó, khi inference được kích hoạt, FPS giảm còn khoảng 1,77 FPS. Nhiệt độ CPU dao động từ 40,9°C đến 48,7°C, cho thấy hệ thống vẫn hoạt động ổn định trong quá trình thử nghiệm.

Trong điều kiện có hai vật đi qua vùng quan sát, hệ thống được vận hành trong khoảng 5 phút trên Raspberry Pi với backend NCNN và chế độ AI bật. Kết quả ghi log cho thấy FPS trung bình đạt 11,10 FPS, FPS nhỏ nhất là 1,26 FPS và FPS lớn nhất là 105,04 FPS. So với trường hợp có một vật, FPS trung bình giảm do hệ thống phải xử lý nhiều trạng thái phát hiện, suy luận mô hình và chờ vật rời khỏi vùng phân loại. Khi không thực hiện inference, FPS tức thời trung bình đạt khoảng 63,31 FPS; trong khi đó, khi inference được kích hoạt, FPS giảm còn khoảng 1,78 FPS. Nhiệt độ CPU dao động từ 41,4°C đến 49,2°C, cho thấy hệ thống vẫn hoạt động ổn định trong quá trình thử nghiệm.

Đối với điều kiện chạy liên tục 30 phút, hệ thống được vận hành trong thời gian dài nhằm đánh giá độ ổn định khi hoạt động thực tế. Trong quá trình đo, các vật được đưa qua vùng quan sát theo chu kỳ để hệ thống luân phiên giữa trạng thái chờ, phát hiện và xử lý nhận diện. Kết quả ghi log cho thấy FPS trung bình đạt 32,07 FPS, FPS nhỏ nhất là 1,26 FPS và FPS lớn nhất là 103,43 FPS. Khi không thực hiện inference, FPS trung bình đạt khoảng 73,48 FPS; trong khi đó, khi mô hình YOLOv11 NCNN thực hiện inference, FPS giảm còn khoảng 1,78 FPS. Nhiệt độ CPU dao động từ 40,9°C đến 49,2°C, cho thấy Raspberry Pi hoạt động ổn định và chưa xuất hiện hiện tượng quá nhiệt trong thời gian thử nghiệm.

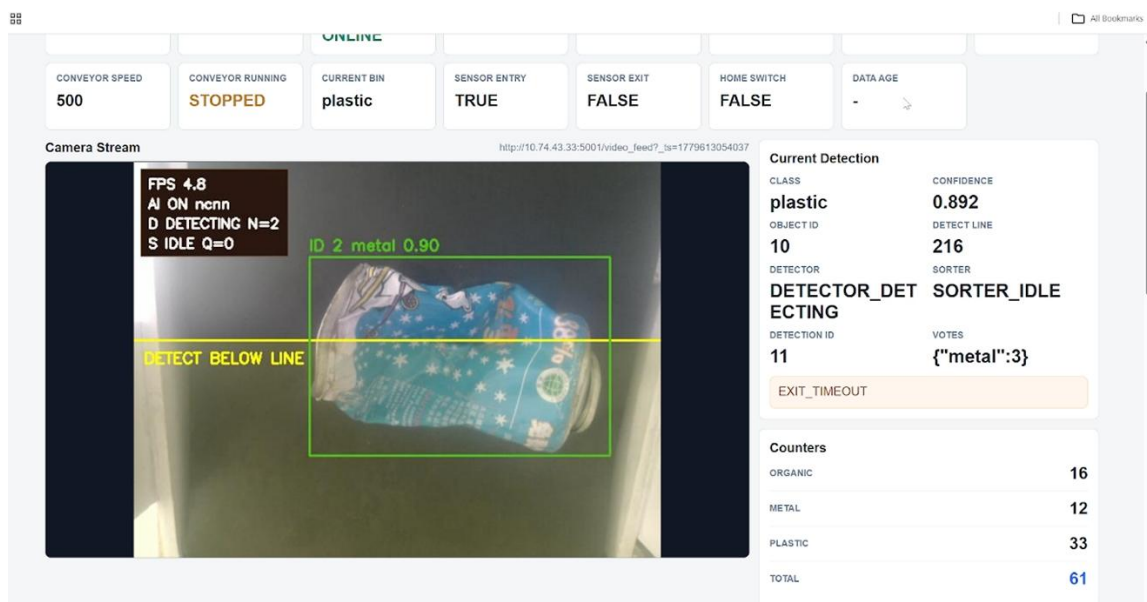
5.2.2 Độ chính xác phân loại theo từng nhóm rác

Độ chính xác thực tế cần được đo bằng số lần thử nghiệm trên mô hình băng tải sau khi cố định camera, ánh sáng và tốc độ băng tải. Trong phạm vi artifact cung cấp, chưa có file log thử nghiệm vật lý nên báo cáo sử dụng kết quả validation làm căn cứ mô hình và dành bảng này cho quá trình nghiệm thu.

Bảng 5. 2 Kết quả phân loại thực tế

Nhóm rác	Số lần thử	Đúng	Sai	Tỉ lệ chính xác (%)
Organic	84	80	4	95.24%
Plastic	70	58	12	82.86%

Metal	70	64	6	91,5%
Tổng cộng	224	202	22	89.87%



Hình 5. 7 Ảnh nhận diện thực tế với bounding box

5.2.3 Kết quả trong điều kiện ánh sáng khác nhau

Bảng 5. 3 Ảnh hưởng của ánh sáng

Điều kiện ánh sáng	FPS	Độ chính xác (%)	Ghi chú
Ánh sáng đầy đủ	21,87	94.3%	Điều kiện khuyến nghị
Ánh sáng yếu	17,95	87.8%	Dễ giảm confidence
Ngược sáng	21,83	70%	Dễ sai bbox

5.3 ĐÁNH GIÁ CƠ CHẾ XỬ LÝ LOGIC NHẬN DIỆN

5.3.1 Hiệu quả của voting và detection window

Voting giúp hệ thống không phản ứng tức thời với một frame nhiễu. Với cửa sổ 2,5 s và detect mỗi 2 frame, hệ thống có khoảng 25 cơ hội lấy mẫu trong một vật

thể. Điều kiện tối thiểu 3 vote và ratio 0,5 loại bỏ trường hợp dự đoán chia đều giữa các lớp.

Bảng 5. 4 So sánh kết quả có/không có voting

Phương pháp	Số lần thử	Đúng	Sai do nhận diện lẻ	Tỉ lệ chính xác (%)
Không dùng voting (single frame)	150	134	16	89.3%
Có dùng voting (N≈25)	224	202	22	89.97 % kỳ vọng ổn định hơn

5.3.2 Đánh giá khi 2 vật đi liên tiếp trên băng tải

Trường hợp hai vật liên tiếp được xử lý bằng entry lock, queue và sorter state. Nếu vật thứ hai đi vào khi sorter đang WAIT_MOTOR_DONE, runtime vẫn có nhánh cho phép detector chạy trong lúc motor bận nhưng không dừng băng tải, đồng thời ghi cảnh báo. Nếu khoảng cách quá gần, lỗi có thể xảy ra khi entry sensor chưa clear đủ thời gian hoặc queue đầy.

Bảng 5. 5 Kết quả khi 2 vật đi liên tiếp

Khoảng cách giữa 2 vật	Số lần thử	Phân loại đúng cả 2	Lỗi 1 vật	Lỗi cả 2	Ghi chú
Xa	120	107	13	0	Entry clear ổn định, hệ thống xử lý tuần tự
Trung bình	33	26	3	4	Có lúc vật thứ hai đến gần khi vật thứ nhất chưa xử lý xong
Rất gần	30	25	0	5	Dễ bị gộp detection window hoặc entry chưa reset kịp

Kết quả thử nghiệm trong Bảng 5.5 cho thấy khoảng cách giữa hai vật có ảnh hưởng rõ rệt đến khả năng xử lý của hệ thống. Khi hai vật được đặt ở khoảng cách xa, hệ thống phân loại đúng cả hai vật trong 107/120 lần thử, tương ứng độ

chính xác 90%. Điều này cho thấy khi cảm biến entry có đủ thời gian reset và state machine hoàn tất xử lý vật thứ nhất, hệ thống có thể nhận diện và phân loại tuần tự ổn định.

Ở khoảng cách trung bình, hệ thống phân loại đúng cả hai vật trong 26/33 lần thử, còn 7 lần chỉ lỗi một vật. Nguyên nhân chủ yếu là vật thứ hai đi vào vùng nhận diện khi vật thứ nhất chưa xử lý xong hoàn toàn, làm cho quá trình detection window, voting hoặc queue xử lý chưa kịp tách riêng hai lần nhận diện. Tuy nhiên, kết quả này vẫn cho thấy hệ thống có khả năng xử lý tương đối tốt trong trường hợp các vật đi liên tiếp với khoảng cách vừa phải.

Đối với trường hợp hai vật ở khoảng cách rất gần, số lần phân loại đúng cả hai giảm còn 25/30 lần, trong khi có 5 lần lỗi một vật. Hiện tượng này xảy ra do hai vật dễ bị gộp vào cùng một detection window hoặc cảm biến entry chưa kịp reset trước khi vật thứ hai đi vào. Vì vậy, hệ thống có thể bỏ sót vật thứ hai hoặc chỉ ghi nhận một lần phân loại cho cả hai vật.

Nhìn chung, kết quả thử nghiệm cho thấy hệ thống hoạt động ổn định khi các vật có khoảng cách đủ xa hoặc trung bình. Tuy nhiên, khi hai vật đi quá gần nhau, độ chính xác giảm do giới hạn của cơ chế phát hiện theo cảm biến entry, detection window và hàng đợi xử lý. Do đó, trong quá trình vận hành thực tế, cần đảm bảo khoảng cách tối thiểu giữa các vật trên băng tải để hệ thống có đủ thời gian nhận diện, voting và điều khiển cơ cấu phân loại.

5.3.3 Hiệu quả debounce và entry/exit sensor

Debounce được cài đặt ở cả ESP32 DEVKIT-V1 và Raspberry Pi để giảm nhiễu tín hiệu cảm biến. ESP32 lọc tín hiệu cảm biến với thời gian 40 ms, trong khi Raspberry Pi lọc cạnh lên của cảm biến entry trong 0,25 s và exit trong 0,2 s. Ngoài ra, cơ chế entry lock chỉ cho phép tạo đối tượng mới khi cảm biến entry đã hết active liên tục trong 0,15 s, giúp tránh việc một vật đang che cảm biến bị ghi nhận thành nhiều đối tượng.

5.3.4 Kết quả giao tiếp UART và điều khiển servo

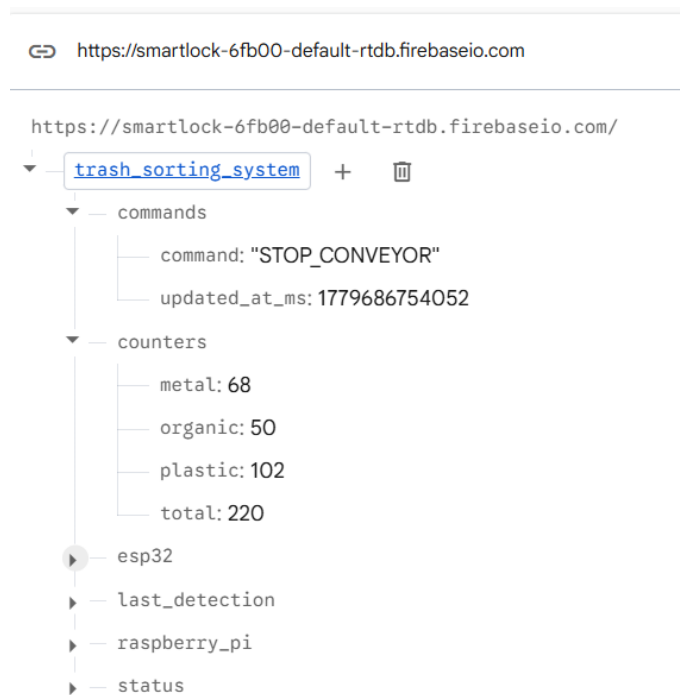
Giao tiếp UART dùng chuỗi text nên dễ debug. Lệnh SORT:<class> được gửi một lần qua `_send_sort_once()`, runtime chờ ACK/DONE và timeout theo `MOTOR_DONE_TIMEOUT_SEC`. Firmware servo giữ 700 ms trước khi DONE, sau đó có thể start lại conveyor. Nếu firmware báo ERROR hoặc timeout, state chuyển `SORTER_ERROR`.

Bảng 5. 6 Kết quả đo độ trễ UART và servo

Chỉ số	Giá trị đo được	Yêu cầu	Đạt/Không đạt
Baudrate	115200 bps	Ổn định	Đạt theo cấu hình
Thời gian servo	700 ms	< 3 s/vật	Đạt
UART timeout	8 s	Có cơ chế phục hồi	Đạt thiết kế
Độ trễ ACK	Trung bình 93,61 ms; nhỏ nhất 17,57 ms; lớn nhất 657,73 ms	< 200 ms	Đạt 92,97%; còn 108 mẫu vượt ngưỡng

5.4 KẾT QUẢ ĐỒNG BỘ FIREBASE VÀ HIỂN THỊ THÔNG KÊ

FirebaseSync chạy nền với chu kỳ 1 s, cập nhật status, raspberry_pi, esp32, counters và last_detection. Vì logic phân loại không phụ thuộc Firebase, mất mạng chỉ làm dashboard chậm cập nhật, không làm gián đoạn quá trình điều khiển băng tải và servo.



Hình 5. 8 Screenshot dữ liệu Firebase thực tế

Bảng 5. 7 Kết quả thống kê một phiên thực nghiệm

Nhóm rác	Số lần phân loại	Tỉ lệ (%)
organic	50	22,7%
plastic	102	46,4%
metal	68	30.9%

5.4.1 Đánh giá tổng thể và so sánh với yêu cầu đặt ra

Bảng 5. 8 Đánh giá đạt yêu cầu hệ thống

Yêu cầu	Chỉ tiêu	Kết quả thực tế	Đánh giá
FPS	≥ 5 FPS	Không vật: 73,46 FPS; có 1 vật: 13,81 FPS; có 2 vật: 11,10 FPS; chạy liên tục 30 phút: 32,07 FPS	Đạt yêu cầu
Độ chính xác	$\geq 85\%$	Validation mAP@50 = 0,9783; mAP50-95 = 0,7590	Đạt yêu cầu
Độ trễ phản hồi UART/ACK	< 200 ms	TB = 93,61 ms; 1428/1536 mẫu đạt	Đạt 92,97%

Thời gian phân loại/vật	< 3 s	Detection window 2,5 s + servo khoảng 0,7 s	Gần ngưỡng, cần tối ưu
Đồng bộ Firebase	< 5 s	Chu kỳ đồng bộ khoảng 1 s	Đạt thiết kế
Hoạt động liên tục	> 8 h	Đã thử nghiệm liên tục khoảng 30 phút, nhiệt độ CPU dưới 50°C	Chưa đủ 8 h, cần kiểm thử dài hạn

Dựa trên kết quả thực nghiệm, hệ thống cơ bản đáp ứng được các yêu cầu chính đã đặt ra. Về tốc độ xử lý, hệ thống đạt FPS trung bình 73,46 FPS trong điều kiện không có vật, 13,81 FPS khi có một vật, 11,10 FPS khi có hai vật và 32,07 FPS trong thử nghiệm chạy liên tục khoảng 30 phút. Các giá trị này đều đạt hoặc vượt ngưỡng yêu cầu tối thiểu 5 FPS. Tuy nhiên, FPS có sự dao động lớn do hệ thống luân phiên giữa trạng thái chờ và trạng thái thực hiện suy luận mô hình. Khi mô hình YOLOv11 NCNN thực hiện inference, FPS tức thời giảm đáng kể so với khi hệ thống không xử lý vật.

Về độ chính xác nhận diện, mô hình đạt $mAP@50 = 0,9783$ và $mAP50-95 = 0,7590$ trên tập validation, vượt ngưỡng yêu cầu 85%. Kết quả này cho thấy mô hình có khả năng nhận diện tốt ba nhóm rác gồm metal, organic và plastic. Cơ chế detection window kết hợp voting cũng góp phần tăng độ ổn định của kết quả phân loại, đặc biệt trong trường hợp vật di chuyển qua vùng quan sát của camera.

Đối với các yêu cầu về hệ thống, chu kỳ đồng bộ Firebase khoảng 1 giây đáp ứng yêu cầu nhỏ hơn 5 giây. Thời gian phân loại một vật gồm detection window khoảng 2,5 giây và thời gian servo khoảng 0,7 giây, tổng thời gian xấp xỉ 3,2 giây, gần với ngưỡng yêu cầu nên vẫn cần tối ưu thêm. Độ trễ UART hiện chưa được đo trực tiếp bằng timestamp nên cần bổ sung trong các thử nghiệm tiếp theo. Ngoài ra, hệ thống đã được kiểm tra chạy liên tục khoảng 30 phút với nhiệt độ CPU dưới 50°C, cho thấy khả năng vận hành ổn định trong thời gian ngắn, tuy nhiên vẫn cần kiểm thử dài hạn trên 8 giờ để đánh giá đầy đủ độ bền của hệ thống.

5.5 KẾT LUẬN CHƯƠNG

Kết quả thực nghiệm cho thấy mô hình YOLOv11 có khả năng nhận diện tốt ba nhóm rác gồm hữu cơ, nhựa và kim loại trong phạm vi dữ liệu kiểm định, với Precision = 0,9497, Recall = 0,9631, mAP50 = 0,9783 và mAP50-95 = 0,7590 tại epoch tốt nhất 140. Khi được chuyển đổi sang định dạng NCNN và triển khai trên Raspberry Pi, hệ thống có thể thực hiện nhận diện và điều khiển cơ cấu phân loại trong mô hình băng tải mô phỏng quá trình xử lý rác tại bãi tập kết.

Bên cạnh đó, cơ chế voting, detection window và debounce giúp hệ thống hạn chế ảnh hưởng của nhiễu trong từng khung hình và tín hiệu cảm biến. Giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1 hỗ trợ truyền lệnh điều khiển và nhận phản hồi trạng thái, trong khi Firebase giúp theo dõi dữ liệu phân loại. Tuy nhiên, hệ thống vẫn còn bị ảnh hưởng bởi ánh sáng, khoảng cách giữa các vật thể và thời gian xử lý khi rác đi liên tiếp. Những kết quả này là cơ sở để tổng kết đề tài và đề xuất các hướng phát triển phù hợp trong Chương 6.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Từ quá trình nghiên cứu, thiết kế, thi công và thực nghiệm hệ thống, chương này tổng kết những kết quả chính đã đạt được của đề tài phân loại rác tự động tại bãi tập kết sau quá trình thu gom. Đồng thời, chương cũng trình bày những ưu điểm, hạn chế còn tồn tại và đề xuất các hướng phát triển nhằm nâng cao khả năng xử lý, mở rộng phạm vi nhận diện cũng như tăng tính ứng dụng của hệ thống trong điều kiện thực tế.

6.1 KẾT LUẬN

6.1.1 Những kết quả đạt được

1. Thư mục kết quả huấn luyện mới ghi nhận 946 nhãn đối tượng thuộc ba lớp metal, organic và plastic; số ảnh train/val/test không có trong thư mục kết quả mới.
2. Huấn luyện YOLOv11 trong 150 epoch, epoch tốt nhất 140 đạt Precision = 94,97%, Recall = 96,31%, mAP50 = 97,83% và mAP50-95 = 75,90%.
3. Export mô hình sang NCNN với input 416x416, batch 1, stride 32, task detect, gồm model.ncnn.param và model.ncnn.bin.
4. Cài đặt detection window 2.5 s, object stable delay 0.6 s, MIN_VOTES = 3 và dominant ratio = 0,5.
5. Xây dựng giao thức UART đầy đủ gồm SORT, BIN, CONVEYOR, SPEED, HOME, ACK, DONE, ERROR, SENSOR và STATUS.
6. Thiết kế StateStore cục bộ và Firebase mirror cho status, counters, esp32, raspberry_pi và last_detection.
7. Tích hợp dashboard Flask API để theo dõi trạng thái và gửi lệnh điều khiển.
8. Xây dựng được Dashboard Android app hiển thị dữ liệu thống kê, tình trạng thiết bị.

6.1.2 Ưu điểm của hệ thống

Hệ thống có ưu điểm là suy luận AI tại biên, không phụ thuộc cloud cho quyết định phân loại; phần cứng chi phí thấp; phần mềm chia module rõ; voting và debounce giúp tăng độ ổn định; UART text protocol dễ debug; Firebase giúp mở rộng sang dashboard và Android app. Cấu trúc config bằng biến môi trường cũng giúp dễ tinh chỉnh khi triển khai trên Raspberry Pi thật.

6.1.3 Hạn chế còn tồn tại

Hệ thống còn phụ thuộc ánh sáng, góc camera và khoảng cách giữa các vật trên băng tải. Android app mới ở mức định hướng, chưa có log thực nghiệm dài hạn về FPS, độ trễ UART và độ chính xác thực tế. Mô hình mới nhận ba lớp nên chưa xử lý giấy, thủy tinh, rác nguy hại hoặc vật thể lạ có hình dạng khác biệt.

6.2 HƯỚNG PHÁT TRIỂN

6.2.1 Phát triển ứng dụng Android tích hợp Firebase

Ứng dụng Android sẽ đọc node Firebase để hiển thị bộ đếm, trạng thái online/offline, camera stream URL và cảnh báo lỗi. Các tính năng nâng cao gồm push notification khi thùng đầy, lọc lịch sử theo ngày, đặt lại counter và cấu hình ngưỡng từ xa.

6.2.2 Mở rộng nhận diện thêm nhóm rác

Có thể bổ sung các lớp giấy, thủy tinh, rác điện tử và rác nguy hại. Khi mở rộng, cần thu thập dữ liệu cân bằng hơn, bổ sung ảnh ở nhiều ánh sáng và kiểm tra nhầm lẫn giữa lớp có hình dạng tương tự.

6.2.3 Cải thiện cơ cấu cơ khí và tốc độ xử lý

Cơ cấu servo có thể thay bằng gạt nhanh hơn hoặc nhiều cửa phân loại. Đồng thời, cần một cơ cấu cơ khí bóc tách rác trước khi cho lên băng tải. Raspberry Pi

5, NPU rời hoặc accelerator có thể tăng FPS. Detection window có thể rút ngắn nếu camera và mô hình đủ ổn định.

6.2.4 Ứng dụng AI phân tích xu hướng rác thải

Dữ liệu Firebase có thể được lưu lịch sử để phân tích xu hướng theo giờ/ngày, dự đoán thời điểm thùng đầy và tối ưu lịch thu gom. Đây là bước chuyển từ hệ thống phân loại sang hệ thống quản lý rác thông minh.

6.2.5 Triển khai thực tế

Hệ thống có thể thử nghiệm tại khuôn viên trường đại học. Khi triển khai thực tế, cần tăng độ bền cơ khí, che chắn cảm biến, chuẩn hóa ánh sáng, bổ sung an toàn điện và kết nối với quy trình thu gom rác hiện hữu.

TÀI LIỆU THAM KHẢO

- [1] “World Bank – Solid and Industrial Hazardous Waste Management Assessment”.
- [2] J. Redmon, S. Divvala, R. Girshick, và A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, trong *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, thg. 6 2016, tr 779–788. doi: 10.1109/CVPR.2016.91.
- [3] “Ultralytics YOLO11 | Ultralytics Docs”. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: https://docs.ultralytics.com/models/yolo11?utm_source=chatgpt.com#overview
- [4] “GitHub - Tencent/mxnet: mxnet is a high-performance neural network inference framework optimized for the mobile platform · GitHub”. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://github.com/Tencent/mxnet>
- [5] “Ultralytics YOLO NCNN Export”, Ultralytics Docs. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.ultralytics.com/integrations/mxnet>
- [6] D. T. Tran và T. B. T. Nguyen, “An Intelligent Plastic Waste Classification System Based on Deep Learning and Delta Robot”, *Journal of Technical Education Science*, tập 20, số 01, tr 33–42, thg. 2 2025, doi: 10.54644/jte.2025.1555.
- [7] Nguyễn Mạnh Chính, “PHÁT HIỆN RÁC THẢI (VD: RÁC THẢI NHỰA, CHAI LỖ, VỎ HỘP SỮA BỎ ĐI CẦN PHÂN LOẠI)”.
- [8] M. Yang và G. Thung, “Classification of Trash for Recyclability Status”.
- [9] K. Andhy Panca Saputra, “Waste Object Detection and Classification using Deep Learning Algorithm: YOLOv4 and YOLOv4-tiny”, *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, tập 12, số 6, tr 5583–5595, thg. 4 2021, doi: 10.17762/turcomat.v12i6.9746.

- [10] R. Szeliski, “Computer Vision: Algorithms and Applications”.
- [11] “Model Validation with Ultralytics YOLO”, Ultralytics Docs. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.ultralytics.com/modes/val>
- [12] ultralytics, “ultralytics/ultralytics/cfg/models/11/yolo11.yaml at main · ultralytics/ultralytics”, GitHub. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/11/yolo11.yaml>
- [13] “RP-008341-DS-1-raspberry-pi-4-datasheet.pdf”. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008341-DS-1-raspberry-pi-4-datasheet.pdf>
- [14] “esp32-wroom-32_datasheet_en.pdf”. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf
- [15] “Universal Asynchronous Receiver/Transmitter (UART) - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation”. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html>
- [16] “Firebase Realtime Database”, Firebase. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://firebase.google.com/docs/database?hl=vi>
- [17] “Object Detection Datasets Overview”, Ultralytics Docs. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.ultralytics.com/datasets/detect>
- [18] “Data Augmentation using Ultralytics YOLO”, Ultralytics Docs. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.ultralytics.com/guides/yolo-data-augmentation>

[19] “Model Training with Ultralytics YOLO”, Ultralytics Docs. Truy cập: 4 Tháng Sáu 2026. [Online]. Có tại: <https://docs.ultralytics.com/modes/train>

PHỤ LỤC A – SOURCE CODE PYTHON

Main.py

```
from __future__ import annotations

import logging
import threading
import time
from collections import deque
from dataclasses import dataclass
from typing import Any

from flask import Flask, Response, jsonify

import config
from detec import Detection, YoloDetector
from firebase_sync import FirebaseSync
from state_store import StateStore, now_ms
from uart_client import UartClient, UartEvent

logging.basicConfig(level=getattr(logging, config.LOG_LEVEL,
logging.INFO), format="%(asctime)s %(levelname)s %(name)s:
%(message)s")
logger = logging.getLogger("trash_sorting.main")

DETECTOR_IDLE = "DETECTOR_IDLE"
DETECTOR_WAIT_OBJECT_STABLE = "DETECTOR_WAIT_OBJECT_STABLE"
```

```

DETECTOR_DETECTING = "DETECTOR_DETECTING"
DETECTOR_OBJECT_CONFIRMED = "DETECTOR_OBJECT_CONFIRMED"
DETECTOR_OBJECT_CREATED = "DETECTOR_OBJECT_CREATED"
DETECTOR_COOLDOWN = "DETECTOR_COOLDOWN"
DETECTOR_ERROR = "DETECTOR_ERROR"

SORTER_IDLE = "SORTER_IDLE"
SORTER_WAIT_EXIT = "SORTER_WAIT_EXIT"
SORTER_SORT_COMMAND_SENT = "SORTER_SORT_COMMAND_SENT"
SORTER_WAIT_MOTOR_DONE = "SORTER_WAIT_MOTOR_DONE"
SORTER_ERROR = "SORTER_ERROR"

@dataclass
class SortJob:
    id: str
    class_name: str
    confidence: float
    created_at: float
    object_id: int | None = None
    source: str = "yolo"
    status: str = "queued"
    ack_at: float = 0.0
    sent_at: float = 0.0

    def to_queue_item(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "class_name": self.class_name,
            "confidence": round(self.confidence, 4),
            "status": self.status,
        }

    def to_status_item(self) -> dict[str, Any]:
        payload = self.to_queue_item()

```



```

        payload.update(
            {
                "object_id": self.object_id,
                "source": self.source,
                "created_at_ms": int(self.created_at * 1000),
                "sent_at_ms": int(self.sent_at * 1000) if
self.sent_at else None,
            }
        )
        return payload

class CameraStream:
    def __init__(self, store: StateStore) -> None:
        self.store = store
        self.capture = None
        self.lock = threading.RLock()
        self.frame = None
        self.annotated = None
        self.encoded = b""
        self.frame_seq = 0
        self.encoded_seq = -1
        self.status = "UNKNOWN"
        self.last_error = ""
        self.next_open_at = 0.0

    def read(self):
        try:
            import cv2

            if self.capture is None:
                if time.time() < self.next_open_at:
                    self.status = "RECONNECTING"
                    return None
                self.capture = self._open(cv2)
            if self.capture is None or not self.capture.isOpened():

```

```

        self._fail("CAMERA_ERROR: cannot open camera")
        return None
    ok, frame = self.capture.read()
    if not ok:
        self._fail("CAMERA_ERROR: cannot read frame")
        return None
    with self.lock:
        self.frame = frame.copy()
        self.frame_seq += 1
        self.status = "ONLINE"
        self.last_error = ""
        return frame
    except Exception as exc:
        self._fail(f"CAMERA_ERROR:{exc}")
        return None

def set_annotated(self, frame: Any) -> None:
    if frame is None:
        return
    with self.lock:
        self.annotated = frame.copy()
        self.frame_seq += 1

def jpeg(self) -> bytes:
    try:
        import cv2

        with self.lock:
            if self.encoded and self.encoded_seq ==
self.frame_seq:
                return self.encoded
            frame = self.annotated if self.annotated is not
None else self.frame
            frame = None if frame is None else frame.copy()
            if frame is None:
                frame = self._placeholder("Waiting for camera")

```

```

        jpeg_quality = max(30, min(95,
int(config.STREAM_JPEG_QUALITY)))
        ok, buffer = cv2.imencode(".jpg", frame,
[int(cv2.IMWRITE_JPEG_QUALITY), jpeg_quality])
        data = buffer.tobytes() if ok else b""
        if data:
            with self.lock:
                self.encoded = data
                self.encoded_seq = self.frame_seq
            return data
    except Exception:
        return b""

def close(self) -> None:
    if self.capture is not None:
        try:
            self.capture.release()
        except Exception:
            pass
    self.capture = None

def _open(self, cv2):
    backend = {
        "auto": None,
        "default": None,
        "v4l2": getattr(cv2, "CAP_V4L2", None),
        "dshow": getattr(cv2, "CAP_DSHOW", None),
        "msmf": getattr(cv2, "CAP_MSMF", None),
    }.get(config.CAMERA_BACKEND, None)
    capture = cv2.VideoCapture(config.CAMERA_INDEX) if backend
is None else cv2.VideoCapture(config.CAMERA_INDEX, backend)
    capture.set(cv2.CAP_PROP_FRAME_WIDTH, config.CAMERA_WIDTH)
    capture.set(cv2.CAP_PROP_FRAME_HEIGHT,
config.CAMERA_HEIGHT)
    capture.set(cv2.CAP_PROP_FPS, config.CAMERA_FPS)
    capture.set(cv2.CAP_PROP_BUFFERSIZE, 1)

```

```

        if capture.isOpened():
            ok, _ = capture.read()
            if ok:
                logger.info("Camera opened index=%s",
config.CAMERA_INDEX)
                return capture
            capture.release()
        return None

    def _fail(self, message: str) -> None:
        self.close()
        self.status = "CAMERA_ERROR"
        self.last_error = message
        self.next_open_at = time.time() +
config.CAMERA_RECONNECT_DELAY_SEC
        self.store.update_status(camera_status="CAMERA_ERROR",
last_error=message)
        logger.warning(message)

    def _placeholder(self, message: str):
        import cv2
        import numpy as np

        frame = np.zeros((config.CAMERA_HEIGHT, config.CAMERA_WIDTH,
3), dtype=np.uint8)
        cv2.putText(frame, message[:80], (20, config.CAMERA_HEIGHT
// 2), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 220, 255), 2)
        return frame

class TrashSortingRuntime:
    def __init__(self) -> None:
        self.store = StateStore()
        self.camera = CameraStream(self.store)
        self.detector = YoloDetector()

```

```

self.uart = UartClient(logger=logger)
self.firebase = FirebaseSync(self.store, logger=logger)
self.queue_lock = threading.RLock()
self.object_queue: deque[SortJob] = deque()
self.pending_job: SortJob | None = None
self.detector_state = DETECTOR_IDLE
self.sorter_state = SORTER_IDLE
self.runtime_state = self._runtime_state_text()
self.last_sent_class = ""
self.last_sent_at = 0.0
self.last_queued_class = ""
self.last_queued_at = 0.0
self.running = False
self.thread: threading.Thread | None = None
self.frame_index = 0
self.fps = 0.0
self.ai_enabled = True
self.last_entry_state = False
self.last_entry_event_state = False
self.last_exit_state = False
self.last_entry_edge_at = 0.0
self.last_exit_edge_at = 0.0
self.last_entry_trigger_time = 0.0
self.last_exit_trigger_time = 0.0
self.entry_locked = False
self.entry_clear_started_at = 0.0
self.entry_locked_at = 0.0
self.detection_session_active = False
self.object_in_progress = False
self.last_entry_detect_frame = -1000000
self.last_preview_detection_at = 0.0
self.resume_conveyor_after_entry_sort = False
self.next_runtime_object_id = 1
self.current_detection_id: int | None = None
self.current_detection_class: str | None = None
self.current_object_id: int | None = None
self.current_class: str | None = None

```

```

self.confirmed_class: str | None = None
self.object_already_created = False
self.is_processing_object = False
self.sort_command_sent = False
self.motor_done = False
self.detect_start_time = 0.0
self.entry_trigger_time = 0.0
self.exit_trigger_time = 0.0
self.wait_exit_started_at = 0.0
self.last_reset_time = 0.0
self.last_detector_reset_time = 0.0
self.detection_votes: dict[str, int] = {}
self.detection_best_confidence: dict[str, float] = {}
self.valid_detection_count = 0
self.current_cycle_job: SortJob | None = None
self.last_detect_debug_at = 0.0
self.exit_trigger_count = 0

def start(self) -> None:
    self.running = True
    self.uart.start(self._on_uart_event)
    self.firebase.start()
    self.thread = threading.Thread(target=self._loop,
name="trash-runtime", daemon=True)
    self.thread.start()
    self._publish_runtime_state()
    self.store.update_status(system_status="RUNNING",
camera_stream_url=config.PUBLIC_STREAM_URL)

def stop(self) -> None:
    self.running = False
    self.uart.stop()
    self.firebase.stop()
    self.camera.close()

def stream_frames(self):

```

```

        delay = 1.0 / max(1, config.CAMERA_FPS)
        while True:
            yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" +
self.camera.jpeg() + b"\r\n"
            time.sleep(delay)

    def _runtime_state_text(self) -> str:
        if self.detector_state == DETECTOR_IDLE and
self.sorter_state == SORTER_IDLE:
            return "IDLE"
        return f"{self.detector_state}/{self.sorter_state}"

    def _publish_runtime_state(self, **status: Any) -> None:
        self.runtime_state = self._runtime_state_text()
        values = {
            "runtime_state": self.runtime_state,
            "current_state": self.runtime_state,
            "detector_state": self.detector_state,
            "sorter_state": self.sorter_state,
            "detect_phase": self.detector_state,
            "sort_phase": self.sorter_state,
            **self._cycle_status(),
            **status,
        }
        self.store.update_status(**values)

    def _set_runtime_state(self, state: str, **status: Any) -> None:
        self.runtime_state = state
        values = {"runtime_state": state, "current_state": state,
**self._cycle_status(), **status}
        self.store.update_status(**values)

    def _cycle_status(self) -> dict[str, Any]:
        return {
            "current_object_id": self.current_object_id,
            "current_detection_id": self.current_detection_id,

```

```

        "current_class": self.current_class or "unknown",
        "current_detection_class": self.current_detection_class
or "unknown",
        "confirmed_class": self.confirmed_class,
        "object_already_created": self.object_already_created,
        "is_processing_object": self.is_processing_object,
        "entry_locked": self.entry_locked,
        "entry_clear_started_at": self.entry_clear_started_at,
        "entry_locked_at": self.entry_locked_at,
        "last_entry_active": self.last_entry_state,
        "last_entry_event": self.last_entry_event_state,
        "detection_session_active":
self.detection_session_active,
        "object_in_progress": self.object_in_progress,
        "sorting_in_progress": self._sorter_busy(),
        "sorter_busy": self._sorter_busy(),
        "sort_command_sent": self.sort_command_sent,
        "motor_done": self.motor_done,
        "detection_votes": dict(self.detection_votes),
        "valid_detection_count": self.valid_detection_count,
        "queue_size": self._queue_depth(include_pending=False),
        "pending_job": self._active_object_payload(),
    }

    def _transition_detector(self, state: str, reason: str = "",
**status: Any) -> None:
        previous = self.detector_state
        self.detector_state = state
        self._publish_runtime_state(**status)
        if previous != state:
            message = f"[DETECTOR] {previous.replace('DETECTOR_',
'')} -> {state.replace('DETECTOR_', '')} "
            if reason:
                message = f"{message} ({reason})"
            self.store.add_log(message, "INFO")

```



```

    def _transition_sorter(self, state: str, reason: str = "",
**status: Any) -> None:
        previous = self.sorter_state
        self.sorter_state = state
        self._publish_runtime_state(**status)
        if previous != state:
            message = f"[SORTER] {previous.replace('SORTER_', '')}"
-> {state.replace('SORTER_', '')}"
            if reason:
                message = f"{message} ({reason})"
            self.store.add_log(message, "INFO")

    def _transition(self, state: str, reason: str = "", **status:
Any) -> None:
        self._set_runtime_state(state, **status)

    def _queue_depth(self, include_pending: bool = False) -> int:
        with self.queue_lock:
            depth = len(self.object_queue)
            if include_pending and self.pending_job is not None:
                depth += 1
            return depth

    def _queue_payload(self) -> list[dict[str, Any]]:
        with self.queue_lock:
            return [job.to_queue_item() for job in
self.object_queue]

    def _active_object_payload(self) -> dict[str, Any] | None:
        with self.queue_lock:
            return self.pending_job.to_status_item() if
self.pending_job is not None else None

    def _next_idle_or_waiting_state(self) -> str:
        if self._queue_depth() > 0:

```

```

        return SORTER_WAIT_EXIT
    if self._active_object_payload() is not None:
        return SORTER_WAIT_MOTOR_DONE
    return SORTER_IDLE

def _next_object_id(self) -> int:
    object_id = self.next_runtime_object_id
    self.next_runtime_object_id += 1
    return object_id

def _sort_uart_line(self, class_name: str) -> str:
    return f"{config.SORTING_COMMAND_PREFIX}:{class_name}"

def _is_busy(self) -> bool:
    return self.detector_state != DETECTOR_IDLE

def _max_queue_size(self) -> int:
    return max(1, int(getattr(config, "MAX_OBJECT_QUEUE_SIZE",
config.QUEUE_MAX_SIZE)))

def _queue_full(self) -> bool:
    return self._queue_depth(include_pending=False) >=
self._max_queue_size()

def _sorter_busy(self) -> bool:
    return self.sorter_state == SORTER_WAIT_MOTOR_DONE or
self._active_object_payload() is not None

def _lock_entry_for_object(self, now: float) -> None:
    self.entry_locked = True
    self.entry_locked_at = now
    self.entry_clear_started_at = 0.0
    self.store.add_log("[ENTRY] New object candidate accepted",
"INFO")

```

```

    def _update_entry_clear_state(self, entry_value: bool, now:
float) -> None:
        if entry_value:
            self.entry_clear_started_at = 0.0
            return
        if self.entry_locked and self.entry_clear_started_at <= 0:
            self.entry_clear_started_at = now

    def _maybe_unlock_entry(self, now: float) -> None:
        if not self.entry_locked or self.last_entry_state:
            return
        if self.entry_clear_started_at <= 0:
            self.entry_clear_started_at = now
            return
        if now - self.entry_clear_started_at <
config.ENTRY_CLEAR_DEBOUNCE_SEC:
            return
        self.entry_locked = False
        self.entry_locked_at = 0.0
        self.entry_clear_started_at = 0.0
        self.store.add_log("[ENTRY] Cleared: ready for next
object", "INFO")
        self._publish_runtime_state()

    def _ignore_locked_entry(self, now: float) -> None:
        if now - self.last_entry_edge_at <
config.ENTRY_SENSOR_DEBOUNCE_SEC:
            return
        self.last_entry_edge_at = now
        self.last_entry_trigger_time = now
        self.store.add_log("[ENTRY] Ignored: same sensor
occupation", "WARN")

    def _start_new_cycle(self, now: float, stopped_conveyor: bool =
True) -> None:

```

```

self.detection_session_active = True
self.object_in_progress = True
self.is_processing_object = True
self.current_detection_id = self._next_object_id()
self.current_object_id = self.current_detection_id
self.current_detection_class = None
self.current_class = None
self.confirmed_class = None
self.object_already_created = False
self.entry_trigger_time = now
self.detect_start_time = 0.0
self.exit_trigger_time = 0.0
self.wait_exit_started_at = 0.0
self.detection_votes = {}
self.detection_best_confidence = {}
self.valid_detection_count = 0
self.current_cycle_job = None
self.last_detect_debug_at = 0.0
self.last_entry_detect_frame = -1000000
self.resume_conveyor_after_entry_sort = stopped_conveyor
and config.RESUME_CONVEYOR_AFTER_SORT
    self.store.add_log("[DETECTION] Start session", "INFO")

def _reset_detector_cycle(self, reason: str, level: str =
"INFO") -> None:
    if self.resume_conveyor_after_entry_sort and
config.ENTRY_NO_DETECTION_RESUME:
        self._resume_conveyor_after_entry(reason,
update_state=False)
        self.detection_session_active = False
        self.object_in_progress = False
        self.current_detection_id = None
        self.current_detection_class = None
        self.current_object_id = None
        self.current_class = None
        self.confirmed_class = None
        self.object_already_created = False

```

```

self.is_processing_object = False
self.detect_start_time = 0.0
self.entry_trigger_time = 0.0
self.detection_votes = {}
self.detection_best_confidence = {}
self.valid_detection_count = 0
self.current_cycle_job = None
self.last_detector_reset_time = time.time()
self.last_reset_time = self.last_detector_reset_time
self._transition_detector(DETECTOR_COOLDOWN, reason)
if level == "INFO":
    self.store.add_log("[DETECTOR] reset to COOLDOWN",
"INFO")
else:
    self.store.add_log(f"[DETECTOR] reset after error:
{reason}", level)

def _begin_reset(self, reason: str, level: str = "INFO") ->
None:
    self._reset_detector_cycle(reason, level)

def _start_voting(self, now: float) -> None:
    self.detect_start_time = now
    self.detection_votes = {}
    self.detection_best_confidence = {}
    self.valid_detection_count = 0
    self.last_entry_detect_frame = -1000000
    self._transition_detector(DETECTOR_DETECTING, "object
stable")
    self.store.add_log("[DETECT] Voting started", "INFO")
    self.store.add_log(
        "[DETECT] thresholds "
        f"stable={config.OBJECT_STABLE_DELAY_SEC:.2f}s
window={config.DETECTION_WINDOW_SEC:.2f}s "
        f"min_votes={config.MIN_VOTES_TO_CONFIRM}
min_conf={config.MIN_CONFIDENCE:.2f} "

```

```

        f"ratio={config.MIN_DOMINANT_RATIO:.2f}
yolo_conf={config.YOLO_CONFIDENCE:.2f}",
        "INFO",
    )

    def _record_detection_vote(self, detection: Detection) -> None:
        if self.object_already_created:
            self.store.add_log("[SKIP] Object already created,
ignore detection frame", "WARN")
            return
        class_name = config.normalize_class(detection.class_name)
        if class_name not in config.TARGET_CLASSES:
            self.store.add_log(
                f"[DETECT] skip class={detection.class_name}
reason=unknown_or_unsupported
confidence={detection.confidence:.2f}",
                "WARN",
            )
            return
        if detection.confidence < config.MIN_CONFIDENCE:
            self.store.add_log(
                f"[DETECT] skip class={class_name}
reason=low_confidence confidence={detection.confidence:.2f}
min={config.MIN_CONFIDENCE:.2f}",
                "WARN",
            )
            return
        self.detection_votes[class_name] =
self.detection_votes.get(class_name, 0) + 1
        self.detection_best_confidence[class_name] =
max(self.detection_best_confidence.get(class_name, 0.0),
detection.confidence)
        self.valid_detection_count += 1

    self.store.update_status(detection_votes=dict(self.detection_votes),
valid_detection_count=self.valid_detection_count)
        self.store.add_log(

```

```

        f"[DETECT] vote {class_name}"
count={self.detection_votes[class_name]}
confidence={detection.confidence:.2f}",
        "INFO",
    )

    def _detection_text(self, detection: Detection) -> str:
        bbox = detection.bbox if detection.bbox is not None else "-"
        center = detection.center if detection.center is not None
else "-"
        return (
            f"class={detection.class_name}"
confidence={detection.confidence:.2f} "
            f"below_line={detection.below_line}"
overlap={detection.below_overlap_ratio:.2f} "
            f"center={center} bbox={bbox}"
        )

    def _log_detect_debug(self, frame: Any, detections:
list[Detection]) -> None:
        now = time.time()
        if now - self.last_detect_debug_at < 0.15:
            return
        self.last_detect_debug_at = now
        height, width = frame.shape[:2] if hasattr(frame, "shape")
else ("?", "?")
        all_detections = list(getattr(self.detector,
"last_all_detections", []) or [])
        rejected = list(getattr(self.detector,
"last_rejected_detections", []) or [])
        self.store.add_log(
            f"[DETECT] frame ok size={width}x{height}"
active={len(detections)} raw={len(all_detections)} "
            f"votes={dict(self.detection_votes)}"
total_valid_votes={self.valid_detection_count}",

```

```

        "INFO",
    )
    if not all_detections:
        self.store.add_log(
            f"[DETECT] YOLO no object above
YOLO_CONFIDENCE={config.YOLO_CONFIDENCE:.2f}",
            "WARN",
        )
    return
    for detection in all_detections[:3]:
        self.store.add_log(f"[DETECT] raw
{self._detection_text(detection)}", "INFO")
        if not detections and rejected:
            for detection in rejected[:3]:
                self.store.add_log(f"[DETECT] skip ROI
{self._detection_text(detection)}", "WARN")

    def _confirm_detection_votes(self) -> bool:
        total = self.valid_detection_count
        if not self.detection_votes or total <= 0:
            self.store.add_log("[ERROR] Detect timeout: no valid
YOLO detection", "ERROR")
            return False
        class_name, votes = max(self.detection_votes.items(),
key=lambda item: item[1])
        ratio = votes / max(1, total)
        if votes < config.MIN_VOTES_TO_CONFIRM or ratio <
config.MIN_DOMINANT_RATIO:
            self.store.add_log(
                f"[ERROR] Detect timeout, no stable class
votes={votes} total={total} ratio={ratio:.2f}",
                "ERROR",
            )
            return False
        self.confirmed_class = class_name
        self.current_class = class_name
        self.current_detection_class = class_name

```



```

        self._transition_detector(DETECTOR_OBJECT_CONFIRMED,
"stable class")

        self.store.add_log(f"[CONFIRM] class={class_name}
votes={votes} total={total} ratio={ratio:.2f}", "INFO")

        confidence = self.detection_best_confidence.get(class_name,
0.0)

        self.store.add_log(f"[DETECTION] Confirmed
class={class_name} votes={votes} confidence={confidence:.2f}",
"INFO")

        return True

def _create_object_once(self) -> bool:
    if self.detector_state != DETECTOR_OBJECT_CONFIRMED or not
self.is_processing_object or self.confirmed_class is None:
        return False

    if self.object_already_created:
        self.store.add_log("[SKIP] Object already created,
ignore detection frame", "WARN")
        return False

    confidence =
self.detection_best_confidence.get(self.confirmed_class, 0.0)
    detection = Detection(self.confirmed_class, confidence,
object_id=self.current_detection_id)

    if not self._maybe_enqueue(detection, source="entry"):
        self._reset_detector_cycle("object create failed",
"ERROR")

        return False

    self.object_already_created = True
    self.current_class = self.confirmed_class
    self.current_detection_class = self.confirmed_class
    self._transition_detector(DETECTOR_OBJECT_CREATED, "object
created")

    self.store.add_log(f"[OBJECT] created
id={self.current_detection_id} class={self.confirmed_class}",
"INFO")

    if self.sorter_state == SORTER_IDLE:
        self.wait_exit_started_at = time.time()

```

```

        self._transition_sorter(SORTER_WAIT_EXIT, "queue has
job")

        self._resume_conveyor_after_entry("object created; waiting
for exit")

        self._reset_detector_cycle("job queued", "INFO")
        return True

def _send_sort_once(self) -> bool:
    if self.pending_job is None:
        self.store.add_log("[WARN] Sort requested but no
pending job", "WARN")
        return False
    if self.sorter_state != SORTER_SORT_COMMAND_SENT:
        self.store.add_log(f"[SKIP] Sort ignored in
state={self.sorter_state}", "WARN")
        return False
    if self.sort_command_sent:
        self.store.add_log("[SKIP] Sort command already sent",
"WARN")
        return False
    with self.queue_lock:
        job = self.pending_job
        job.sent_at = time.time()
        job.status = "sorting_sending"

    line = self._sort_uart_line(job.class_name)
    if not self.uart.send_line(line):
        job.status = "error"
        with self.queue_lock:
            self.pending_job = None
            self.sort_command_sent = False
            self._transition_sorter(SORTER_ERROR, "sort send
failed")

self.store.update_status(last_error=self.uart.last_error,
uart_status="UART_ERROR")

```

```

        self.store.add_log(self.uart.last_error or "[ERROR]
UART send failed", "ERROR")
        return False

    job.status = "sorting_sent"
    self.sort_command_sent = True
    self.motor_done = False
    self.last_sent_class = job.class_name
    self.last_sent_at = job.sent_at
    self.store.update_status(
        queue_depth=self._queue_depth(include_pending=True),
        object_queue=self._queue_payload(),
        active_object=job.to_status_item(),
        current_class=job.class_name,
        current_object_id=job.object_id,
        confidence=round(job.confidence, 4),
    )
    self.store.add_log(f"[UART] sent {line}", "INFO")
    self._transition_sorter(SORTER_WAIT_MOTOR_DONE, "waiting
motor done")
    return True

def _loop(self) -> None:
    last_frame_at = 0.0
    while self.running:
        self._consume_web_command()
        frame = self.camera.read()
        if frame is None:

self.store.update_status(camera_status=self.camera.status,
uart_status="CONNECTED" if self.uart.connected else "DISCONNECTED",
last_error=self.camera.last_error)
        if self.detector_state in
{DETECTOR_WAIT_OBJECT_STABLE, DETECTOR_DETECTING}:
            self.store.add_log(
                f"[DETECT] camera frame unavailable

```

```

status={self.camera.status} error={self.camera.last_error}",
        "ERROR",
    )
    self._check_timeout()
    time.sleep(0.1)
    continue

    self.frame_index += 1
    now = time.time()
    if last_frame_at > 0:
        instant_fps = 1.0 / max(0.001, now - last_frame_at)
        self.fps = instant_fps if self.fps <= 0 else
(self.fps * 0.85) + (instant_fps * 0.15)
        last_frame_at = now

    annotated = frame
    detections: list[Detection] = []
    entry_detect_due = self.frame_index -
self.last_entry_detect_frame >= max(1,
config.ENTRY_DETECT_EVERY_N_FRAMES)

    if self.detector_state == DETECTOR_WAIT_OBJECT_STABLE:
        annotated = self.detector.draw_idle(frame)
        if self.entry_trigger_time > 0 and now -
self.entry_trigger_time >= config.OBJECT_STABLE_DELAY_SEC:
            self._start_voting(now)
    elif self.detector_state == DETECTOR_DETECTING:
        if self.ai_enabled and entry_detect_due:
            self.last_entry_detect_frame = self.frame_index
            detections, annotated =
self.detector.detect(frame)
            self._log_detect_debug(frame, detections)
            if detections:
                self._record_detection_vote(detections[0])
        else:
            annotated = self.detector.draw_idle(frame)

```

```

        if self.detect_start_time > 0 and now -
self.detect_start_time >= config.DETECTION_WINDOW_SEC:
            if self._confirm_detection_votes():
                self._create_object_once()
            else:
                self.store.add_log("[ENTRY] Detection
failed: no confirmed class", "WARN")
                self._reset_detector_cycle("detect
timeout", "ERROR")
            else:
                annotated = self.detector.draw_idle(frame)
                annotated = self._draw_runtime_overlay(annotated)
                self.camera.set_annotated(annotated)
                self._check_timeout()
                self._publish_local_status()
                time.sleep(config.MAIN_LOOP_SLEEP_SEC)

def _draw_runtime_overlay(self, frame: Any) -> Any:
    try:
        import cv2

        height, width = frame.shape[:2]
        ai_text = "AI ON" if self.ai_enabled else "AI OFF"
        stride = config.ENTRY_DETECT_EVERY_N_FRAMES if
self.detector_state == DETECTOR_DETECTING else
config.DETECT_EVERY_N_FRAMES
        lines = [
            f"FPS {self.fps:.1f}",
            f"{ai_text} {self.detector.backend}",
            f"D {self.detector_state.replace('DETECTOR_', '')}
N={max(1, stride)}",
            f"S {self.sorter_state.replace('SORTER_', '')}
Q={self._queue_depth()}",
        ]
        font = cv2.FONT_HERSHEY_SIMPLEX
        scale = 0.56

```

```

        thickness = 2
        sizes = [cv2.getTextSize(line, font, scale,
thickness)[0] for line in lines]
        box_width = min(width - 12, max(size[0] for size in
sizes) + 18)
        box_height = len(lines) * 24 + 10
        x1, y1 = 8, 8
        x2, y2 = x1 + box_width, min(height - 8, y1 +
box_height)
        cv2.rectangle(frame, (x1, y1), (x2, y2), (15, 23, 42),
-1)
        cv2.rectangle(frame, (x1, y1), (x2, y2), (70, 92, 120),
1)

        for index, line in enumerate(lines):
            y = y1 + 24 + index * 24
            cv2.putText(frame, line, (x1 + 9, y), font, scale,
(245, 248, 252), thickness)
        return frame
    except Exception:
        return frame

def _publish_detection_preview(self, detection: Detection) ->
None:
    now = time.time()
    if now - self.last_preview_detection_at < 0.5:
        return
    self.last_preview_detection_at = now
    self.store.update_detection(
        {
            "class_name": detection.class_name,
            "confidence": round(detection.confidence, 4),
            "object_id": detection.object_id,
            "bbox": detection.bbox,
            "center": detection.center,
            "line_y": self.detector.last_line_y,
            "zone": "below_line_preview",
            "source": "preview",

```

```

    }

    )

    def _maybe_enqueue(self, detection: Detection, source: str =
"yolo") -> bool:
        if self.object_already_created:
            self.store.add_log("[SKIP] Object already created,
ignore detection frame", "WARN")
            return False
        class_name = config.normalize_class(detection.class_name)
        if class_name not in config.TARGET_CLASSES:
            return False
        if source == "entry" and (self.confirmed_class is None or
class_name != self.confirmed_class):
            return False
        now = time.time()
        if source != "entry":
            if class_name == self.last_queued_class and now -
self.last_queued_at < config.SAME_CLASS_COOLDOWN_SEC:
                return False
            if class_name == self.last_sent_class and now -
self.last_sent_at < config.SAME_CLASS_COOLDOWN_SEC:
                return False

        object_id = self.current_detection_id if source == "entry"
else detection.object_id
        if object_id is None:
            object_id = self._next_object_id()
        detection.object_id = object_id
        job = SortJob(f"object_{object_id}", class_name,
detection.confidence, now, object_id=object_id, source=source)
        with self.queue_lock:
            if len(self.object_queue) >= self._max_queue_size():
                self.store.add_log("[QUEUE] Full: entry ignored",
"WARN")

            return False

```

```

        self.object_queue.append(job)
        self.current_cycle_job = job

    self.last_queued_class = class_name
    self.last_queued_at = now
    self.store.update_detection(
        {
            "class_name": class_name,
            "confidence": round(detection.confidence, 4),
            "object_id": detection.object_id,
            "bbox": detection.bbox,
            "center": detection.center,
            "line_y": self.detector.last_line_y,
            "zone": "below_line",
            "source": source,
            "job_id": job.id,
            "votes": dict(self.detection_votes),
            "valid_detection_count":
self.valid_detection_count,
        }
    )

    self.store.update_status(queue_depth=self._queue_depth(include_pending=True), object_queue=self._queue_payload())
    object_text = f" ID={detection.object_id}" if
detection.object_id is not None else ""
    self.store.add_log(f"[QUEUE] job added id={object_id}
class={class_name} queue_size={self._queue_depth()}", "INFO")
    self.store.add_log(f"[QUEUE] Added object
class={class_name} size={self._queue_depth()}", "INFO")
    return True

    def _complete_entry_detection(self, detection: Detection) ->
None:
        if self.detector_state == DETECTOR_DETECTING:
            self._record_detection_vote(detection)

```



```

def _check_timeout(self) -> None:
    now = time.time()
    self._maybe_unlock_entry(now)
    if self.detector_state == DETECTOR_COOLDOWN:
        if now - self.last_detector_reset_time >=
config.DETECTOR_COOLDOWN_SEC:
            self._transition_detector(DETECTOR_IDLE, "cooldown
complete")

            self.store.add_log("[DETECTOR] reset to IDLE",
"INFO")

            if self.detector_state in {DETECTOR_WAIT_OBJECT_STABLE,
DETECTOR_DETECTING} and self.entry_trigger_time > 0:
                detect_timeout = config.OBJECT_STABLE_DELAY_SEC +
config.DETECTION_WINDOW_SEC + max(1.0,
config.ENTRY_DETECT_TIMEOUT_SEC)
                if now - self.entry_trigger_time > detect_timeout:
                    self.store.update_status(system_status="ERROR",
last_error="DETECT_TIMEOUT")
                    if self.valid_detection_count <= 0:
                        self.store.add_log("[ERROR] Detect timeout: no
valid YOLO detection", "ERROR")
                    else:
                        self.store.add_log(
                            f"[ERROR] Detect timeout, no stable class
votes={dict(self.detection_votes)}
total={self.valid_detection_count}",
                            "ERROR",
                        )
                        self.store.add_log("[ENTRY] Detection failed: no
confirmed class", "WARN")
                        self._reset_detector_cycle("detect timeout",
"ERROR")

                if self.sorter_state == SORTER_WAIT_EXIT and
self.wait_exit_started_at > 0 and self._queue_depth() > 0:
                    if now - self.wait_exit_started_at >
config.EXIT_TIMEOUT_SEC:

```

```

        self.store.update_status(system_status="ERROR",
last_error="EXIT_TIMEOUT")
        self.store.add_log("[ERROR] Exit timeout", "ERROR")
        self._transition_sorter(SORTER_ERROR, "exit
timeout")

        self.wait_exit_started_at = 0.0
        return
    if self.sorter_state != SORTER_WAIT_MOTOR_DONE:
        return
    with self.queue_lock:
        pending_job = self.pending_job
    if pending_job is None:
        return
    if now - pending_job.sent_at <=
config.MOTOR_DONE_TIMEOUT_SEC:
        return
    message = f"UART_TIMEOUT:{pending_job.class_name}"
    self.store.update_status(system_status="UART_ERROR",
uart_status="UART_TIMEOUT", last_error=message)
    self.store.add_log("[ERROR] Motor timeout", "ERROR")
    with self.queue_lock:
        self.pending_job = None
        self.sort_command_sent = False
        self.motor_done = False
        self._transition_sorter(SORTER_ERROR, "motor timeout")

def _handle_entry_sensor(self, entry_event: Any, entry_active:
Any | None = None) -> None:
    if not isinstance(entry_event, bool):
        return
    if entry_active is None:
        entry_active = entry_event
    if not isinstance(entry_active, bool):
        return
    now = time.time()
    self.store.add_log(
        f"[ENTRY] Raw active={entry_active} event={entry_event}

```

```

locked={self.entry_locked} "
        f"detection_active={self.detection_session_active}
queue={self._queue_depth()}",
        "INFO",
    )
    previous_entry_event_state = self.last_entry_event_state
    self.last_entry_event_state = entry_event
    self.last_entry_state = entry_active
    self._update_entry_clear_state(entry_active, now)
    self._maybe_unlock_entry(now)
    rising_event = entry_event and not
previous_entry_event_state
    if not entry_event:
        return
    if self.entry_locked:
        if rising_event:
            self._ignore_locked_entry(now)
            return
    if not rising_event:
        return
    if now - self.last_entry_edge_at <
config.ENTRY_SENSOR_DEBOUNCE_SEC:
        return
    self.last_entry_edge_at = now
    self.last_entry_trigger_time = now
    if self._queue_full():
        self.store.add_log("[QUEUE] Full: entry ignored",
"WARN")
        return
    if self.detection_session_active:
        self.store.add_log("[SKIP] Detector session active,
ignore entry", "WARN")
        return
    if self.detector_state not in {DETECTOR_IDLE,
DETECTOR_COOLDOWN}:
        self.store.add_log("[SKIP] Detector busy, ignore
entry", "WARN")

```

```

        return

    if self.sorter_state == SORTER_WAIT_MOTOR_DONE:
        self._lock_entry_for_object(now)
        self._start_new_cycle(now, stopped_conveyor=False)
        self._transition_detector(DETECTOR_WAIT_OBJECT_STABLE,
"entry sensor")
        self.store.add_log("[WAIT] Object stable delay
started", "INFO")
        self.store.add_log("[WARN] Detector running while
sorter motor busy; conveyor stop skipped", "WARN")
        return

    if self.uart.send_line("CONVEYOR:STOP"):
        self._lock_entry_for_object(now)
        self._start_new_cycle(now, stopped_conveyor=True)
        self._transition_detector(DETECTOR_WAIT_OBJECT_STABLE,
"entry sensor")
        self.store.add_log("[WAIT] Object stable delay
started", "INFO")
        return

    self.store.update_status(last_error=self.uart.last_error,
uart_status="UART_ERROR")
    self.store.add_log(self.uart.last_error or "Failed to stop
conveyor on entry sensor", "ERROR")

def _handle_exit_sensor(self, exit_value: Any) -> None:
    if not isinstance(exit_value, bool):
        return
    now = time.time()
    rising_edge = exit_value and not self.last_exit_state
    self.last_exit_state = exit_value
    if not rising_edge:
        return
    if now - self.last_exit_edge_at <

```

```

config.EXIT_SENSOR_DEBOUNCE_SEC:
    return
    self.last_exit_edge_at = now
    self.last_exit_trigger_time = now
    self.exit_trigger_count += 1
    self.exit_trigger_time = now
    with self.queue_lock:
        queue_empty = len(self.object_queue) <= 0
        pending_job = self.pending_job
    if queue_empty:
        self.store.add_log("[EXIT] Ignored: object_queue
empty", "WARN")
        if pending_job is None and self.sorter_state !=
SORTER_WAIT_MOTOR_DONE:
            self._transition_sorter(SORTER_IDLE, "exit with
empty queue")
            return
        if self.sorter_state == SORTER_WAIT_MOTOR_DONE:
            self.store.add_log("[WARN] Exit triggered while motor
busy", "WARN")
            self.uart.send_line("CONVEYOR:STOP")
            return
        if self.sorter_state not in {SORTER_IDLE, SORTER_WAIT_EXIT,
SORTER_ERROR}:
            self.store.add_log(f"[SKIP] Exit ignored in
sorter_state={self.sorter_state}", "WARN")
            return
        self._dispatch_next_exit_job()

def _dispatch_next_exit_job(self) -> None:
    with self.queue_lock:
        job = self.object_queue.popleft() if self.object_queue
else None
        if job is not None:
            self.pending_job = job
    if job is None:
        self.store.add_log("[EXIT] Ignored: object_queue

```

```

empty", "WARN")

        self._transition_sorter(SORTER_IDLE, "exit with empty
queue")

        return

        self.sort_command_sent = False
        self.motor_done = False
        self.store.add_log(f"[EXIT] Trigger accepted
class={job.class_name} queue_after={self._queue_depth()}", "INFO")
        self.store.add_log(f"[EXIT] pop job id={job.object_id}
class={job.class_name} queue_size={self._queue_depth()}", "INFO")
        self._transition_sorter(SORTER_SORT_COMMAND_SENT, "exit
accepted")

        self._send_sort_once()

def _handle_entry_no_detection(self) -> None:
    if self.detector_state == DETECTOR_DETECTING:
        self.store.add_log("[ERROR] Detect timeout: no valid
YOLO detection", "ERROR")
        self.store.add_log("[ENTRY] Detection failed: no
confirmed class", "WARN")
        self._reset_detector_cycle("no detection", "ERROR")

def _resume_conveyor_after_entry(self, reason: str,
update_state: bool = True) -> None:
    if not self.resume_conveyor_after_entry_sort:
        return
    self.resume_conveyor_after_entry_sort = False
    if self.uart.send_line("CONVEYOR:START"):
        if update_state:
            self._publish_runtime_state()
            self.store.add_log(f"Conveyor resumed after entry
detection: {reason}", "INFO")
        else:

self.store.update_status(last_error=self.uart.last_error,
uart_status="UART_ERROR")

```

```

        self.store.add_log(self.uart.last_error or "Failed to
resume conveyor after detection", "ERROR")

def _consume_web_command(self) -> None:
    command = self.store.consume_pending_command()
    if not command:
        return
    line = str(command.get("line") or "").strip()
    action = str(command.get("action") or "")
    if action == "ai_start":
        self.ai_enabled = True
        self.store.complete_command(command.get("id", ""),
"done", "AI enabled")
        return
    if action == "ai_stop":
        self.ai_enabled = False
        self.store.complete_command(command.get("id", ""),
"done", "AI disabled")
        return
    if action == "camera_restart":
        self.camera.close()
        self.store.complete_command(command.get("id", ""),
"done", "camera restart requested")
        return
    if not line:
        self.store.complete_command(command.get("id", ""),
"error", "empty command line")
        return
    upper_line = line.upper()
    with self.queue_lock:
        pending_job = self.pending_job
        if (upper_line.startswith("SORT:") or
upper_line.startswith("BIN:")) and (
            pending_job is not None or self.sorter_state ==
SORTER_WAIT_MOTOR_DONE
        ):
            self.store.complete_command(command.get("id", ""),

```

```

"error", "sort busy")
        self.store.add_log("Rejected web bin/sort because a
command is already pending", "WARN")
        return
        command_job: SortJob | None = None
        if upper_line.startswith("SORT:") or
upper_line.startswith("BIN:"):
            class_name = config.normalize_class(line.split(":",
1)[1])
            if class_name in config.TARGET_CLASSES:
                command_job = SortJob(
                    id=str(command.get("id") or f"job_{now_ms()}"),
                    class_name=class_name,
                    confidence=1.0,
                    created_at=time.time(),
                    source="web",
                    status="sending",
                    sent_at=time.time(),
                )
                with self.queue_lock:
                    self.pending_job = command_job
                    self.sort_command_sent = True
                    self._transition_sorter(SORTER_WAIT_MOTOR_DONE,
"web sort")
                    if self.uart.send_line(line):
                        if command_job is not None:
                            command_job.status = "sent"
                            self.store.complete_command(command.get("id", ""),
"sent", line)
                        else:
                            if command_job is not None:
                                with self.queue_lock:
                                    self.pending_job = None
                                    self.sort_command_sent = False

self._transition_sorter(self._next_idle_or_waiting_state(), "web
command failed")

```



```

        self.store.complete_command(command.get("id", ""),
"error", self.uart.last_error)

def _on_uart_event(self, event: UartEvent) -> None:
    if event.kind == "ACK":
        self._mark_link_ok()
        command = event.command.lower()
        if "sort" in command or command.startswith("bin"):
            with self.queue_lock:
                if self.pending_job is not None:
                    self.pending_job.ack_at = time.time()
                    self.pending_job.status = "ack"
                self.store.add_log(event.raw, "INFO")
            return
        if event.kind == "DONE":
            self._mark_link_ok(recovered=True)
            command = event.command.lower()
            with self.queue_lock:
                pending_job = self.pending_job
                if pending_job is not None and self.sorter_state ==
SORTER_WAIT_MOTOR_DONE and ("sort" in command or
command.startswith("bin")):
                    completed_job = pending_job
                    completed_job.status = "done"

self.store.increment_counter(completed_job.class_name)
                self.motor_done = True
                object_text = f" ID={completed_job.object_id}" if
completed_job.object_id is not None else ""
                operation = "bin" if command.startswith("bin") else
"sort"

                self.store.add_log(f"[MOTOR] DONE job
id={completed_job.object_id} class={completed_job.class_name}",
"INFO")

                with self.queue_lock:
                    self.pending_job = None
                    self.sort_command_sent = False

```

```

        next_state = SORTER_WAIT_EXIT if
self._queue_depth() > 0 else SORTER_IDLE
        if next_state == SORTER_WAIT_EXIT:
            self.wait_exit_started_at = time.time()
        else:
            self.wait_exit_started_at = 0.0
        self._transition_sorter(next_state, "motor done",
active_object=None)
        self.uart.send_line("CONVEYOR:START")
    else:
        if "sort" in command or command.startswith("bin"):
            self.store.add_log("[SKIP] DONE ignored, no
pending job", "WARN")
            return
        self.store.add_log(event.raw, "INFO")
    return
    if event.kind == "ERROR":
        self.store.update_status(system_status="ERROR",
last_error=event.raw)
        self.store.add_log(event.raw, "ERROR")
        with self.queue_lock:
            self.pending_job = None
            self.sort_command_sent = False
            self._transition_sorter(SORTER_ERROR, "uart error")
        return
    if event.kind == "SENSOR":
        self._mark_link_ok()
        fields = event.fields or {}
        entry_event = fields.get("entry_event")
        if not isinstance(entry_event, bool):
            entry_event = fields.get("entry")
        entry_active = fields.get("entry_active")
        if not isinstance(entry_active, bool):
            entry_active = fields.get("entry")
        esp32_updates = {
            "sensor_entry": entry_active,
            "sensor_entry_event": entry_event,

```

```

        "sensor_entry_active": entry_active,
        "sensor_exit": fields.get("exit"),
        "home_switch": fields.get("home"),
    }
    if fields.get("home") is True:
        esp32_updates["current_bin"] = config.HOME_CLASS
    self.store.update_esp32(**esp32_updates)
    self._handle_entry_sensor(entry_event, entry_active)
    self._handle_exit_sensor(fields.get("exit"))
    return

if event.kind == "STATUS":
    fields = event.fields or {}
    esp32_state = str(fields.get("state", "")).upper()
    self._mark_link_ok(recovered=esp32_state != "ERROR")
    self.store.update_esp32(
        conveyor_running=fields.get("conveyor"),
        current_bin=fields.get("current_bin"),
        conveyor_speed_sps=fields.get("speed"),
        state=fields.get("state"),
    )
    if esp32_state == "ERROR":
        self.store.update_status(system_status="ERROR",
uart_status="CONNECTED")

def _publish_local_status(self) -> None:
    current_status = self.store.snapshot().get("status", {})
    uart_status = "CONNECTED" if self.uart.connected else
"DISCONNECTED"
    if self.uart.last_error and not self.uart.connected:
        uart_status = "UART_ERROR"
    if current_status.get("uart_status") == "UART_TIMEOUT" and
str(current_status.get("last_error",
"")).startswith("UART_TIMEOUT"):
        uart_status = "UART_TIMEOUT"
    system_status = "RUNNING"
    if current_status.get("system_status") in {"ERROR",
"UART_ERROR"} and current_status.get("last_error"):

```

```

        system_status = current_status.get("system_status")
        self.store.update_raspberry_pi(online=True,
fps=round(self.fps, 2), ai_enabled=self.ai_enabled)
        self.store.update_status(
            system_status=system_status,
            camera_status=self.camera.status,
            uart_status=uart_status,
            fps=round(self.fps, 2),
            queue_depth=self._queue_depth(include_pending=True),
            object_queue=self._queue_payload(),
            active_object=self._active_object_payload(),
            camera_stream_url=config.PUBLIC_STREAM_URL,
            runtime_state=self._runtime_state_text(),
            current_state=self._runtime_state_text(),
            detector_state=self.detector_state,
            sorter_state=self.sorter_state,
            yolo_ready=self.detector.enabled,
            yolo_backend=self.detector.backend,
            yolo_error=self.detector.last_error,
            detection_line_y=self.detector.last_line_y,
            detection_line_ratio=config.DETECTION_LINE_Y_RATIO,

entry_triggered_detection=config.ENTRY_TRIGGERED_DETECTION,
            sorting_command_prefix=config.SORTING_COMMAND_PREFIX,
            home_class=config.HOME_CLASS,
            **self._cycle_status(),
        )

def _mark_link_ok(self, recovered: bool = False) -> None:
    current_status = self.store.snapshot().get("status", {})
    last_error = str(current_status.get("last_error", ""))
    values: dict[str, Any] = {"uart_status": "CONNECTED"}
    if last_error.startswith("UART_TIMEOUT"):
        values.update({"system_status": "RUNNING",
"last_error": ""})
    elif recovered or current_status.get("system_status") not
in {"ERROR", "UART_ERROR"}:

```

```

        values["system_status"] = "RUNNING"
        self.store.update_status(**values)

runtime = TrashSortingRuntime()
stream_app = Flask(__name__)

@stream_app.get("/")
def index():
    return jsonify({"service": "trash_sorting_runtime",
"video_feed": "/video_feed", "status": "/api/status"})

@stream_app.get("/video_feed")
def video_feed():
    return Response(
        runtime.stream_frames(),
        mimetype="multipart/x-mixed-replace; boundary=frame",
        headers={"Cache-Control": "no-store, no-cache, must-
revalidate", "X-Accel-Buffering": "no"},
    )

@stream_app.get("/api/status")
def api_status():
    return jsonify(runtime.store.dashboard_payload())

if __name__ == "__main__":
    runtime.start()
    try:
        stream_app.run(host=config.STREAM_HOST,
port=config.STREAM_PORT, threaded=True)

```

```
finally:
    runtime.stop()
```

app.py

```
from __future__ import annotations

import json
import logging
import time
from typing import Any

from flask import Flask, Response, jsonify, render_template,
request, stream_with_context

import config
from state_store import StateStore

logging.basicConfig(level=getattr(logging, config.LOG_LEVEL,
logging.INFO))
logger = logging.getLogger(__name__)

app = Flask(__name__)
store = StateStore()

def normalize_command(command: str) -> str:
    value = str(command or "").strip().lower().replace("-",
"_").replace(" ", "_")
    aliases = {
        "start": "conveyor_start",
        "stop": "conveyor_stop",
        "start_conveyor": "conveyor_start",
        "stop_conveyor": "conveyor_stop",
        "reset": "home",
```

```

        "emergency_stop": "conveyor_stop",
    }
    if value.startswith("sort:"):
        value = "sort_" + config.normalize_class(value.split(":",
1)[1])
    if value.startswith("bin:"):
        value = "rotate_" + config.normalize_class(value.split(":",
1)[1])
    if value.startswith("rotate_"):
        value = "rotate_" +
config.normalize_class(value.replace("rotate_", "", 1))
    return aliases.get(value, value)

def command_to_uart(action: str, value: Any = None) -> str:
    if action == "conveyor_start":
        return "CONVEYOR:START"
    if action == "conveyor_stop":
        return "CONVEYOR:STOP"
    if action == "home":
        return "HOME"
    if action == "reset_counter":
        return "RESET_COUNTER"
    if action == "conveyor_speed":
        speed = int(float(value if value not in (None, "") else
config.CONVEYOR_DEFAULT_SPEED_SPS))
        if speed < config.CONVEYOR_SPEED_MIN or speed >
config.CONVEYOR_SPEED_MAX:
            raise ValueError(f"conveyor_speed must be between
{config.CONVEYOR_SPEED_MIN} and {config.CONVEYOR_SPEED_MAX}")
        return f"SPEED:{speed}"
    if action.startswith("rotate_"):
        class_name =
config.normalize_class(action.replace("rotate_", "", 1))
        if class_name == "unknown":
            raise ValueError(f"Unsupported bin class: {action}")

```

```

        return f"BIN:{class_name}"
    if action.startswith("sort_"):
        class_name = config.normalize_class(action.replace("sort_",
"", 1))
        if class_name == "unknown":
            raise ValueError(f"Unsupported sort class: {action}")
        return f"SORT:{class_name}"
    raise ValueError(f"Unsupported command: {action}")

@app.get("/")
def index():
    return render_template("index.html")

@app.get("/api/status")
def api_status():
    return jsonify(store.dashboard_payload())

@app.get("/api/events")
def api_events():
    def event_stream():
        while True:
            payload = json.dumps(store.dashboard_payload(),
separators=(",", " "))
            yield f"event: status\ndata: {payload}\n\n"
            time.sleep(config.DASHBOARD_PUSH_INTERVAL_SEC)

    return Response(
        stream_with_context(event_stream()),
        mimetype="text/event-stream",
        headers={"Cache-Control": "no-store, no-cache, must-
revalidate", "X-Accel-Buffering": "no"},
    )

```



```

@app.post("/api/command")
def api_command():
    payload = request.get_json(silent=True) or {}
    action = normalize_command(str(payload.get("command") or ""))
    if action not in config.COMMANDS:
        return jsonify({"ok": False, "message": f"Unsupported
command: {action}"}), 400

    if action in {"ai_start", "ai_stop", "camera_restart"}:
        command = store.set_pending_command(action, action.upper(),
source="web")
        return jsonify({"ok": True, "command": command})

    try:
        line = command_to_uart(action, payload.get("value") if
"value" in payload else payload.get("speed"))
    except ValueError as exc:
        return jsonify({"ok": False, "message": str(exc)}), 400

    if action == "reset_counter":
        store.reset_counters()
        command = store.set_pending_command(action, line,
payload.get("value"), source="web")
        logger.info("Web command queued action=%s line=%s", action,
line)
        return jsonify({"ok": True, "command": command})

@app.post("/api/reset_counter")
def api_reset_counter():
    store.reset_counters()
    command = store.set_pending_command("reset_counter",
"RESET_COUNTER", source="web")

```

```

        return jsonify({"ok": True, "command": command})

@app.get("/api/health")
def api_health():
    payload = store.dashboard_payload()
    return jsonify(
        {
            "ok": True,
            "service": "trash_sorting_dashboard",
            "state_file": str(config.STATE_FILE),
            "raspberry_pi":
payload["status"].get("raspberry_pi_status"),
            "esp32": payload["status"].get("esp32_status"),
            "firebase_mode": "mirror_only",
        }
    )

@app.get("/api/firebase-debug")
def api_firebase_debug():
    payload = store.dashboard_payload()
    return jsonify({"firebase": payload.get("firebase", {}),
"status": payload.get("status", {})}))

if __name__ == "__main__":
    app.run(host=config.APP_HOST, port=config.APP_PORT,
threaded=True)

```

detec.py

```

from __future__ import annotations

import logging
import math

```

```

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import config

logger = logging.getLogger("trash_sorting.detec")

@dataclass
class Detection:
    class_name: str
    confidence: float
    bbox: tuple[int, int, int, int] | None = None
    object_id: int | None = None
    center: tuple[int, int] | None = None
    below_line: bool = False
    below_overlap_ratio: float = 0.0

@dataclass
class Track:
    object_id: int
    class_name: str
    center: tuple[int, int]
    bbox: tuple[int, int, int, int]
    last_seen_frame: int

class DetectionTracker:
    def __init__(self) -> None:
        self.next_id = 1
        self.tracks: dict[int, Track] = {}

```

```

    def update(self, detections: list[Detection], frame_index: int)
-> list[Detection]:
        self._drop_stale(frame_index)
        used_tracks: set[int] = set()
        for detection in sorted(detections, key=lambda item:
item.confidence, reverse=True):
            if detection.bbox is None:
                continue
            center = bbox_center(detection.bbox)
            detection.center = center
            object_id = self._match_track(detection.class_name,
center, used_tracks)
            if object_id is None:
                object_id = self.next_id
                self.next_id += 1
            used_tracks.add(object_id)
            detection.object_id = object_id
            self.tracks[object_id] = Track(object_id,
detection.class_name, center, detection.bbox, frame_index)
        return detections

    def _match_track(self, class_name: str, center: tuple[int, int],
used_tracks: set[int]) -> int | None:
        best_id: int | None = None
        best_distance = float("inf")
        max_distance = max(1,
int(config.DETECTION_TRACK_MAX_DISTANCE))
        for object_id, track in self.tracks.items():
            if object_id in used_tracks or track.class_name !=
class_name:
                continue
            distance = math.dist(center, track.center)
            if distance < best_distance and distance <=
max_distance:
                best_id = object_id

```

```

        best_distance = distance
    return best_id

def _drop_stale(self, frame_index: int) -> None:
    max_age = max(1,
int(config.DETECTION_TRACK_MAX_AGE_FRAMES))
        stale_ids = [object_id for object_id, track in
self.tracks.items() if frame_index - track.last_seen_frame >
max_age]
        for object_id in stale_ids:
            self.tracks.pop(object_id, None)

class YoloDetector:
    def __init__(self) -> None:
        self.model = None
        self.enabled = False
        self.last_error = ""
        self.backend = "unknown"
        self.class_names = self._load_class_names()
        self.mock_index = 0
        self.frame_index = 0
        self.last_line_y = 0
        self.last_all_detections: list[Detection] = []
        self.last_active_detections: list[Detection] = []
        self.last_rejected_detections: list[Detection] = []
        self.tracker = DetectionTracker()
        self._load()

    def detect(self, frame: Any) -> tuple[list[Detection], Any]:
        if frame is None:
            self.last_all_detections = []
            self.last_active_detections = []
            self.last_rejected_detections = []
            return [], None
        self.frame_index += 1

```

```

        if config.MOCK_DETECTION:
            detection =
Detection(config.normalize_class(config.MOCK_CLASS), 0.99,
self._mock_bbox(frame))
            active_detections = self._mark_detection_zone(frame,
[detection])
            self.last_all_detections = [detection]
            self.last_active_detections = list(active_detections)
            self.last_rejected_detections = [item for item in
[detection] if not item.below_line]
            self.tracker.update(active_detections,
self.frame_index)
            return active_detections, self.draw(frame, [detection])
        if not self.enabled or self.model is None:
            self.last_all_detections = []
            self.last_active_detections = []
            self.last_rejected_detections = []
            return [], self._draw_message(frame, self.last_error or
"YOLO not ready")
        try:
            if self.backend == "ncnn":
                detections = self._predict_ncnn(frame)
            else:
                detections = self._predict_ultralytics(frame)
        except Exception as exc:
            self.last_error = f"DETECTION_INFERENCE_ERROR:{exc}"
            logger.exception(self.last_error)
            self.last_all_detections = []
            self.last_active_detections = []
            self.last_rejected_detections = []
            return [], self._draw_message(frame, self.last_error)
        active_detections = self._mark_detection_zone(frame,
detections)
        self.last_all_detections = list(detections)
        self.last_active_detections = list(active_detections)
        self.last_rejected_detections = [detection for detection in
detections if not detection.below_line]

```

```

        self.tracker.update(active_detections, self.frame_index)
        return active_detections, self.draw(frame, detections)

def draw_idle(self, frame: Any) -> Any:
    return self.draw(frame, [])

def draw(self, frame: Any, detections: list[Detection]) -> Any:
    try:
        import cv2

        annotated = frame.copy()
        line_y = self._line_y(annotated)
        height, width = annotated.shape[:2]
        overlay = annotated.copy()
        cv2.rectangle(overlay, (0, line_y), (width, height),
(0, 255, 255), -1)
        cv2.addWeighted(overlay, 0.10, annotated, 0.90, 0,
annotated)
        cv2.line(annotated, (0, line_y), (width, line_y), (0,
255, 255), 2)
        cv2.putText(annotated, "DETECT BELOW LINE", (12,
min(height - 12, line_y + 28)), cv2.FONT_HERSHEY_SIMPLEX, 0.62, (0,
255, 255), 2)
        for detection in detections:
            if detection.bbox is None:
                continue
            x1, y1, x2, y2 = detection.bbox
            if detection.below_line:
                color = (30, 190, 80)
                label_id = f"ID {detection.object_id}" if
detection.object_id is not None else "ID -"
                label = f"{label_id} {detection.class_name}
{detection.confidence:.2f}"
            else:
                color = (170, 170, 170)
                label = f"WAIT {detection.class_name}"

```

```

{detection.confidence:.2f}"
        cv2.rectangle(annotated, (x1, y1), (x2, y2), color,
2)

        cv2.putText(annotated, label, (x1, max(22, y1 -
8)), cv2.FONT_HERSHEY_SIMPLEX, 0.62, color, 2)
        return annotated
    except Exception:
        return frame

def _load(self) -> None:
    self.backend = self._infer_backend(config.YOLO_MODEL_PATH)
    if config.MOCK_DETECTION:
        self.enabled = True
        self.backend = "mock"
        logger.warning("MOCK_DETECTION enabled")
        return
    if self.backend == "ncnn":
        self._load_ncnn()
        return
    try:
        from ultralytics import YOLO

        self.model = YOLO(str(config.YOLO_MODEL_PATH),
task="detect")
        self.enabled = True
        logger.info("YOLO loaded: %s backend=%s",
config.YOLO_MODEL_PATH, self.backend)
    except Exception as exc:
        self.enabled = False
        self.last_error = f"YOLO_LOAD_FAILED:{exc}"
        logger.exception(self.last_error)

def _load_ncnn(self) -> None:
    try:
        import ncnn

```



```

        model_dir = Path(config.YOLO_MODEL_PATH)
        param_path = model_dir / "model.ncnn.param"
        bin_path = model_dir / "model.ncnn.bin"
        net = ncnn.Net()
        try:
            net.opt.num_threads = int(config.NCNN_NUM_THREADS)
        except Exception:
            pass
        net.load_param(str(param_path))
        net.load_model(str(bin_path))
        self.model = net
        self.enabled = True
        self.last_error = ""
        logger.info("NCNN loaded: %s", model_dir)
    except Exception as exc:
        self.model = None
        self.enabled = False
        self.last_error = f"NCNN_LOAD_FAILED:{exc}"
        logger.exception(self.last_error)

def _predict_ncnn(self, frame: Any) -> list[Detection]:
    import ncnn
    import numpy as np

    tensor, scale, pad_x, pad_y = self._preprocess_ncnn(frame)
    extractor = self.model.create_extractor()
    extractor.input("in0", ncnn.Mat(tensor).clone())
    ret, output = extractor.extract("out0")
    if ret != 0:
        raise RuntimeError(f"NCNN extract out0 failed")
    ret={ret})
    raw = np.array(output)
    return self._decode_ncnn(raw, frame.shape[1],
frame.shape[0], scale, pad_x, pad_y)

def _preprocess_ncnn(self, frame: Any) -> tuple[Any, float, int,

```

```

int]:
    import cv2
    import numpy as np

    height, width = frame.shape[:2]
    image_size = int(config.YOLO_IMGSZ)
    scale = min(image_size / width, image_size / height)
    resized_width = int(round(width * scale))
    resized_height = int(round(height * scale))
    resized = cv2.resize(frame, (resized_width,
resized_height), interpolation=cv2.INTER_LINEAR)
    canvas = np.full((image_size, image_size, 3), 114,
dtype=np.uint8)
    pad_x = (image_size - resized_width) // 2
    pad_y = (image_size - resized_height) // 2
    canvas[pad_y : pad_y + resized_height, pad_x : pad_x +
resized_width] = resized
    rgb = cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)
    tensor = rgb.astype("float32") / 255.0
    return tensor.transpose(2, 0, 1).copy(), scale, pad_x,
pad_y

    def _decode_ncnn(self, raw: Any, frame_width: int, frame_height:
int, scale: float, pad_x: int, pad_y: int) -> list[Detection]:
        import numpy as np

        output = np.asarray(raw, dtype=np.float32)
        class_count = max(1, len(self.class_names))
        if output.ndim == 3:
            output = output.reshape(output.shape[-2],
output.shape[-1])
            if output.shape[0] < output.shape[1] and output.shape[0] >=
4 + class_count:
                predictions = output.T
            else:
                predictions = output

```

```

candidates: list[Detection] = []
for row in predictions:
    if row.shape[0] < 4 + class_count:
        continue
    scores = row[4 : 4 + class_count]
    class_index = int(np.argmax(scores))
    confidence = float(scores[class_index])
    if confidence < config.YOLO_CONFIDENCE:
        continue
    x_center, y_center, box_width, box_height = (float(v)
for v in row[:4])
    x1 = (x_center - box_width / 2.0 - pad_x) / scale
    y1 = (y_center - box_height / 2.0 - pad_y) / scale
    x2 = (x_center + box_width / 2.0 - pad_x) / scale
    y2 = (y_center + box_height / 2.0 - pad_y) / scale
    bbox = clamp_bbox((x1, y1, x2, y2), frame_width,
frame_height)
    if bbox is None:
        continue
    class_name =
config.normalize_class(self.class_names.get(class_index,
str(class_index)))
    if class_name == "unknown":
        continue
    candidates.append(Detection(class_name, confidence,
bbox))
return self._nms(candidates)

def _predict_ultralytics(self, frame: Any) -> list[Detection]:
    args: dict[str, Any] = {"conf": config.YOLO_CONFIDENCE,
"imgsz": config.YOLO_IMGsz, "verbose": False}
    if config.YOLO_DEVICE:
        args["device"] = config.YOLO_DEVICE
    return self._parse_ultralytics(self.model.predict(frame,
**args))

```

```

def _parse_ultralytics(self, results: Any) -> list[Detection]:
    detections: list[Detection] = []
    for result in results or []:
        names = getattr(result, "names", {}) or {}
        boxes = getattr(result, "boxes", None)
        if boxes is None:
            continue
        for box in boxes:
            cls_value = int(box.cls[0])
            class_name =
config.normalize_class(names.get(cls_value, str(cls_value)))
            confidence = float(box.conf[0])
            if class_name == "unknown" or confidence <
config.YOLO_CONFIDENCE:
                continue
            bbox = tuple(int(v) for v in box.xyxy[0])
            detections.append(Detection(class_name, confidence,
bbox))
        detections.sort(key=lambda item: item.confidence,
reverse=True)
    return detections

def _nms(self, detections: list[Detection]) -> list[Detection]:
    selected: list[Detection] = []
    by_class: dict[str, list[Detection]] = {}
    for detection in detections:
        by_class.setdefault(detection.class_name,
[]).append(detection)
    for class_detections in by_class.values():
        class_selected: list[Detection] = []
        for detection in sorted(class_detections, key=lambda
item: item.confidence, reverse=True):
            if all(box_iou(detection.bbox, kept.bbox) <
config.NMS_IOU_THRESHOLD for kept in class_selected):
                class_selected.append(detection)

```

```

        selected.extend(class_selected)
        selected.sort(key=lambda item: item.confidence,
reverse=True)
        return selected

    def _mark_detection_zone(self, frame: Any, detections:
list[Detection]) -> list[Detection]:
        line_y = self._line_y(frame)
        active: list[Detection] = []
        for detection in detections:
            if detection.bbox is None:
                continue
            detection.below_line = False
            detection.below_overlap_ratio = 0.0
            center = bbox_center(detection.bbox)
            x1, y1, x2, y2 = detection.bbox
            box_height = max(1, y2 - y1)
            below_pixels = max(0, y2 - max(y1, line_y))
            below_ratio = below_pixels / box_height
            detection.center = center
            detection.below_overlap_ratio = below_ratio
            has_center_in_zone = center[1] >= line_y
            has_enough_box_in_zone = (
                below_pixels >= max(1,
int(config.DETECTION_ZONE_MIN_PIXELS))
                and below_ratio >= max(0.0, min(1.0,
float(config.DETECTION_ZONE_MIN_OVERLAP)))
            )
            if not (has_center_in_zone or has_enough_box_in_zone):
                continue
            detection.below_line = True
            active.append(detection)
        return active

    def _draw_message(self, frame: Any, message: str) -> Any:
        try:

```

```

import cv2

    annotated = self.draw(frame, [])
    cv2.putText(annotated, message[:90], (14, 32),
cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 220, 255), 2)
    return annotated
except Exception:
    return frame

def _line_y(self, frame: Any) -> int:
    height = int(frame.shape[0])
    ratio = min(0.95, max(0.05,
float(config.DETECTION_LINE_Y_RATIO)))
    self.last_line_y = int(height * ratio)
    return self.last_line_y

def _mock_bbox(self, frame: Any) -> tuple[int, int, int, int]:
    height, width = frame.shape[:2]
    line_y = self._line_y(frame)
    y1 = min(height - 80, line_y + 24)
    y2 = min(height - 20, y1 + max(56, height // 5))
    return (width // 3, y1, width * 2 // 3, y2)

@staticmethod
def _infer_backend(model_path: Path) -> str:
    if config.YOLO_BACKEND != "auto":
        return config.YOLO_BACKEND
    path = Path(model_path)
    if path.is_dir() and (path / "model.ncnn.param").exists()
and (path / "model.ncnn.bin").exists():
        return "ncnn"
    suffix = path.suffix.lower()
    if suffix in {".pt", ".torchscript"}:
        return "pytorch"
    if suffix == ".onnx":
        return "onnx"

```

```

        return suffix.lstrip(".") or "auto"

    @staticmethod
    def _load_class_names() -> dict[int, str]:
        metadata_path = Path(config.YOLO_MODEL_PATH) /
"metadata.yaml"
        names: dict[int, str] = {}
        try:
            in_names = False
            for line in metadata_path.read_text(encoding="utf-
8").splitlines():
                stripped = line.strip()
                if stripped == "names:":
                    in_names = True
                    continue
                if in_names and stripped and not line.startswith("
"):
                    break
                if in_names and ":" in stripped:
                    key, value = stripped.split(":", 1)
                    if key.strip().isdigit():
                        names[int(key.strip())] =
value.strip().strip("'\"")
            except Exception:
                pass
            if names:
                return names
            return {index: class_name for index, class_name in
enumerate(config.TARGET_CLASSES)}

def bbox_center(bbox: tuple[int, int, int, int]) -> tuple[int, int]:
    x1, y1, x2, y2 = bbox
    return ((x1 + x2) // 2, (y1 + y2) // 2)

```

```

def clamp_bbox(bbox: tuple[float, float, float, float], frame_width:
int, frame_height: int) -> tuple[int, int, int, int] | None:
    x1, y1, x2, y2 = bbox
    x1 = max(0, min(frame_width - 1, int(round(x1))))
    y1 = max(0, min(frame_height - 1, int(round(y1))))
    x2 = max(0, min(frame_width - 1, int(round(x2))))
    y2 = max(0, min(frame_height - 1, int(round(y2))))
    if x2 <= x1 or y2 <= y1:
        return None
    return (x1, y1, x2, y2)

def box_iou(left: tuple[int, int, int, int] | None, right:
tuple[int, int, int, int] | None) -> float:
    if left is None or right is None:
        return 0.0
    lx1, ly1, lx2, ly2 = left
    rx1, ry1, rx2, ry2 = right
    inter_x1 = max(lx1, rx1)
    inter_y1 = max(ly1, ry1)
    inter_x2 = min(lx2, rx2)
    inter_y2 = min(ly2, ry2)
    inter_width = max(0, inter_x2 - inter_x1)
    inter_height = max(0, inter_y2 - inter_y1)
    intersection = inter_width * inter_height
    if intersection <= 0:
        return 0.0
    left_area = max(0, lx2 - lx1) * max(0, ly2 - ly1)
    right_area = max(0, rx2 - rx1) * max(0, ry2 - ry1)
    union = left_area + right_area - intersection
    return intersection / union if union > 0 else 0.0

```

config.py

```

from __future__ import annotations

```



```

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
PROJECT_ROOT = BASE_DIR.parent
RUNTIME_DIR = BASE_DIR / ".runtime"
RUNTIME_DIR.mkdir(exist_ok=True)

def env_bool(name: str, default: bool = False) -> bool:
    value = os.getenv(name)
    if value is None:
        return default
    return value.strip().lower() in {"1", "true", "yes", "on"}

def env_list(name: str, default: list[str]) -> list[str]:
    value = os.getenv(name)
    if not value:
        return default
    return [item.strip() for item in value.split(",") if
item.strip()]

def first_existing_path(candidates: tuple[Path, ...]) -> Path:
    for path in candidates:
        if path.exists():
            return path
    return candidates[0]

# =====
# Services

```

```

# =====

APP_HOST = os.getenv("WEB_HOST", "0.0.0.0")
APP_PORT = int(os.getenv("WEB_PORT", "8000"))
STREAM_HOST = os.getenv("STREAM_HOST", "0.0.0.0")
STREAM_PORT = int(os.getenv("STREAM_PORT", "5001"))
DEVICE_ID = os.getenv("DEVICE_ID", "trash_sorter_pi_01")
LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO").upper()
STATE_FILE = Path(os.getenv("STATE_FILE", str(RUNTIME_DIR /
"state.json"))))

# =====
# UART Pi <-> ESP32
# =====

UART_PORTS = env_list("UART_PORTS", ["/dev/ttyUSB0", "/dev/ttyACM0",
"/dev/serial0", "/dev/ttyAMA0"])
UART_BAUDRATE = int(os.getenv("UART_BAUDRATE", "115200"))
UART_READ_TIMEOUT_SEC = float(os.getenv("UART_READ_TIMEOUT_SEC",
"0.05"))
UART_RECONNECT_INTERVAL_SEC =
float(os.getenv("UART_RECONNECT_INTERVAL_SEC", "1.0"))
UART_COMMAND_TIMEOUT_SEC =
float(os.getenv("UART_COMMAND_TIMEOUT_SEC", "8.0"))
COMMAND_COOLDOWN_SEC = float(os.getenv("COMMAND_COOLDOWN_SEC",
"1.2"))

# =====
# Camera / YOLO
# =====

CAMERA_INDEX = int(os.getenv("CAMERA_INDEX", "0"))
CAMERA_WIDTH = int(os.getenv("CAMERA_WIDTH", "640"))

```

```

CAMERA_HEIGHT = int(os.getenv("CAMERA_HEIGHT", "480"))
CAMERA_FPS = int(os.getenv("CAMERA_FPS", "20"))
CAMERA_BACKEND = os.getenv("CAMERA_BACKEND", "v4l2" if os.name !=
"nt" else "dshow").strip().lower()
CAMERA_RECONNECT_DELAY_SEC =
float(os.getenv("CAMERA_RECONNECT_DELAY_SEC", "1.0"))
PUBLIC_STREAM_URL = os.getenv("PUBLIC_STREAM_URL",
f"http://127.0.0.1:{STREAM_PORT}/video_feed")

TARGET_CLASSES = ["organic", "metal", "plastic"]
BIN_CLASSES = ["organic", "plastic", "metal"]
HOME_CLASS = os.getenv("HOME_CLASS", "organic").strip().lower()
if HOME_CLASS not in BIN_CLASSES:
    HOME_CLASS = "organic"
YOLO_CONFIDENCE = float(os.getenv("YOLO_CONFIDENCE", "0.35"))
YOLO_IMGSZ = int(os.getenv("YOLO_IMGSZ", "416"))
YOLO_DEVICE = os.getenv("YOLO_DEVICE", "").strip()
DETECT_EVERY_N_FRAMES = int(os.getenv("DETECT_EVERY_N_FRAMES",
"2"))
ENTRY_DETECT_EVERY_N_FRAMES =
int(os.getenv("ENTRY_DETECT_EVERY_N_FRAMES",
str(DETECT_EVERY_N_FRAMES)))
DETECTION_PREVIEW_WHEN_IDLE =
env_bool("DETECTION_PREVIEW_WHEN_IDLE", True)
MOCK_DETECTION = env_bool("MOCK_DETECTION", False)
MOCK_CLASS = os.getenv("MOCK_CLASS", "plastic")
STREAM_JPEG_QUALITY = int(os.getenv("STREAM_JPEG_QUALITY", "75"))

LOCAL_NCNN_MODEL_PATH = BASE_DIR / "best_ncnn_model"
SHARED_NCNN_MODEL_PATH = PROJECT_ROOT / "best_ncnn_model" /
"best_ncnn_model"
YOLO_MODEL_PATH = Path(os.getenv("YOLO_MODEL_PATH",
str(first_existing_path((LOCAL_NCNN_MODEL_PATH,
SHARED_NCNN_MODEL_PATH))))))
YOLO_BACKEND = os.getenv("YOLO_BACKEND", "auto").strip().lower()
NCNN_NUM_THREADS = int(os.getenv("NCNN_NUM_THREADS", "4"))

```

```

NMS_IΟΥ_THRESHOLD = float(os.getenv("NMS_IΟΥ_THRESHOLD", "0.45"))
DETECTION_LINE_Y_RATIO = float(os.getenv("DETECTION_LINE_Y_RATIO",
"0.55"))
DETECTION_ZONE_MIN_OVERLAP =
float(os.getenv("DETECTION_ZONE_MIN_OVERLAP", "0.20"))
DETECTION_ZONE_MIN_PIXELS =
int(os.getenv("DETECTION_ZONE_MIN_PIXELS", "12"))
DETECTION_TRACK_MAX_DISTANCE =
int(os.getenv("DETECTION_TRACK_MAX_DISTANCE", "90"))
DETECTION_TRACK_MAX_AGE_FRAMES =
int(os.getenv("DETECTION_TRACK_MAX_AGE_FRAMES", "30"))
ENTRY_TRIGGERED_DETECTION = env_bool("ENTRY_TRIGGERED_DETECTION",
True)
ENTRY_DETECT_TIMEOUT_SEC =
float(os.getenv("ENTRY_DETECT_TIMEOUT_SEC", "2.0"))
ENTRY_NO_DETECTION_RESUME = env_bool("ENTRY_NO_DETECTION_RESUME",
True)
RESUME_CONVEYOR_AFTER_SORT = env_bool("RESUME_CONVEYOR_AFTER_SORT",
True)
SORT_ON_EXIT_SENSOR = env_bool("SORT_ON_EXIT_SENSOR", True)
ENTRY_SENSOR_DEBOUNCE_SEC =
float(os.getenv("ENTRY_SENSOR_DEBOUNCE_SEC", "0.25"))
ENTRY_CLEAR_DEBOUNCE_SEC =
float(os.getenv("ENTRY_CLEAR_DEBOUNCE_SEC", "0.15"))
ENTRY_OCCUPIED_LOCK_SEC =
float(os.getenv("ENTRY_OCCUPIED_LOCK_SEC", "0.4"))
EXIT_SENSOR_DEBOUNCE_SEC =
float(os.getenv("EXIT_SENSOR_DEBOUNCE_SEC", "0.2"))
RESET_COOLDOWN_SEC = float(os.getenv("RESET_COOLDOWN_SEC", "0.5"))
DETECTOR_COOLDOWN_SEC = float(os.getenv("DETECTOR_COOLDOWN_SEC",
"0.1"))
OBJECT_STABLE_DELAY_SEC =
float(os.getenv("OBJECT_STABLE_DELAY_SEC", "0.6"))
DETECTION_WINDOW_SEC = float(os.getenv("DETECTION_WINDOW_SEC",
"2.5"))
MIN_VOTES_TO_CONFIRM = int(os.getenv("MIN_VOTES_TO_CONFIRM", "3"))
MIN_CONFIDENCE = float(os.getenv("MIN_CONFIDENCE", "0.35"))

```

```

MIN_DOMINANT_RATIO = float(os.getenv("MIN_DOMINANT_RATIO", "0.5"))
EXIT_TIMEOUT_SEC = float(os.getenv("EXIT_TIMEOUT_SEC", "15.0"))
SORTING_COMMAND_PREFIX = os.getenv("SORTING_COMMAND_PREFIX",
"SORT").strip().upper()
if SORTING_COMMAND_PREFIX not in {"SORT", "BIN"}:
    SORTING_COMMAND_PREFIX = "SORT"

# =====
# Queue / Realtime behavior
# =====

QUEUE_MAX_SIZE = int(os.getenv("QUEUE_MAX_SIZE", "10"))
MAX_OBJECT_QUEUE_SIZE = int(os.getenv("MAX_OBJECT_QUEUE_SIZE",
str(QUEUE_MAX_SIZE)))
SAME_CLASS_COOLDOWN_SEC =
float(os.getenv("SAME_CLASS_COOLDOWN_SEC", "3.0"))
SORT_COMMAND_TIMEOUT_SEC =
float(os.getenv("SORT_COMMAND_TIMEOUT_SEC",
str(UART_COMMAND_TIMEOUT_SEC)))
MOTOR_DONE_TIMEOUT_SEC = float(os.getenv("MOTOR_DONE_TIMEOUT_SEC",
str(SORT_COMMAND_TIMEOUT_SEC)))
MAIN_LOOP_SLEEP_SEC = float(os.getenv("MAIN_LOOP_SLEEP_SEC",
"0.01"))
DASHBOARD_PUSH_INTERVAL_SEC =
float(os.getenv("DASHBOARD_PUSH_INTERVAL_SEC", "0.5"))
STATUS_STALE_TIMEOUT_SEC =
float(os.getenv("STATUS_STALE_TIMEOUT_SEC", "5.0"))
ESP32_OFFLINE_TIMEOUT_SEC =
float(os.getenv("ESP32_OFFLINE_TIMEOUT_SEC", "5.0"))

# =====
# Firebase mirror only
# =====

```

```

FIREBASE_ENABLED = env_bool("FIREBASE_ENABLED", True)
FIREBASE_DATABASE_URL = os.getenv("FIREBASE_DATABASE_URL",
"https://smartlock-6fb00-default-rtdb.firebaseio.com").strip()
FIREBASE_SERVICE_ACCOUNT_PATH = os.getenv(
    "FIREBASE_SERVICE_ACCOUNT_PATH",
    str(first_existing_path((PROJECT_ROOT / "firebase" /
"firebase_key.json", PROJECT_ROOT / "firebase" / "smartlock-6fb00-
firebase-adminsdk-fbsvc-3f661b3583.json"))),
).strip()
FIREBASE_ROOT_PATH = os.getenv("FIREBASE_ROOT_PATH",
"trash_sorting_system").strip("/")
FIREBASE_SYNC_INTERVAL_SEC =
float(os.getenv("FIREBASE_SYNC_INTERVAL_SEC", "1.0"))
FIREBASE_CONNECT_RETRY_SEC =
float(os.getenv("FIREBASE_CONNECT_RETRY_SEC", "5.0"))

FIREBASE_STATUS_PATH = f"{FIREBASE_ROOT_PATH}/status"
FIREBASE_RASPBERRY_PI_PATH = f"{FIREBASE_ROOT_PATH}/raspberry_pi"
FIREBASE_ESP32_PATH = f"{FIREBASE_ROOT_PATH}/esp32"
FIREBASE_COUNTER_PATH = f"{FIREBASE_ROOT_PATH}/counters"
FIREBASE_LAST_DETECTION_PATH =
f"{FIREBASE_ROOT_PATH}/last_detection"
FIREBASE_LOG_PATH = f"{FIREBASE_ROOT_PATH}/logs"

# =====
# Web commands
# =====

CONVEYOR_SPEED_MIN = int(os.getenv("CONVEYOR_SPEED_MIN", "40"))
CONVEYOR_SPEED_MAX = int(os.getenv("CONVEYOR_SPEED_MAX", "600"))
CONVEYOR_DEFAULT_SPEED_SPS =
int(os.getenv("CONVEYOR_DEFAULT_SPEED_SPS", "140"))

```

```

COMMANDS = {
    "start",
    "stop",
    "home",
    "reset_counter",
    "conveyor_start",
    "conveyor_stop",
    "conveyor_speed",
    "rotate_organic",
    "rotate_metal",
    "rotate_plastic",
    "sort_organic",
    "sort_metal",
    "sort_plastic",
    "ai_start",
    "ai_stop",
    "camera_restart",
}

HARDWARE_PROFILE = {
    "controller": "ESP32 DevKit V1",
    "edge": "Raspberry Pi + YOLOv11",
    "command_flow": "Camera -> YOLO -> UART -> ESP32; Firebase
mirror only",
    "uart": {"esp32_rx2": 16, "esp32_tx2": 17, "baudrate":
UART_BAUDRATE},
    "conveyor_stepper": {"step": 25, "dir": 26, "en": 27},
    "bin_actuator": {"owner": "ESP32", "type": "servo_360", "home":
HOME_CLASS},
    "sorting_bins": BIN_CLASSES,
    "sensors": {"entry": 32, "exit": 33, "home": 34},
    "peripherals": {"lcd_sda": 21, "lcd_scl": 22, "buzzer": 23,
"status_led": 2},
}

```

```

def normalize_class(value: object) -> str:
    text = str(value or "unknown").strip().lower().replace("-",
"_").replace(" ", "_")
    aliases = {
        "banana_peel": "organic",
        "orange_peel": "organic",
        "food": "organic",
        "food_waste": "organic",
        "huu_co": "organic",
        "kim_loai": "metal",
        "can": "metal",
        "nhua": "plastic",
        "bottle": "plastic",
        "plastic_bottle": "plastic",
    }
    text = aliases.get(text, text)
    return text if text in TARGET_CLASSES else "unknown"

```

firebase_sync.py

```

from __future__ import annotations

import logging
import threading
import time
from pathlib import Path
from typing import Any

import config
from state_store import StateStore

class FirebaseSync:
    """Best-effort Firebase mirror. Never controls realtime

```



```

sorting."""

    def __init__(self, store: StateStore, logger: logging.Logger |
None = None) -> None:
        self.store = store
        self.logger = logger or logging.getLogger(__name__)
        self.db = None
        self.running = False
        self.thread: threading.Thread | None = None
        self.next_connect_at = 0.0
        self.last_error = ""

    def start(self) -> None:
        if self.running:
            return
        self.running = True
        self.thread = threading.Thread(target=self._run,
name="firebase-sync", daemon=True)
        self.thread.start()

    def stop(self) -> None:
        self.running = False

    def sync_once(self) -> bool:
        if not config.FIREBASE_ENABLED:
            self.store.set_firebase_status(False, "disabled")
            return False
        if not self._connect():
            self.store.set_firebase_status(False, self.last_error)
            return False
        try:
            payload = self.store.dashboard_payload()

self.db.reference(config.FIREBASE_STATUS_PATH).update(payload["status"])

```

```

self.db.reference(config.FIREBASE_RASPBERRY_PI_PATH).update(payload[
"raspberry_pi"])

self.db.reference(config.FIREBASE_ESP32_PATH).update(payload["esp32"
])

self.db.reference(config.FIREBASE_COUNTER_PATH).set(payload["counter
s"])

        if payload.get("last_detection"):

self.db.reference(config.FIREBASE_LAST_DETECTION_PATH).set(payload["
last_detection"])

        self.last_error = ""
        self.store.set_firebase_status(True, "")
        return True
    except Exception as exc:
        self.last_error = f"FIREBASE_SYNC_FAILED:{exc}"
        self.logger.warning(self.last_error)
        self.store.set_firebase_status(False, self.last_error)
        return False

def _run(self) -> None:
    while self.running:
        self.sync_once()
        time.sleep(max(0.2, config.FIREBASE_SYNC_INTERVAL_SEC))

def _connect(self) -> bool:
    if self.db is not None:
        return True
    if time.time() < self.next_connect_at:
        return False
    self.next_connect_at = time.time() +
config.FIREBASE_CONNECT_RETRY_SEC
    try:
        import firebase_admin
        from firebase_admin import credentials, db

```

```

        if not firebase_admin._apps:
            key_path = str(config.FIREBASE_SERVICE_ACCOUNT_PATH
or "").strip()

            if key_path:
                path = Path(key_path)
                if not path.is_absolute():
                    path = config.BASE_DIR / path
                if not path.exists():
                    self.last_error =
f"FIREBASE_KEY_NOT_FOUND:{path}"
                    return False
                cred = credentials.Certificate(str(path))
                firebase_admin.initialize_app(cred,
{"databaseURL": config.FIREBASE_DATABASE_URL})
            else:

firebase_admin.initialize_app(options={"databaseURL":
config.FIREBASE_DATABASE_URL})
                self.db = db
                self.last_error = ""
                return True
        except Exception as exc:
            self.db = None
            self.last_error = f"FIREBASE_CONNECT_FAILED:{exc}"
            self.logger.warning(self.last_error)
            return False

```

state_store.py

```

from __future__ import annotations

import json
import os
import tempfile
import threading
import time
from pathlib import Path

```

```

from typing import Any

import config

def now_ms() -> int:
    return int(time.time() * 1000)

def now_text() -> str:
    return time.strftime("%Y-%m-%d %H:%M:%S")

def as_int(value: Any, default: int = 0) -> int:
    try:
        return int(float(value))
    except (TypeError, ValueError):
        return default

class StateStore:
    """Small file-backed state store shared by dashboard and YOLO
    process."""

    def __init__(self, path: Path | None = None) -> None:
        self.path = Path(path or config.STATE_FILE)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.lock = threading.RLock()
        if not self.path.exists():
            self.write(self._default_state())

    def snapshot(self) -> dict[str, Any]:
        with self.lock:

```

```

        return self._read()

    def write(self, data: dict[str, Any]) -> None:
        with self.lock:
            data.setdefault("schema", 1)
            data["updated_at_ms"] = now_ms()
            data["updated_at"] = now_text()
            fd, tmp_name = tempfile.mkstemp(prefix="state-",
suffix=".json", dir=str(self.path.parent))
            try:
                with os.fdopen(fd, "w", encoding="utf-8") as tmp:
                    json.dump(data, tmp, separators=(",", ":"),
ensure_ascii=True)
                os.replace(tmp_name, self.path)
            finally:
                if os.path.exists(tmp_name):
                    os.unlink(tmp_name)

    def update(self, section: str, values: dict[str, Any]) ->
dict[str, Any]:
        with self.lock:
            data = self._read()
            target = data.setdefault(section, {})
            if not isinstance(target, dict):
                target = {}
            data[section] = target
            target.update(values)
            target["updated_at_ms"] = now_ms()
            target["updated_at"] = now_text()
            self.write(data)
            return data

    def update_status(self, **values: Any) -> None:
        self.update("status", values)

    def update_esp32(self, **values: Any) -> None:

```

```

        values.setdefault("online", True)
        values.setdefault("last_seen_ms", now_ms())
        self.update("esp32", values)

def update_raspberry_pi(self, **values: Any) -> None:
    values.setdefault("online", True)
    values.setdefault("last_seen_ms", now_ms())
    self.update("raspberry_pi", values)

def update_detection(self, detection: dict[str, Any]) -> None:
    with self.lock:
        data = self._read()
        payload = dict(detection)
        payload.setdefault("created_at_ms", now_ms())
        payload.setdefault("created_at", now_text())
        data["last_detection"] = payload
        status = data.setdefault("status", {})
        status.update(
            {
                "current_class": payload.get("class_name",
"unknown"),
                "current_object_id": payload.get("object_id"),
                "confidence": payload.get("confidence", 0.0),
                "detection_line_y": payload.get("line_y"),
                "detection_zone": payload.get("zone"),
                "detection_source": payload.get("source"),
                "last_detection_ms": payload["created_at_ms"],
            }
        )
        self.write(data)

def increment_counter(self, class_name: str) -> None:
    with self.lock:
        data = self._read()
        counters = data.setdefault("counters", {"organic": 0,
"metal": 0, "plastic": 0, "total": 0})

```

```

        if class_name in {"organic", "metal", "plastic"}:
            counters[class_name] =
as_int(counters.get(class_name), 0) + 1
            counters["total"] = as_int(counters.get("organic"), 0)
+ as_int(counters.get("metal"), 0) + as_int(counters.get("plastic"),
0)

        self.write(data)

    def reset_counters(self) -> None:
        with self.lock:
            data = self._read()
            data["counters"] = {"organic": 0, "metal": 0,
"plastic": 0, "total": 0}
            self.add_log("Counters reset", "INFO", data=data)
            self.write(data)

    def add_log(self, message: str, level: str = "INFO", data:
dict[str, Any] | None = None) -> None:
        with self.lock:
            state = data if data is not None else self._read()
            logs = state.setdefault("logs", [])
            logs.append({"created_at_ms": now_ms(), "created_at":
now_text(), "level": level, "message": message})
            state["logs"] = logs[-100:]
            if data is None:
                self.write(state)

    def set_pending_command(self, action: str, line: str, value: Any
= None, source: str = "web") -> dict[str, Any]:
        with self.lock:
            data = self._read()
            command = {
                "id": f"cmd_{now_ms()}",
                "action": action,
                "line": line,
                "value": value,

```

```

        "source": source,
        "status": "pending",
        "created_at_ms": now_ms(),
        "created_at": now_text(),
    }
    data["pending_command"] = command
    data["last_command"] = command
    self.write(data)
    return command

def consume_pending_command(self) -> dict[str, Any] | None:
    with self.lock:
        data = self._read()
        command = data.get("pending_command")
        if not isinstance(command, dict) or
command.get("status") != "pending":
            return None
        command = dict(command)
        data["pending_command"] = {**command, "status":
"consumed", "consumed_at_ms": now_ms()}
        data["last_command"] = data["pending_command"]
        self.write(data)
        return command

def complete_command(self, command_id: str, status: str,
message: str = "") -> None:
    with self.lock:
        data = self._read()
        command = dict(data.get("last_command") or {})
        if command_id and command.get("id") != command_id:
            return
        command.update({"status": status, "message": message,
"completed_at_ms": now_ms(), "completed_at": now_text()})
        data["last_command"] = command
        self.write(data)

```



```

def set_firebase_status(self, connected: bool, error: str = "")
-> None:
    self.update("firebase", {"enabled":
config.FIREBASE_ENABLED, "connected": connected, "error": error,
"mode": "mirror"})

def dashboard_payload(self) -> dict[str, Any]:
    data = self.snapshot()
    status = {**data.get("status", {})}
    esp32 = data.get("esp32", {}) if
isinstance(data.get("esp32"), dict) else {}
    pi = data.get("raspberrypi", {}) if
isinstance(data.get("raspberrypi"), dict) else {}
    counters = data.get("counters", {}) if
isinstance(data.get("counters"), dict) else {}

    checked_ms = now_ms()
    esp32_last_seen_ms = as_int(esp32.get("last_seen_ms"), 0)
    esp32_age_ms = checked_ms - esp32_last_seen_ms if
esp32_last_seen_ms else None
    esp32_online = bool(esp32.get("online")) and esp32_age_ms
is not None and esp32_age_ms <= int(config.ESP32_OFFLINE_TIMEOUT_SEC
* 1000)

    pi_last_seen_ms = as_int(pi.get("last_seen_ms"), 0)
    pi_age_ms = checked_ms - pi_last_seen_ms if pi_last_seen_ms
else None
    pi_online = bool(pi.get("online")) and pi_age_ms is not
None and pi_age_ms <= int(config.STATUS_STALE_TIMEOUT_SEC * 1000)

    status.update(
        {
            "system_status": status.get("system_status",
"RUNNING" if pi_online else "OFFLINE"),
            "runtime_state": status.get("runtime_state",
"IDLE"),

```

```

        "object_queue": status.get("object_queue", []),
        "active_object": status.get("active_object"),
        "raspberry_pi_status": "ONLINE" if pi_online else
"OFFLINE",
        "esp32_status": "ONLINE" if esp32_online else
"OFFLINE",
        "esp32_display_status": "ESP32 ONLINE" if
esp32_online else "ESP32 OFFLINE",
        "esp32_age_ms": esp32_age_ms,
        "esp32_last_seen_ms": esp32_last_seen_ms,
        "camera_stream_url":
status.get("camera_stream_url") or config.PUBLIC_STREAM_URL,
        "camera_status": status.get("camera_status",
"UNKNOWN"),
        "uart_status": status.get("uart_status",
"DISCONNECTED"),
        "queue_depth": status.get("queue_depth", 0),
        "conveyor_speed_sps":
esp32.get("conveyor_speed_sps", status.get("conveyor_speed_sps",
config.CONVEYOR_DEFAULT_SPEED_SPS)),
        "current_bin": esp32.get("current_bin",
status.get("current_bin")),
        "conveyor_running": esp32.get("conveyor_running",
status.get("conveyor_running")),
        "sensor_entry": esp32.get("sensor_entry",
status.get("sensor_entry")),
        "sensor_exit": esp32.get("sensor_exit",
status.get("sensor_exit")),
        "home_switch": esp32.get("home_switch",
status.get("home_switch")),
        "last_error": status.get("last_error", ""),
    }
)
counters.setdefault("organic", 0)
counters.setdefault("metal", 0)
counters.setdefault("plastic", 0)
counters["total"] = as_int(counters.get("total"),

```

```

as_int(counters.get("organic")) + as_int(counters.get("metal")) +
as_int(counters.get("plastic")))
    return {
        "ok": True,
        "status": status,
        "counters": counters,
        "raspberry_pi": pi,
        "esp32": esp32,
        "last_detection": data.get("last_detection", {}),
        "object_queue": status.get("object_queue", []),
        "active_object": status.get("active_object"),
        "logs": data.get("logs", [])[-30:],
        "last_command": data.get("last_command", {}),
        "hardware": config.HARDWARE_PROFILE,
        "firebase": data.get("firebase", {"enabled":
config.FIREBASE_ENABLED, "connected": False, "mode": "mirror",
"error": ""}),
    }

def _read(self) -> dict[str, Any]:
    try:
        return json.loads(self.path.read_text(encoding="utf-
8"))
    except Exception:
        return self._default_state()

def _default_state(self) -> dict[str, Any]:
    timestamp = now_ms()
    return {
        "schema": 1,
        "updated_at_ms": timestamp,
        "status": {
            "system_status": "STARTING",
            "runtime_state": "IDLE",
            "current_state": "IDLE",
            "detector_state": "DETECTOR_IDLE",

```

```

        "sorter_state": "SORTER_IDLE",
        "raspberry_pi_status": "UNKNOWN",
        "esp32_status": "UNKNOWN",
        "camera_status": "UNKNOWN",
        "uart_status": "DISCONNECTED",
        "camera_stream_url": config.PUBLIC_STREAM_URL,
        "current_class": "unknown",
        "current_object_id": None,
        "current_detection_id": None,
        "current_detection_class": "unknown",
        "confirmed_class": None,
        "object_already_created": False,
        "is_processing_object": False,
        "sort_command_sent": False,
        "motor_done": False,
        "detection_votes": {},
        "valid_detection_count": 0,
        "queue_size": 0,
        "pending_job": None,
        "confidence": 0.0,
        "detection_line_y": None,
        "detection_zone": "",
        "detection_source": "",
        "fps": 0.0,
        "queue_depth": 0,
        "conveyor_speed_sps":
config.CONVEYOR_DEFAULT_SPEED_SPS,
        "last_error": "",
        "object_queue": [],
        "active_object": None,
    },
    "raspberry_pi": {"online": False, "last_seen_ms": 0},
    "esp32": {"online": False, "last_seen_ms": 0},
    "counters": {"organic": 0, "metal": 0, "plastic": 0,
"total": 0},
    "last_detection": {},
    "logs": [],

```

```

        "last_command": {},
        "pending_command": {},
        "firebase": {"enabled": config.FIREBASE_ENABLED,
"connected": False, "mode": "mirror", "error": ""},
    }

```

uart_client.py

```

from __future__ import annotations

import logging
import os
import queue
import threading
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Callable

import config

@dataclass
class UartEvent:
    kind: str
    raw: str
    command: str = ""
    fields: dict[str, Any] | None = None

class UartClient:
    def __init__(
        self,
        ports: list[str] | None = None,
        baudrate: int | None = None,
        logger: logging.Logger | None = None,

```

```

    ) -> None:
        self.ports = ports or self._default_ports()
        self.baudrate = baudrate or config.UART_BAUDRATE
        self.logger = logger or logging.getLogger(__name__)
        self.serial = None
        self.lock = threading.RLock()
        self.running = False
        self.thread: threading.Thread | None = None
        self.callbacks: list[Callable[[UartEvent], None]] = []
        self.events: "queue.Queue[UartEvent]" =
queue.Queue(maxsize=200)
        self.last_error = ""
        self.last_rx_at = 0.0
        self.last_connect_attempt = 0.0

    @property
    def connected(self) -> bool:
        with self.lock:
            return bool(self.serial and self.serial.is_open and
self._port_present())

    def start(self, callback: Callable[[UartEvent], None] | None =
None) -> None:
        if callback is not None:
            self.callbacks.append(callback)
        if self.running:
            return
        self.running = True
        self.thread = threading.Thread(target=self._run,
name="uart-client", daemon=True)
        self.thread.start()

    def stop(self) -> None:
        self.running = False
        self.close()

```

```

def send_line(self, line: str) -> bool:
    text = str(line or "").strip()
    if not text:
        self.last_error = "UART empty line"
        return False
    if not self._ensure_connected():
        return False
    try:
        with self.lock:
            self.serial.write((text + "\n").encode("utf-8"))
            self.serial.flush()
            self.logger.info("UART -> ESP32: %s", text)
            return True
    except Exception as exc:
        self.last_error = f"UART_WRITE_FAILED:{exc}"
        self.logger.exception(self.last_error)
        self.close()
        return False

def get_event(self, timeout: float = 0.0) -> UartEvent | None:
    try:
        return self.events.get(timeout=timeout)
    except queue.Empty:
        return None

def close(self) -> None:
    with self.lock:
        if self.serial is not None:
            try:
                self.serial.close()
            except Exception:
                pass
        self.serial = None

def _run(self) -> None:
    while self.running:

```

```

        if not self._ensure_connected():
            time.sleep(0.1)
            continue
        try:
            with self.lock:
                raw = self.serial.readline()
            if not raw:
                continue
            line = raw.decode("utf-8",
errors="replace").strip()
            if not line:
                continue
            event = self.parse_line(line)
            self.last_rx_at = time.time()
            self.logger.info("UART <- ESP32: %s", line)
            self._publish(event)
        except Exception as exc:
            self.last_error = f"UART_READ_FAILED:{exc}"
            self.logger.exception(self.last_error)
            self.close()
            time.sleep(config.UART_RECONNECT_INTERVAL_SEC)

def _publish(self, event: UartEvent) -> None:
    try:
        self.events.put_nowait(event)
    except queue.Full:
        try:
            self.events.get_nowait()
            self.events.put_nowait(event)
        except queue.Empty:
            pass
    for callback in list(self.callbacks):
        try:
            callback(event)
        except Exception:
            self.logger.exception("UART callback failed")

```



```

def _ensure_connected(self) -> bool:
    if self.connected:
        return True
    if time.time() - self.last_connect_attempt <
config.UART_RECONNECT_INTERVAL_SEC:
        return False
    self.last_connect_attempt = time.time()
    return self._connect()

def _connect(self) -> bool:
    try:
        import serial
    except Exception as exc:
        self.last_error = f"PYSERIAL_NOT_AVAILABLE:{exc}"
        self.logger.error(self.last_error)
        return False

    for port in self.ports:
        if os.name != "nt" and port.startswith("/") and not
Path(port).exists():
            continue
        try:
            candidate = serial.Serial(port, self.baudrate,
timeout=config.UART_READ_TIMEOUT_SEC)
            with self.lock:
                self.serial = candidate
                self.last_error = ""
                self.logger.info("UART connected port=%s baud=%s",
port, self.baudrate)
            return True
        except Exception as exc:
            self.last_error = f"{port}:{exc}"
            self.logger.warning("UART open failed: %s",
self.last_error)
    return False

```

```

def _port_present(self) -> bool:
    if self.serial is None:
        return False
    port = str(getattr(self.serial, "port", "") or "")
    if os.name != "nt" and port.startswith("/"):
        return Path(port).exists()
    return True

def _default_ports(self) -> list[str]:
    ports = list(config.UART_PORTS)
    if os.name == "nt":
        ports.extend([f"COM{i}" for i in range(1, 20)])
    return list(dict.fromkeys(ports))

@staticmethod
def parse_line(line: str) -> UartEvent:
    text = line.strip()
    upper = text.upper()
    if upper.startswith("ACK:"):
        return UartEvent("ACK", text, command=text.split(":",
1) [1].strip())
    if upper.startswith("DONE:"):
        return UartEvent("DONE", text, command=text.split(":",
1) [1].strip())
    if upper.startswith("ERROR:"):
        return UartEvent("ERROR", text, command=text.split(":",
1) [1].strip())
    if upper.startswith("SENSOR:"):
        return UartEvent("SENSOR", text,
fields=UartClient._parse_fields(text.split(":", 1) [1]))
    if upper.startswith("STATUS:"):
        return UartEvent("STATUS", text,
fields=UartClient._parse_fields(text.split(":", 1) [1]))
    return UartEvent("LINE", text)

```

```

@staticmethod
def _parse_fields(payload: str) -> dict[str, Any]:
    fields: dict[str, Any] = {}
    for part in payload.split(","):
        if "=" not in part:
            continue
        key, value = part.split("=", 1)
        key = key.strip().lower()
        raw = value.strip()
        if raw in {"0", "1"}:
            fields[key] = raw == "1"
        else:
            fields[key] = raw
    return fields

```

PHỤ LỤC B – FIRMWARE ESP32 DEVKIT-V1

```

#include <Arduino.h>
#include <ESP32Servo.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

// ESP32 realtime UART firmware for Trash Sorting System.
// Raspberry Pi UART:
//   Pi TX -> ESP32 RX2 GPIO16
//   Pi RX -> ESP32 TX2 GPIO17
//   GND   -> ESP32 GND
//
// Bin actuator:
//   Servo 180 signal -> GPIO18
//   home position    -> organic
//
// Detection LED:
//   LED -> GPIO2
//
// Home button:
//   Button -> GPIO34, press to return servo to organic angle.

```

```

// GPIO34 has no internal pull-up/pull-down, so use an external
resistor.

constexpr uint32_t UART_BAUDRATE = 115200;

constexpr int UART_RX2_PIN = 16;
constexpr int UART_TX2_PIN = 17;

constexpr int CONVEYOR_STEP_PIN = 27;
constexpr int CONVEYOR_DIR_PIN = 26;
constexpr int CONVEYOR_EN_PIN = 25;

constexpr int BIN_SERVO_PIN = 18;
constexpr int DETECTED_LED_PIN = 2;
constexpr int HOME_BUTTON_PIN = 34;

constexpr int SENSOR_ENTRY_PIN = 32;
constexpr int SENSOR_EXIT_PIN = 33;

constexpr bool STEPPER_ENABLE_ACTIVE_LOW = true;
constexpr bool SENSOR_ACTIVE_LOW = true;
constexpr bool HOME_BUTTON_ACTIVE_LOW = true;
constexpr bool DETECTED_LED_ACTIVE_HIGH = true;

constexpr float CONVEYOR_DEFAULT_SPEED_SPS = 140.0f;
constexpr float CONVEYOR_MIN_SPEED_SPS = 40.0f;
constexpr float CONVEYOR_MAX_SPEED_SPS = 600.0f;
constexpr bool CONVEYOR_FORWARD_POSITIVE = true;
constexpr bool STEPPER_HOLD_WHEN_STOPPED = false;
constexpr uint32_t A4988_STEP_HIGH_US = 4;
constexpr uint32_t STEPPER_TASK_IDLE_MS = 5;
constexpr uint32_t STEPPER_TASK_STACK_WORDS = 2048;
constexpr UBBaseType_t STEPPER_TASK_PRIORITY = 3;
constexpr BaseType_t STEPPER_TASK_CORE = 0;

```

```

// 180-degree servo bin positions. Adjust these values on hardware
if needed.

constexpr int SERVO_ORGANIC_ANGLE = 90;
constexpr int SERVO_PLASTIC_ANGLE = 5;
constexpr int SERVO_METAL_ANGLE = 175;
constexpr uint32_t SERVO_MOVE_MS = 700;
constexpr uint32_t DETECTED_LED_HOLD_MS = 1500;

constexpr uint32_t SENSOR_DEBOUNCE_MS = 40;
constexpr uint32_t STATUS_INTERVAL_MS = 1000;
constexpr uint32_t SENSOR_REPORT_INTERVAL_MS = 250;
// Entry is reported as both a one-shot object event and the live
beam state.
constexpr uint32_t ENTRY_OBJECT_TIMEOUT_MS = 8000;
constexpr uint32_t ENTRY_REARM_CLEAR_MS = 150;

enum class RunState : uint8_t {
    IDLE,
    RUNNING,
    SORTING_SERVO,
    BIN_SERVO,
    ERROR_STATE
};

class DebouncedInput {
public:
    void begin(int gpioPin, bool activeLow, uint8_t mode) {
        pin = gpioPin;
        inverted = activeLow;
        pinMode(pin, mode);
        stableState = readActive();
        lastRawState = stableState;
        changedAt = millis();
    }
}

```

```

void update() {
    const bool raw = readActive();
    const uint32_t now = millis();
    roseEvent = false;
    fellEvent = false;

    if (raw != lastRawState) {
        lastRawState = raw;
        changedAt = now;
    }

    if ((now - changedAt) >= SENSOR_DEBOUNCE_MS && stableState !=
raw) {
        const bool previous = stableState;
        stableState = raw;
        roseEvent = !previous && stableState;
        fellEvent = previous && !stableState;
    }
}

bool active() const {
    return stableState;
}

bool rose() {
    const bool value = roseEvent;
    roseEvent = false;
    return value;
}

private:
    bool readActive() const {
        const bool raw = digitalRead(pin);
        return inverted ? !raw : raw;
    }

```

```

    }

    int pin = -1;
    bool inverted = true;
    bool stableState = false;
    bool lastRawState = false;
    bool roseEvent = false;
    bool fellEvent = false;
    uint32_t changedAt = 0;
};

Servo binServo;
DebouncedInput entrySensor;
DebouncedInput exitSensor;
DebouncedInput homeButton;
TaskHandle_t conveyorTaskHandle = nullptr;
portMUX_TYPE conveyorMux = portMUX_INITIALIZER_UNLOCKED;

RunState state = RunState::IDLE;
String currentBin = "organic";
String servoTargetBin = "organic";
String servoDoneCommand = "";
String activeSortClass = "";
String usbRx;
String uartRx;
volatile bool conveyorRunning = false;
bool resumeConveyorAfterServo = false;
volatile float conveyorSpeedSps = CONVEYOR_DEFAULT_SPEED_SPS;
uint32_t servoMoveStartedAt = 0;
uint32_t servoMoveDurationMs = 0;
uint32_t ledOffAt = 0;
uint32_t lastStatusAt = 0;
uint32_t lastSensorReportAt = 0;
bool lastEntry = false;
bool lastEntryActive = false;
bool lastExit = false;

```

```

bool lastHome = true;
bool detectedLedOn = false;
bool entryObjectLocked = false;
bool entryPulsePending = false;
uint32_t entryObjectLockedAt = 0;
uint32_t entryInactiveSince = 0;

bool isConveyorRunning() {
    portENTER_CRITICAL(&conveyorMux);
    const bool running = conveyorRunning;
    portEXIT_CRITICAL(&conveyorMux);
    return running;
}

float getConveyorSpeedSps() {
    portENTER_CRITICAL(&conveyorMux);
    const float speed = conveyorSpeedSps;
    portEXIT_CRITICAL(&conveyorMux);
    return speed;
}

void setConveyorSpeedSps(float speed) {
    speed = constrain(speed, CONVEYOR_MIN_SPEED_SPS,
CONVEYOR_MAX_SPEED_SPS);
    portENTER_CRITICAL(&conveyorMux);
    conveyorSpeedSps = speed;
    portEXIT_CRITICAL(&conveyorMux);
}

void setConveyorRunning(bool running) {
    portENTER_CRITICAL(&conveyorMux);
    conveyorRunning = running;
    portEXIT_CRITICAL(&conveyorMux);
}

```



```

void enableDriver(int pin, bool enabled) {
    const int active = STEPPER_ENABLE_ACTIVE_LOW ? LOW : HIGH;
    const int inactive = STEPPER_ENABLE_ACTIVE_LOW ? HIGH : LOW;
    digitalWrite(pin, enabled ? active : inactive);
}

void writeDetectedLed(bool on) {
    detectedLedOn = on;
    const int active = DETECTED_LED_ACTIVE_HIGH ? HIGH : LOW;
    const int inactive = DETECTED_LED_ACTIVE_HIGH ? LOW : HIGH;
    digitalWrite(DETECTED_LED_PIN, on ? active : inactive);
}

void lightDetectedLed(uint32_t holdMs = 0) {
    writeDetectedLed(true);
    ledOffAt = holdMs > 0 ? millis() + holdMs : 0;
}

void updateDetectedLed() {
    if (detectedLedOn && ledOffAt > 0 && millis() >= ledOffAt &&
state != RunState::SORTING_SERVO) {
        writeDetectedLed(false);
        ledOffAt = 0;
    }
}

const char *stateText() {
    switch (state) {
        case RunState::IDLE:
            return "IDLE";
        case RunState::RUNNING:
            return "RUNNING";
        case RunState::SORTING_SERVO:
            return "SORTING_SERVO";
        case RunState::BIN_SERVO:
            return "BIN_SERVO";
    }
}

```

```

        case RunState::ERROR_STATE:
            return "ERROR";
        }
        return "UNKNOWN";
    }

void sendLine(const String &line) {
    Serial2.println(line);
    Serial.println(line);
}

void sendAck(const String &command) {
    sendLine("ACK:" + command);
}

void sendDone(const String &command) {
    sendLine("DONE:" + command);
}

void sendError(const String &message) {
    sendLine("ERROR:" + message);
}

void startConveyor() {
    digitalWrite(CONVEYOR_STEP_PIN, LOW);
    digitalWrite(CONVEYOR_DIR_PIN, CONVEYOR_FORWARD_POSITIVE ? HIGH :
LOW);
    enableDriver(CONVEYOR_EN_PIN, true);
    delay(10);
    setConveyorRunning(true);
}

void stopConveyor() {
    setConveyorRunning(false);
    digitalWrite(CONVEYOR_STEP_PIN, LOW);
}

```

```

    enableDriver(CONVEYOR_EN_PIN, STEPPER_HOLD_WHEN_STOPPED);
}

void conveyorStepperTask(void *parameter) {
    (void)parameter;
    uint32_t nextStepAt = micros();

    for (;;) {
        const bool running = isConveyorRunning();
        const float speedSps = getConveyorSpeedSps();

        if (!running) {
            digitalWrite(CONVEYOR_STEP_PIN, LOW);
            nextStepAt = micros();
            vTaskDelay(pdMS_TO_TICKS(STEPPER_TASK_IDLE_MS));
            continue;
        }

        const float effectiveSpeedSps = speedSps <
CONVEYOR_MIN_SPEED_SPS ? CONVEYOR_MIN_SPEED_SPS : speedSps;
        const uint32_t intervalUs = static_cast<uint32_t>(1000000.0f /
effectiveSpeedSps);
        const uint32_t now = micros();
        const int32_t remainingUs = static_cast<int32_t>(nextStepAt -
now);

        if (remainingUs > 1000) {
            vTaskDelay(pdMS_TO_TICKS(1));
            continue;
        }
        if (remainingUs > static_cast<int32_t>(A4988_STEP_HIGH_US)) {
            delayMicroseconds(static_cast<uint32_t>(remainingUs) -
A4988_STEP_HIGH_US);
            continue;
        }
    }
}

```

```

        digitalWrite(CONVEYOR_STEP_PIN, HIGH);
        delayMicroseconds(A4988_STEP_HIGH_US);
        digitalWrite(CONVEYOR_STEP_PIN, LOW);
        nextStepAt += intervalUs;

        if (static_cast<int32_t>(micros() - nextStepAt) >
static_cast<int32_t>(intervalUs)) {
            nextStepAt = micros() + intervalUs;
        }
    }
}

bool startConveyorTask() {
    if (conveyorTaskHandle != nullptr) {
        return true;
    }
    const BaseType_t created = xTaskCreatePinnedToCore(
        conveyorStepperTask,
        "conveyor_step",
        STEPPER_TASK_STACK_WORDS,
        nullptr,
        STEPPER_TASK_PRIORITY,
        &conveyorTaskHandle,
        STEPPER_TASK_CORE
    );
    return created == pdPASS;
}

bool validClass(const String &className) {
    return className == "organic" || className == "plastic" ||
className == "metal";
}

int angleForBin(const String &className) {

```

```

    if (className == "organic") {
        return SERVO_ORGANIC_ANGLE;
    }
    if (className == "plastic") {
        return SERVO_PLASTIC_ANGLE;
    }
    if (className == "metal") {
        return SERVO_METAL_ANGLE;
    }
    return SERVO_ORGANIC_ANGLE;
}

bool servoBusy() {
    return state == RunState::SORTING_SERVO || state ==
RunState::BIN_SERVO;
}

bool homeActive() {
    return !servoBusy() && currentBin == "organic";
}

void releaseEntryObjectLock() {
    entryObjectLocked = false;
    entryObjectLockedAt = 0;
    entryInactiveSince = 0;
}

void updateEntryObjectGate() {
    const uint32_t now = millis();

    if (entrySensor.rose()) {
        if (!entryObjectLocked) {
            entryObjectLocked = true;
            entryObjectLockedAt = now;
            entryInactiveSince = 0;
        }
    }
}

```

```

        entryPulsePending = true;
    }
}

if (!entryObjectLocked) {
    return;
}

if (entrySensor.active()) {
    entryInactiveSince = 0;
    return;
}

if (entryInactiveSince == 0) {
    entryInactiveSince = now;
}

if ((now - entryInactiveSince) >= ENTRY_REARM_CLEAR_MS) {
    releaseEntryObjectLock();
}
}

void reportSensors(bool force = false) {
    const uint32_t now = millis();
    const bool entryEvent = entryPulsePending;
    const bool entryActive = entrySensor.active();
    const bool exit = exitSensor.active();
    const bool home = homeActive();
    const bool changed = entryEvent != lastEntry || entryActive !=
lastEntryActive || exit != lastExit || home != lastHome;
    if (!force && !changed && now - lastSensorReportAt <
SENSOR_REPORT_INTERVAL_MS) {
        return;
    }
    lastSensorReportAt = now;
}

```

```

    lastEntry = entryEvent;
    lastEntryActive = entryActive;
    lastExit = exit;
    lastHome = home;
    sendLine(
        String("SENSOR:entry=") + (entryEvent ? "1" : "0") +
        ",entry_event=" + (entryEvent ? "1" : "0") +
        ",entry_active=" + (entryActive ? "1" : "0") +
        ",exit=" + (exit ? "1" : "0") +
        ",home=" + (home ? "1" : "0")
    );
    if (entryPulsePending) {
        entryPulsePending = false;
    }
}

void reportStatus(bool force = false) {
    const uint32_t now = millis();
    if (!force && now - lastStatusAt < STATUS_INTERVAL_MS) {
        return;
    }
    lastStatusAt = now;
    const bool running = isConveyorRunning();
    const float speedSps = getConveyorSpeedSps();
    sendLine(
        String("STATUS:conveyor=") + (running ? "1" : "0") +
        ",current_bin=" + currentBin +
        ",state=" + stateText() +
        ",speed=" + String(speedSps, 0)
    );
}

void clearErrorForCommand() {
    if (state == RunState::ERROR_STATE) {
        state = isConveyorRunning() ? RunState::RUNNING :
RunState::IDLE;

```

```

    }
}

void finishServoMove() {
    const bool wasSorting = state == RunState::SORTING_SERVO;
    currentBin = servoTargetBin;

    if (servoDoneCommand.length()) {
        sendDone(servoDoneCommand);
    }

    if (resumeConveyorAfterServo) {
        startConveyor();
    }

    activeSortClass = "";
    servoDoneCommand = "";
    resumeConveyorAfterServo = false;
    state = isConveyorRunning() ? RunState::RUNNING : RunState::IDLE;
    if (wasSorting) {
        lightDetectedLed(DETECTED_LED_HOLD_MS);
    }
    reportSensors(true);
    reportStatus(true);
}

void startServoMove(const String &className, const String
&doneCommand, bool resumeConveyor, RunState moveState) {
    servoTargetBin = className;
    servoDoneCommand = doneCommand;
    resumeConveyorAfterServo = resumeConveyor;
    activeSortClass = className;
    state = moveState;

    servoMoveDurationMs = SERVO_MOVE_MS;
}

```



```

        servoMoveStartedAt = millis();
        binServo.write(angleForBin(className));
        reportSensors(true);
        reportStatus(true);
    }

void handleSortCommand(const String &className, const String
&original) {
    if (!validClass(className)) {
        sendError("INVALID_SORT_CLASS");
        return;
    }
    if (servoBusy()) {
        sendError("BUSY");
        return;
    }

    clearErrorForCommand();
    sendAck(original);
    lightDetectedLed();
    stopConveyor();
    startServoMove(className, "SORT:" + className, true,
RunState::SORTING_SERVO);
}

void handleBinCommand(const String &className, const String
&original) {
    if (!validClass(className)) {
        sendError("INVALID_BIN_CLASS");
        return;
    }
    if (servoBusy()) {
        sendError("BUSY");
        return;
    }
}

```

```

clearErrorForCommand();
    sendAck(original);
    startServoMove(className, "BIN:" + className, false,
RunState::BIN_SERVO);
}

void handleHomeCommand(const String &original) {
    if (servoBusy()) {
        sendError("BUSY");
        return;
    }

    clearErrorForCommand();
    sendAck(original);
    stopConveyor();
    startServoMove("organic", "HOME", false, RunState::BIN_SERVO);
}

void handleHomeButtonPress() {
    if (servoBusy()) {
        sendError("BUSY:HOME_BUTTON");
        return;
    }

    clearErrorForCommand();
    sendAck("HOME_BUTTON");
    stopConveyor();
    startServoMove("organic", "HOME", false, RunState::BIN_SERVO);
}

void handleCommand(String command) {
    command.trim();
    if (!command.length()) {
        return;
    }
}

```

```

String upper = command;
upper.toUpperCase();

if (upper.startsWith("SORT:")) {
    String className = command.substring(command.indexOf(':') + 1);
    className.trim();
    className.toLowerCase();
    handleSortCommand(className, command);
    return;
}

if (upper.startsWith("BIN:") || upper.startsWith("ROTATE:")) {
    String className = command.substring(command.indexOf(':') + 1);
    className.trim();
    className.toLowerCase();
    handleBinCommand(className, command);
    return;
}

if (upper == "CONVEYOR:START" || upper == "START") {
    if (servoBusy()) {
        sendError("BUSY");
        return;
    }
    clearErrorForCommand();
    sendAck(command);
    startConveyor();
    state = RunState::RUNNING;
    sendDone(command);
    reportStatus(true);
    return;
}

if (upper == "CONVEYOR:STOP" || upper == "STOP") {

```

```

    if (servoBusy()) {
        sendError("BUSY");
        return;
    }
    clearErrorForCommand();
    sendAck(command);
    stopConveyor();
    state = RunState::IDLE;
    sendDone(command);
    reportStatus(true);
    return;
}

if (upper.startsWith("SPEED:")) {
    String value = command.substring(command.indexOf(':') + 1);
    setConveyorSpeedSps(value.toFloat());
    sendAck(command);
    sendDone(String("SPEED:") + String(getConveyorSpeedSps(), 0));
    reportStatus(true);
    return;
}

if (upper == "HOME") {
    handleHomeCommand(command);
    return;
}

if (upper == "RESET_COUNTER") {
    sendAck(command);
    sendDone(command);
    return;
}

if (upper == "STATUS") {
    sendAck(command);
    reportSensors(true);
    reportStatus(true);
}

```

```

        sendDone(command);
        return;
    }

    sendError("INVALID_COMMAND");
}

void readStream(HardwareSerial &stream, String &buffer) {
    while (stream.available() > 0) {
        const char ch = static_cast<char>(stream.read());
        if (ch == '\r') {
            continue;
        }
        if (ch == '\n') {
            handleCommand(buffer);
            buffer = "";
            continue;
        }
        if (buffer.length() < 96) {
            buffer += ch;
        } else {
            buffer = "";
            sendError("COMMAND_TOO_LONG");
        }
    }
}

void updateStateMachine() {
    if (servoBusy() && millis() - servoMoveStartedAt >=
servoMoveDurationMs) {
        finishServoMove();
    }
}

void setup() {
    Serial.begin(UART_BAUDRATE);
}

```

```

    Serial2.begin(UART_BAUDRATE, SERIAL_8N1, UART_RX2_PIN,
UART_TX2_PIN);

pinMode(CONVEYOR_EN_PIN, OUTPUT);
pinMode(CONVEYOR_STEP_PIN, OUTPUT);
pinMode(CONVEYOR_DIR_PIN, OUTPUT);
pinMode(DETECTED_LED_PIN, OUTPUT);
digitalWrite(CONVEYOR_STEP_PIN, LOW);
digitalWrite(CONVEYOR_DIR_PIN, CONVEYOR_FORWARD_POSITIVE ? HIGH :
LOW);
enableDriver(CONVEYOR_EN_PIN, false);
writeDetectedLed(false);

if (!startConveyorTask()) {
    sendError("STEPPER_TASK_CREATE_FAILED");
}

binServo.setPeriodHertz(50);
binServo.attach(BIN_SERVO_PIN, 500, 2500);
binServo.write(SERVO_ORGANIC_ANGLE);
currentBin = "organic";

entrySensor.begin(SENSOR_ENTRY_PIN, SENSOR_ACTIVE_LOW,
INPUT_PULLUP);
exitSensor.begin(SENSOR_EXIT_PIN, SENSOR_ACTIVE_LOW,
INPUT_PULLUP);
homeButton.begin(HOME_BUTTON_PIN, HOME_BUTTON_ACTIVE_LOW, INPUT);

sendLine("READY:esp32_trash_sorter");
reportSensors(true);
reportStatus(true);
}

void loop() {
    entrySensor.update();

```

```

exitSensor.update();
homeButton.update();
updateEntryObjectGate();

readStream(Serial2, uartRx);
readStream(Serial, usbRx);
if (homeButton.rose()) {
    handleHomeButtonPress();
}

updateStateMachine();
updateDetectedLed();
reportSensors(false);
reportStatus(false);
}

```

PHỤ LỤC C – CẤU HÌNH HUẤN LUYỆN (DATA.YAML)

Theo args.yaml của lần huấn luyện mới, file data.yaml được khai báo tại /content/drive/MyDrive/KLTN_YOLO11/trash_detection_2_6/data.yaml; thư mục kết quả huấn luyện mới chỉ lưu args.yaml và các tệp kết quả sau train, không kèm file data.yaml.

```

path: .
train: train/images
val: val/images

nc: 3
names:
  0: metal
  1: organic
  2: plastic

```

PHỤ LỤC D – CẤU TRÚC FIREBASE JSON MẪU

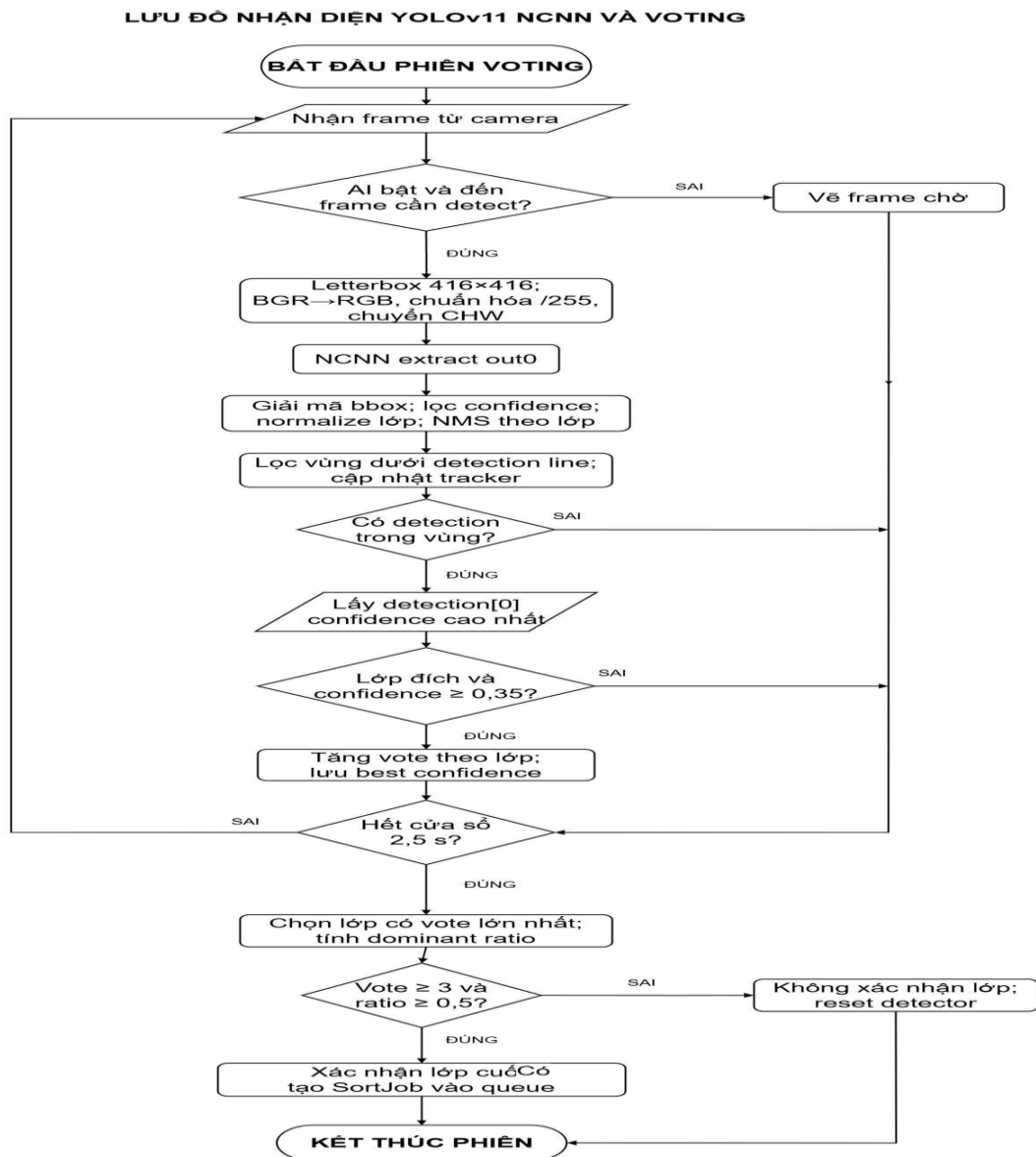
```
{
  "trash_sorting_system": {
    "status": {
      "system_status": "RUNNING",
      "runtime_state": "DETECTOR_IDLE/SORTER_IDLE",
      "camera_status": "ONLINE",
      "uart_status": "CONNECTED",
      "fps": 0.0,
      "queue_depth": 0,
      "current_class": "unknown",
      "confirmed_class": null
    },
    "raspberry_pi": {
      "online": true,
      "last_seen_ms": 0,
      "fps": 0.0,
      "ai_enabled": true
    },
    "esp32": {
      "online": true,
      "last_seen_ms": 0,
      "sensor_entry": false,
      "sensor_exit": false,
      "current_bin": "organic"
    },
    "counters": {
      "organic": 0,
      "metal": 0,
      "plastic": 0,
      "total": 0
    },
    "last_detection": {
      "class_name": "plastic",
      "confidence": 0.92,
      "object_id": 1,
      "source": "entry",
      "votes": {
        "plastic": 6
      }
    }
  }
}
```



```
}  
}  
}
```

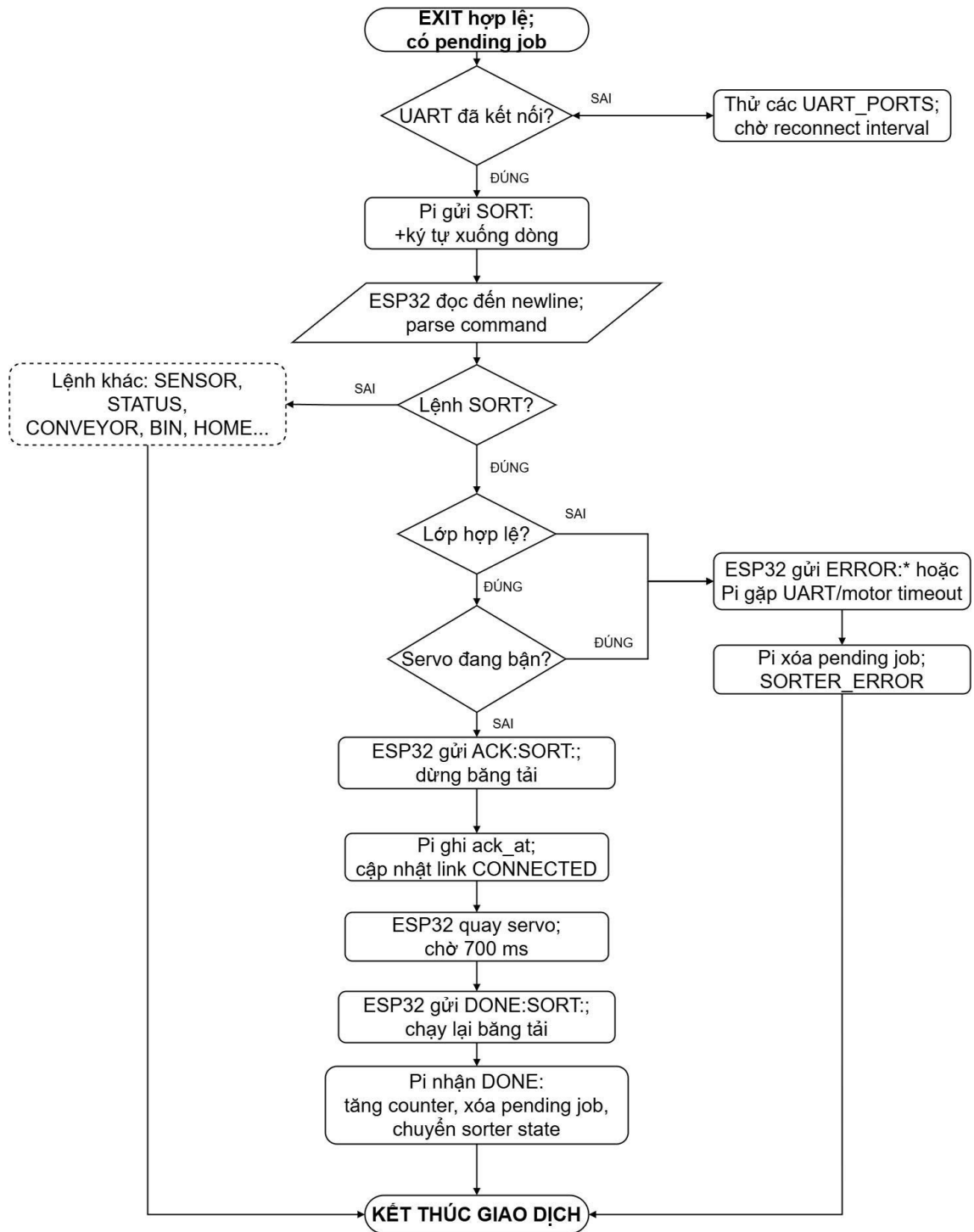
PHỤ LỤC E – CÁC LƯU ĐỒ THUẬT TOÁN CHI TIẾT

Phụ lục này trình bày các lưu đồ thuật toán con được xây dựng nhằm mô tả chi tiết hoạt động của từng khối chức năng trong hệ thống phân loại rác tự động. Các lưu đồ bao gồm quá trình nhận diện và xác nhận lớp bằng cơ chế voting, xử lý tín hiệu cảm biến, giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1, điều khiển cơ cấu servo và đồng bộ dữ liệu lên Firebase Realtime Database.



Hình E. 1 Lưu đồ thuật toán nhận diện vật thể và voting

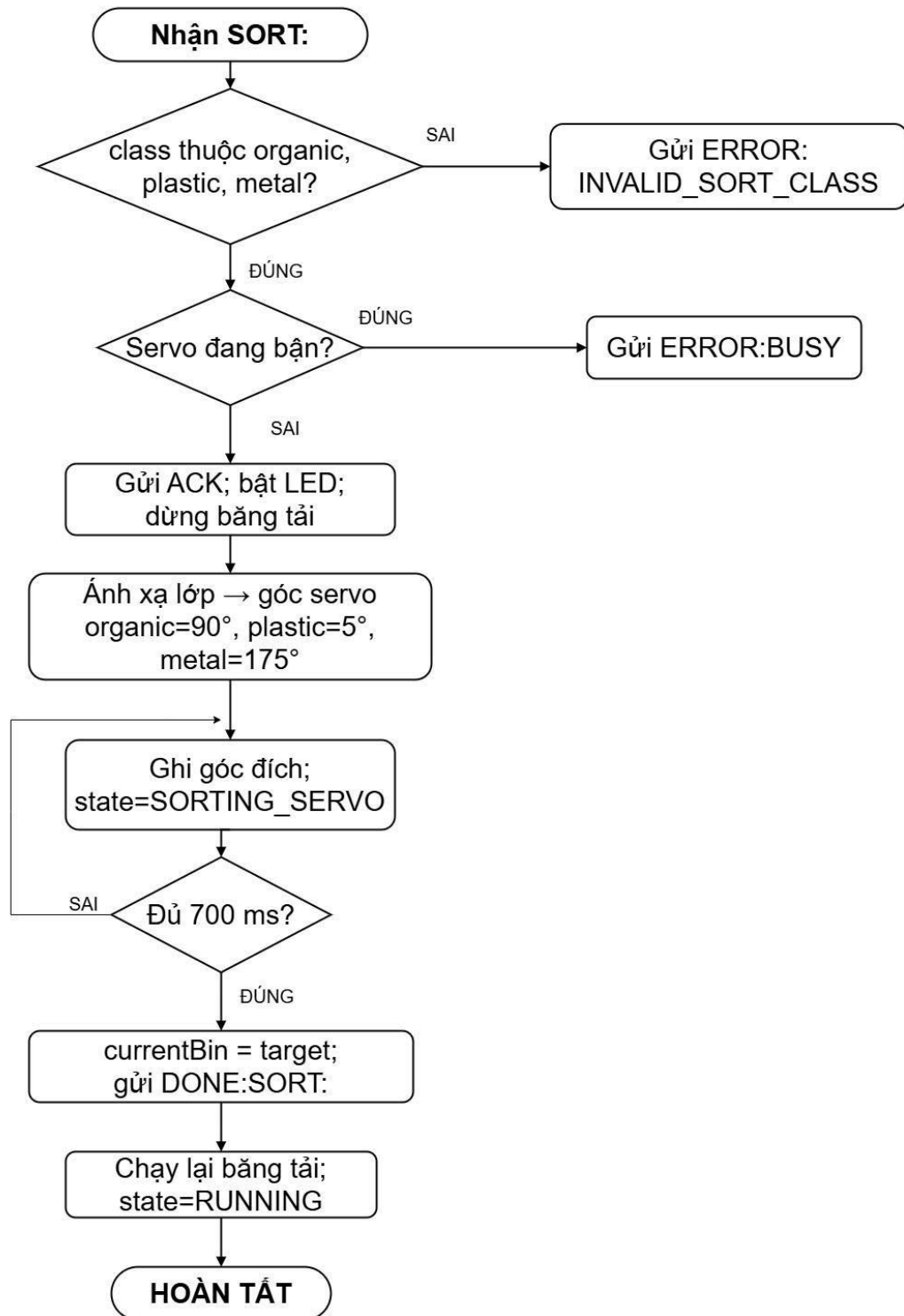
LƯU ĐỒ GIAO TIẾP UART RASPBERRY PI – ESP32



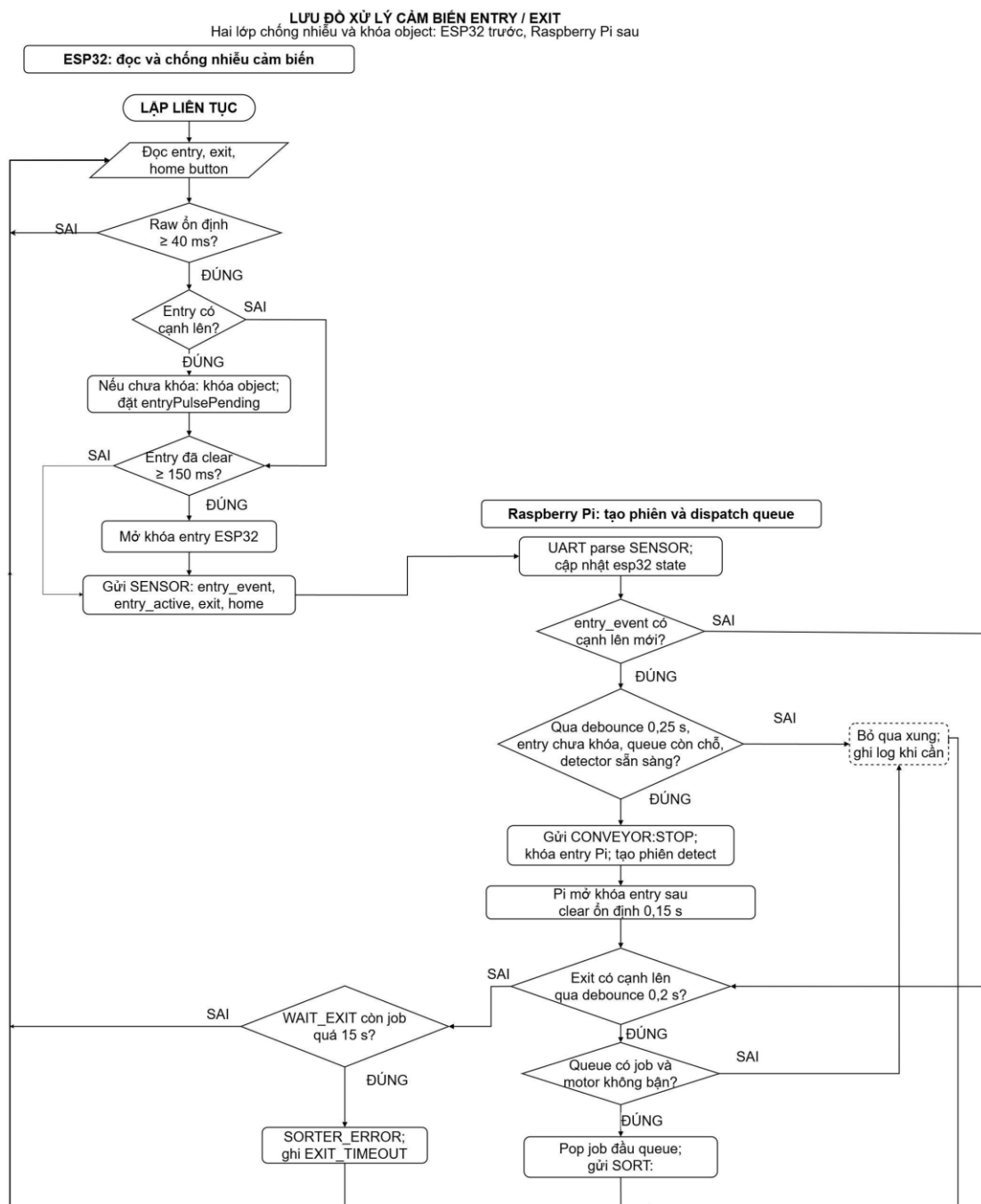
Hình E. 2 Lưu đồ giao tiếp UART giữa Raspberry Pi và ESP32 DevKit V1

LƯU ĐỒ ĐIỀU KHIỂN SERVO PHÂN LOẠI

Firmware ESP32 điều khiển servo 180° theo ba ngăn rác

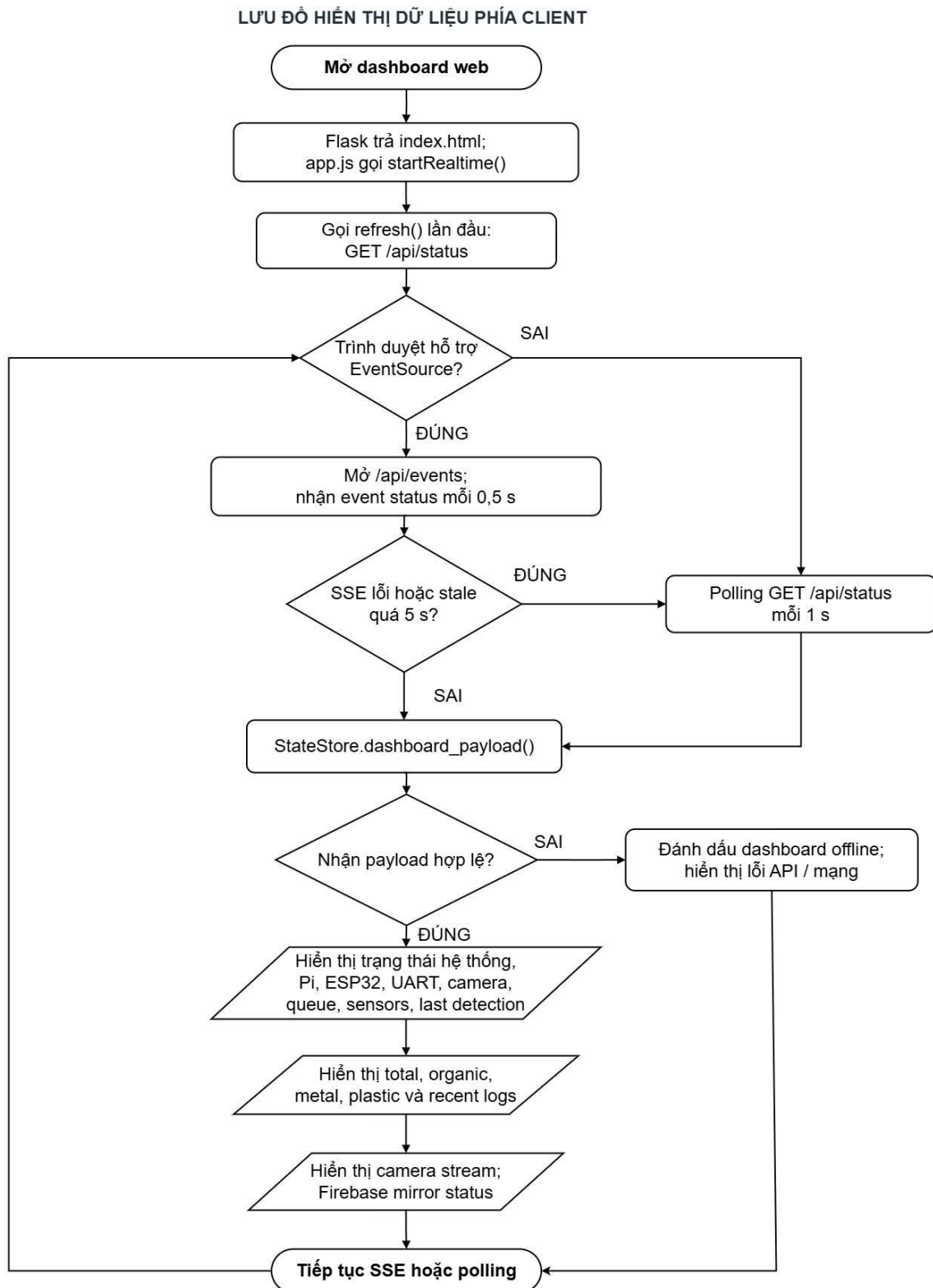


Hình E. 3 Lưu đồ điều khiển servo phân loại

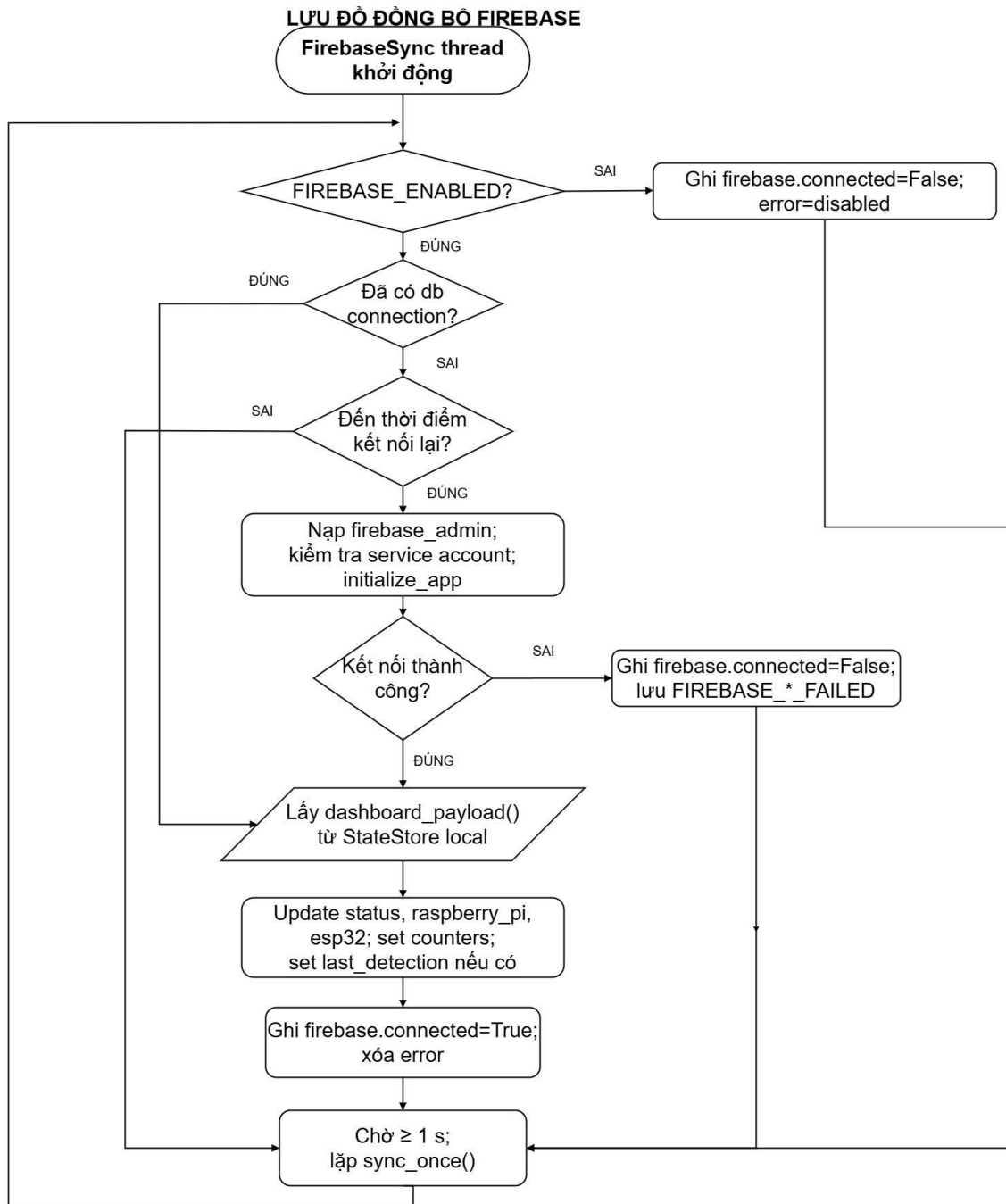


Các cú: entry2sensor, debounceinput, updateEntryObjectState, reportSensor, main.py, _handle_entry_sensor, _handle_exit_sensor, _check_timeout

Hình E. 4 Lưu đồ xử lý cảm biến entry/exit



Hình E. 5 Lưu đồ cập nhật và hiển thị dữ liệu trên dashboard web



Hình E. 6 Lưu đồ đồng bộ Firebase

Minh Le

KLTN Dat v2



KTMT



KLTN2026



Ho Chi Minh City University of Technology and Engineering (HCMUTE)

Thông tin chi tiết về tài liệu

ID bài nộp

trn:oid:::1:3598613011

Ngày nộp

15:59 GMT+7 21 thg 6, 2026

Ngày tải xuống

16:03 GMT+7 21 thg 6, 2026

Tên tệp

KLTN_Dat.pdf

Kích cỡ tệp

5.8 MB

229 Trang

41.640 Từ

206.771 Ký tự

19% Tính tương đồng nói chung

Tổng cộng của tất cả các kết quả trùng khớp, bao gồm cả các nguồn trùng lặp, cho mỗi c...

Nguồn hàng đầu

- 14%  Nguồn Internet
- 6%  Ấn bản
- 12%  Bài tập được nộp (bài của học sinh)

Cờ chỉ báo về tính toàn vẹn

1 cờ chỉ báo về tính toàn vẹn để đánh giá



Ký tự được thay thế

176 ký tự khả nghi trên 63 trang

Các ký tự được hoán đổi bằng những ký tự tương đồng trong một bảng chi

Các thuật toán trong hệ thống của chúng tôi xem xét kỹ lưỡng một tài liệu để phát hiện sự không nhất quán khiến cho tài liệu đó khác với một bài nộp bình thường. Nếu nhận thấy điều gì lạ, chúng tôi sẽ gắn cờ để bạn xem xét.

Cờ không nhất thiết là dấu hiệu chỉ báo có vấn đề. Tuy nhiên, bạn nên chú ý vào phần đó để xem xét thêm.

Nguồn hàng đầu

- 14%  Nguồn Internet
- 6%  Ấn bản
- 12%  Bài tập được nộp (bài của học sinh)

Nguồn hàng đầu

Các nguồn có số lượng kết quả trùng khớp cao nhất trong bài nộp. Các nguồn chồng chéo sẽ không được hiển thị.

1	Bài của học viên	
Ton Duc Thang University		2%
2	Bài của học viên	
Ho Chi Minh University of Technology and Education		1%
3	Internet	
www.ctu.edu.vn		<1%
4	Bài của học viên	
Post and Telecommunications Institute of Technology		<1%
5	Bài của học viên	
Vietnam Maritime University		<1%
6	Internet	
www.slideshare.net		<1%
7	Internet	
5hah33n.github.io		<1%
8	Internet	
tailieu.vn		<1%
9	Bài của học viên	
Thai Nguyen University of Education		<1%
10	Bài của học viên	
University of Northampton		<1%
11	Bài của học viên	
University of Hertfordshire		<1%

12	Bài của học viên	Nha Trang University	<1%
13	Internet	text.123docz.net	<1%
14	Xuất bản	Vietnam National University of Forestry	<1%
15	Bài của học viên	Thai Nguyen University	<1%
16	Bài của học viên	Multiversity	<1%
17	Internet	www.tailieudaihoc.com	<1%
18	Xuất bản	Phẩm Phạm Như, Phạm Văn Điều, Nguyễn Minh Nhật. "Nghiên cứu và ứng dụng..."	<1%
19	Xuất bản	Ton Duc Thang University	<1%
20	Bài của học viên	Van Lang University	<1%
21	Internet	text.xemtailieu.net	<1%
22	Bài của học viên	Vels Institute of Science, Technology & Advanced Studies (VISTAS)	<1%
23	Bài của học viên	National Economics University	<1%
24	Bài của học viên	Swinburne University of Technology	<1%
25	Xuất bản	Magdić, Valentin. "Izgradnja višeeagentnog sustava za analizu tržišta kriptovaluta..."	<1%

26	Bài của học viên	AUT University	<1%
27	Bài của học viên	University of San Diego	<1%
28	Bài của học viên	Durban University of Technology	<1%
29	Bài của học viên	Open University	<1%
30	Bài của học viên	Birkbeck College	<1%
31	Bài của học viên	Ho Chi Minh City Open University	<1%
32	Bài của học viên	Universidad Europea de Madrid	<1%
33	Internet	thelinuxcode.com	<1%
34	Xuất bản	Phan Phương Lan, Huỳnh Hữu Hưng, Huỳnh Xuân Hiệp. "HỆ TƯ VẤN DỰA TRÊN Đ...	<1%
35	Bài của học viên	University of West London	<1%
36	Internet	docs.llamaindex.ai	<1%
37	Internet	www.zbook.vn	<1%
38	Xuất bản	VNUA	<1%
39	Internet	git.liberatedsystems.co.uk	<1%

40	Internet	luanvan.co	<1%
41	Internet	123docz.net	<1%
42	Bài của học viên	Loughborough University	<1%
43	Internet	hvtc.edu.vn	<1%
44	Internet	tweedy.flanigan.us	<1%
45	Internet	www.thuvientailieu.vn	<1%
46	Internet	123doc.org	<1%
47	Internet	telkomnika.uad.ac.id	<1%
48	Internet	themes.trac.wordpress.org	<1%
49	Bài của học viên	Instituto Superior de Artes, Ciencias y Comunicación IACC	<1%
50	Internet	ichi.pro	<1%
51	Bài của học viên	Tilburg University	<1%
52	Internet	docs.ultralytics.com	<1%
53	Internet	elektrika.utm.my	<1%

54	Xuất bản	Hanoi University	<1%
55	Bài của học viên	Manhattan College	<1%
56	Internet	doc.edu.vn	<1%
57	Bài của học viên	Aston University	<1%
58	Bài của học viên	Instituto de Empress S.L.	<1%
59	Bài của học viên	Illinois Institute of Technology	<1%
60	Internet	git.mig5.net	<1%
61	Internet	www.timtailieu.vn	<1%
62	Internet	en.ueh.edu.vn	<1%
63	Bài của học viên	City University	<1%
64	Bài của học viên	Colorado Technical University Online	<1%
65	Internet	codeclimate.com	<1%
66	Internet	anchor.fm	<1%
67	Internet	git.rose-ash.com	<1%

68	Internet	jmst.vimaru.edu.vn	<1%
69	Internet	portal.ptit.edu.vn	<1%
70	Internet	www.mdpi.com	<1%
71	Bài của học viên	Industrial University of Ho Chi Minh City	<1%
72	Internet	www.instructables.com	<1%
73	Xuất bản	Mạc Đình Hiếu, Lê Ngọc Anh, Trần Đức Hùng, Trịnh Văn Hùng, Ngô Minh Phước, ...	<1%
74	Bài của học viên	University of Huddersfield	<1%
75	Bài của học viên	University of Technology Bahrain	<1%
76	Internet	iieta.org	<1%
77	Internet	repo-portal.ito.gov.ir	<1%
78	Bài của học viên	London Metropolitan University	<1%
79	Internet	luanvan.net.vn	<1%
80	Bài của học viên	Coleg Sir Gar	<1%
81	Bài của học viên	University of Derby	<1%

82	Bài của học viên	University of Ulster	<1%
83	Internet	cesti.gov.vn	<1%
84	Internet	dost.hochiminhcity.gov.vn	<1%
85	Bài của học viên	Ho Chi Minh City University of Transport	<1%
86	Bài của học viên	Universiteit Utrecht	<1%
87	Bài của học viên	University of Hong Kong	<1%
88	Xuất bản	Văn Cô Như. "Nghiên cứu thiết kế hệ thống cảnh báo tự động tại đường ngang đ...	<1%
89	Internet	fsb.edu.vn	<1%
90	Internet	techie.vn	<1%
91	Internet	truyenhinhcapsongthu.net	<1%
92	Internet	www.phoronix.com	<1%
93	Xuất bản	Hà Mạnh Hùng, Đặng Việt Hưng. "Phương pháp đa nhiệm xác định hư hỏng tron...	<1%
94	Bài của học viên	University of Lancaster	<1%
95	Bài của học viên	University of Wollongong	<1%

96	Bài của học viên	University of Wolverhampton	<1%
97	Internet	www.inettutor.com	<1%
98	Xuất bản	Mạc Đình Hiếu, Tống Văn Vạn, Bùi Trọng Tùng, Trần Quang Đức, Nguyễn Linh Gia...	<1%
99	Internet	api.python.langchain.com	<1%
100	Internet	docs.agno.com	<1%
101	Internet	dulieu.tailieuhoctap.vn	<1%
102	Internet	elar.khmnu.edu.ua	<1%
103	Internet	git.efi.th-nuernberg.de	<1%
104	Internet	phaply.net.vn	<1%
105	Xuất bản	Dmytro Kozhevín. "Building Serverless Robotics with AWS, AI, and ROS 2", Springe...	<1%
106	Bài của học viên	Institute of Technology, Sligo	<1%
107	Internet	link.sov5.org	<1%
108	Bài của học viên	Bogazici University	<1%
109	Bài của học viên	Monash University	<1%

110	Internet	docs.espressif.com	<1%
111	Internet	icdayroi.com	<1%
112	Internet	jst.tnu.edu.vn	<1%
113	Internet	openi.pcl.ac.cn	<1%
114	Bài của học viên	Asia Pacific University College of Technology and Innovation (UCTI)	<1%
115	Bài của học viên	University of Birmingham	<1%
116	Internet	nguoingho.vn	<1%
117	Internet	www.ijert.org	<1%
118	Bài của học viên	Phenikaa University	<1%
119	Bài của học viên	SETUCW	<1%
120	Bài của học viên	Universidad TecMilenio	<1%
121	Bài của học viên	University of Economics & Law	<1%
122	Bài của học viên	University of the Immaculate Conception	<1%
123	Bài của học viên	Vietnam Aviation Academy	<1%

124	Internet	asyncfunc.mintlify.app	<1%
125	Internet	doan.edu.vn	<1%
126	Internet	docs.flex.ai	<1%
127	Internet	git.linuxiarz.pl	<1%
128	Internet	multitechsystems.github.io	<1%
129	Internet	www.davicogroup.com.vn	<1%
130	Internet	www.sendowl.com	<1%
131	Bài của học viên	National University of Ireland, Maynooth	<1%
132	Bài của học viên	Universidad Nacional Amazonica de Madre de Dios	<1%
133	Internet	dev.to	<1%
134	Internet	docs.encord.com	<1%
135	Internet	jmst.mod.gov.vn	<1%
136	Internet	juejin.cn	<1%
137	Internet	m.trandaiquang.org	<1%

138	Internet	pt.slideshare.net	<1%
139	Internet	text.123doc.org	<1%
140	Internet	vbcsd.vn	<1%
141	Bài của học viên	Universidad de Málaga	<1%
142	Bài của học viên	University of Strathclyde	<1%
143	Internet	cncvina.com.vn	<1%
144	Internet	hannacateringpuncak.net	<1%
145	Internet	hconnect.hanyang.ac.kr	<1%
146	Internet	imi.edu.vn	<1%
147	Internet	ispacedanang.edu.vn	<1%
148	Internet	medium.com	<1%
149	Internet	mysummit.school	<1%
150	Internet	nkvietnam.com	<1%
151	Internet	raw.githubusercontent.com	<1%

152	Internet	tailieuhoc.net	<1%
153	Internet	techphone.vn	<1%
154	Internet	thesai.org	<1%
155	Internet	thuvienso.hcmute.edu.vn	<1%
156	Internet	toc.123doc.org	<1%
157	Internet	tuyhoaship.com	<1%
158	Internet	urenco.com.vn	<1%
159	Internet	vanbanluat.com	<1%
160	Internet	vmpec.com.vn	<1%
161	Internet	wpcom-i18n.svn.automattic.com	<1%
162	Internet	www.hackster.io	<1%
163	Internet	www.journalsofznu.zp.ua	<1%
164	Xuất bản	Đào Vũ Hiệp, Mạc Đình Hiếu, Tống Việt Hùng, Vũ Văn Phán, Hà Quốc Trung. "IVA-...	<1%
165	Internet	2isao.com	<1%

166	Bài của học viên	De La Salle University - Manila	<1%
167	Xuất bản	Trần Thanh Trâm, Trần Thanh Phước, Nguyễn Thị Anh Thư, Nguyễn Thế Hữu, Văn...	<1%
168	Internet	blog.vietnamlab.vn	<1%
169	Internet	blogcongnghe.com.vn	<1%
170	Internet	bogiaeduc.edu.vn	<1%
171	Internet	compamal.com	<1%
172	Internet	doanluanvan.org	<1%
173	Internet	docs.nvidia.com	<1%
174	Internet	es.slideshare.net	<1%
175	Internet	file.allitebooks.com	<1%
176	Internet	filegi.com	<1%
177	Internet	giaodich24.vn	<1%
178	Internet	ksp.mff.cuni.cz	<1%
179	Internet	laptrinhx.com	<1%

180	Internet	nuocmamgiaochau.com.vn	<1%
181	Internet	old-list-archives.xen.org	<1%
182	Internet	pastes.io	<1%
183	Internet	trungviethung.com	<1%
184	Internet	vie.legatechnics.com	<1%
185	Internet	vieclam.vietnamnet.vn	<1%
186	Internet	vinabluetech.vn	<1%
187	Internet	www.blackcat1968.com	<1%
188	Internet	www.conganthanhhoa.gov.vn	<1%
189	Internet	www.coursehero.com	<1%
190	Internet	www.otaviomiranda.com.br	<1%
191	Internet	www.phamthihoai.org	<1%
192	Internet	24htech.vn	<1%
193	Xuất bản	Duong Phi Long, Nguyen Thanh Tuong Vi, Vo Trung Hieu, Vo Duy Nguyen, Nguy...	<1%

194	Xuất bản	Lafreniere, Samuel S.. "Design and Development of an MRI-Compatible Robot to ...	<1%
195	Bài của học viên	RDI Distance Learning	<1%
196	Internet	asia-corp.vn	<1%
197	Internet	binref.github.io	<1%
198	Internet	blog.csdn.net	<1%
199	Internet	cautrismoman.weebly.com	<1%
200	Internet	daotao.sict.udn.vn	<1%
201	Internet	data.uet.vnu.edu.vn:8080	<1%
202	Internet	de.leapcell.io	<1%
203	Internet	docs.lazyllm.ai	<1%
204	Internet	documents.mx	<1%
205	Internet	electronilab.co	<1%
206	Internet	eplusi.net	<1%
207	Internet	eprints.untirta.ac.id	<1%

208	Internet	forum.developer.parrot.com	<1%
209	Internet	freedomjunkshun.com	<1%
210	Internet	giadinh.net.vn	<1%
211	Internet	gitext.gfz-potsdam.de	<1%
212	Internet	grenews.net	<1%
213	Internet	hoinongdanbacgiang.org.vn	<1%
214	Internet	invivoo.com	<1%
215	Internet	lghvac.vn	<1%
216	Internet	lume.ufrgs.br	<1%
217	Internet	mail.zope.dev	<1%
218	Internet	netsystem.vn	<1%
219	Internet	ngulonluc.wordpress.com	<1%
220	Internet	osm.etsi.org	<1%
221	Internet	proceeding.vap.ac.vn	<1%

222	Internet	programtalk.com	<1%
223	Internet	repository.polman-babel.ac.id	<1%
224	Internet	search-labs.elastic.co	<1%
225	Internet	seb60bdf8ae2d5bbd.jimcontent.com	<1%
226	Internet	sites.google.com	<1%
227	Internet	sj.ctu.edu.vn	<1%
228	Internet	tapchi.ftu.edu.vn	<1%
229	Internet	tapchitaichinh.vn	<1%
230	Internet	thuvien24.com	<1%
231	Internet	toc.123docz.net	<1%
232	Internet	v1.overleaf.com	<1%
233	Internet	vimentor.com	<1%
234	Internet	vjol.info.vn	<1%
235	Internet	vualuoi.com	<1%

236	Internet	www.bncustoms.gov.vn	<1%
237	Internet	www.dienmayxanh.com	<1%
238	Internet	www.grad.hcmut.edu.vn	<1%
239	Internet	www.nand.it	<1%
240	Internet	www.niengrang.vn	<1%
241	Internet	www.thichcongnghe.com	<1%
242	Internet	www.tinhdoancamau.com.vn	<1%
243	Internet	www.wifilinksys.com	<1%
244	Xuất bản	Ho Chi Minh National Academy of Politics	<1%
245	Xuất bản	Phan Anh Cang, Nguyễn Thanh Hoàng, Trần Hồ Đạt, Nguyễn Văn Hiếu, Phan Thư...	<1%
246	Xuất bản	Trần Giang Sơn, Herbreteau Tom, Nguyễn Chí Cường, Nghiêm Thị Phương, Đỗ Oa...	<1%
247	Xuất bản	Hanoi National University of Education	<1%
248	Xuất bản	Keqian Zhu, Jingfei Jiang. "Chapter 3 Research on Parallel Acceleration for Deep L...	<1%
249	Xuất bản	Marić, Petar. "Hibridna Softverska Arhitektura Kao Podrška Primeni Harmonijski ...	<1%

250

Xuất bản

Nguyễn Minh Hải, Lê Thị Vĩnh Thanh, Văn Thế Thành, Trần Văn Lãng. "TRA CỨU Ả... <1%

251

Xuất bản

Đỗ Văn Tiến, Lâm Quang Vũ, Phan Lê Sang, Ngô Đức Thành, Lê Đình Duy, Dương ... <1%